

Incrementally Verifiable Computation for NP from Standard Assumptions

Pratish Datta*
NTT Research

Abhishek Jain†
NTT Research and JHU

Zhengzhong Jin‡
Northeastern

Alexis Korb§
UCLA

Surya Mathialagan¶
MIT

Amit Sahai||
UCLA

Abstract

Incrementally verifiable computation (IVC) [Valiant, TCC’08] allows one to iteratively prove that a configuration x_0 reaches another configuration x_T after repeated applications of a (possibly non-deterministic) transition function $\mathcal{M}$. The key requirement is that the size of the proof and the time to update the proof is sublinear in the number of steps T . IVC has numerous applications, notably including proving correctness of virtual machine executions in blockchains.

Currently, IVC for NP is only known to exist in non-standard idealized models, or based on knowledge assumptions. No constructions are known from standard assumptions, or *even in the random oracle model*. Furthermore, as observed in prior works, since IVC for NP implies adaptive succinct non-interactive arguments for NP, the work of Gentry-Wichs [STOC’11] seemingly poses barriers to constructing IVC for NP from falsifiable assumptions.

In this work, we observe that the Gentry-Wichs barrier can be overcome for IVC for NP. We show the following two results:

- Assuming subexponential $i\mathcal{O}$ and LWE (or bilinear maps), we construct IVC for all NP with proof size $\text{poly}(|x_i|, \log T)$.
- Assuming subexponential $i\mathcal{O}$ and injective PRGs, we construct IVC for *trapdoor IVC languages* where the proof-size is $\text{poly}(\log T)$. Informally, an IVC language has a trapdoor if there *exists* a (not necessarily easy to find) polynomial-sized circuit that determines if a configuration x_i is reachable from x_0 in i steps.

*pratish.datta@ntt-research.com. ORCID: 0000-0002-3938-7594.

†abhishek.jain@ntt-research.com. ORCID: 0000-0002-3572-7643.

‡zh.jin@northeastern.edu. ORCID: 0000-0002-2233-3768

§alexiskorb@cs.ucla.edu. ORCID: 0000-0001-6888-5296.

¶smathi@mit.edu. ORCID: 0000-0003-4904-3637

||sahai@cs.ucla.edu. ORCID: 0000-0003-2216-9600.

Contents

1	Introduction	3
1.1	Our Results	4
1.2	Organization of the paper	8
2	Technical Overview	8
2.1	Prior Work	8
2.2	IVC for NP	10
2.3	Fully succinct IVC for Trapdoor Languages	13
3	Preliminaries	23
3.1	Incrementally Verifiable Computation (IVC)	23
3.2	Succinct Non-Interactive Arguments (SNARGs) and Batch Arguments (BARGs) . .	25
3.3	Non-interactive Zero-Knowledge (NIZK)	27
3.4	Public-Key Encryption (PKE)	28
3.5	Indistinguishability Obfuscation	29
3.6	Puncturable Pseudorandom Functions	29
4	IVC for NP	30
4.1	Parameters	31
4.2	Construction	32
4.3	Correctness and Efficiency	35
4.4	Soundness	35
4.5	Zero Knowledge	44
5	Fully Succinct IVC for Trapdoor Languages	46
5.1	Trapdoor Language Definitions	47
5.2	Parameters	48
5.3	Construction	49
5.4	Correctness, Efficiency, and Zero-Knowledge	50
5.5	Soundness	50
6	References	69

1 Introduction

Suppose humanity faces a difficult mathematical problem x_0 . A mathematician, through hard work, reduces x_0 to a simpler problem x_1 with a proof w_1 . Later, another mathematician reduces x_1 to x_2 via a long proof w_2 . Different mathematicians may explore various paths, possibly reducing x_1 to other problems like $x'_2 \neq x_2$. Eventually, after many steps (and perhaps many generations), we obtain a long chain of reductions: $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$, where x_T is an easily verifiable tautology.

To verify the correctness of this long sequence, one traditionally needs to go through all intermediate proofs, which can be time-consuming. Now, imagine that at each step i , the mathematician attaches a certificate π_i to demonstrate that there exists a valid proof from x_0 to x_i . Moreover, this certificate π_i can be generated efficiently from the proof w_i of the reduction $x_{i-1} \rightarrow x_i$ and the previous certificate π_{i-1} . If this process can be done incrementally, then the final certificate at step T could be verified in time much less than T (ideally, in $\text{poly}(\log T)$ time).

Incrementally Verifiable Computation. This concept was introduced by Valiant [Val08] as *incrementally verifiable computation* (IVC). IVC extends the notion of succinct non-interactive arguments (SNARGs) [Mic93]. Recall that SNARGs for NP enable a prover to compress an NP witness into a short certificate that can be efficiently verified. IVC extends SNARGs by allowing the prover to incrementally update the SNARG proof as the computation progresses. As motivated in our earlier example, the setup is as follows: there is a starting configuration x_0 , and a machine M (possibly nondeterministic) that can be applied to x_0 to reach a new state. In an IVC scheme, as in the SNARG setting, a user should be able to produce a proof π that a state x_0 reaches a state x_ℓ in ℓ non-deterministic steps, if the user is aware of the non-deterministic choices that were made to reach x_ℓ . Additionally, we want to be able to *update* the proof π to prove that x_0 reaches some state $x_{\ell+1} = M(x_\ell, w_\ell)$ only knowing $x_0, x_\ell, x_{\ell+1}, \pi$, and w_ℓ ; that is, without knowing the earlier non-deterministic choices made. Crucially, we want that neither update time, verification time, nor proof size scale with ℓ . The case where M is a deterministic procedure that does not take an auxiliary input is called IVC for P, or IVC for deterministic computations.

IVC and its generalizations have enabled a plethora of applications, including enforcing language semantics [CTV13], authenticating images [NT16], blockchains [BMRS20, KS22, CCDW20] and many more. Some of these applications additionally require the proof to satisfy the zero knowledge property [GMR85], namely, that the proof reveals nothing beyond the validity of the statement.

How to construct IVC? Valiant [Val08] provided a construction of IVC for NP via Micali’s CS proofs, and argued its security in a non-standard model that assumes both (1) a random oracle, and seemingly contradictorily, (2) a succinct circuit that computes the oracle. To date, however, no constructions of IVC for NP are known *even in the random oracle model*. In fact, recent works [HAN23, BCG24] have presented barriers to achieving IVC for NP in the random oracle model. This is in stark contrast to the setting of SNARGs, where Micali [Mic93] constructed a SNARG for NP in the random oracle model.

In the recent years, several works have investigated alternative templates for realizing IVC. This includes IVC from SNARKs (succinct non-interactive argument of knowledge) [BCCT13, BCTV14, COS20], folding schemes [KST22], from accumulation and aggregation [BCMS20, BCL⁺21, BDFG21], and arithmetized random oracles [CCG⁺23]. However, all of these constructions involve either non-standard idealized models or non-standard knowledge assumptions. In fact, SNARKs are known to be impossible to achieve via black-box reductions to standard assumptions [CGKS23].

In light of the above, recent works [NPR19, KPY19] have also investigated the weaker problem

of IVC for P. In particular, Paneth and Pass [PP22] and Devadas et al. [DGKV22] gave the first constructions of IVC for P from standard assumptions such as Learning with Errors (LWE).

In general, IVC for NP is more desirable in practice. For example, in the aforementioned mathematical proof scenario, scientific research is a non-deterministic process rather than a deterministic one. Moreover, when IVC is used on the blockchain to verify virtual machine execution, the virtual machine can take additional transactions as input in each step of the execution, making it naturally a non-deterministic computation [KST22].

This leads to our central question:

*Can we build incrementally verifiable computation
for NP from standard assumptions?*

Despite its significance, IVC for NP from standard assumptions has remained elusive. The difficulty lies in the fundamental differences between P and NP. IVC for NP allows the prover to certify multiple computation paths, as they can choose the NP witness, whereas IVC for P has a unique computation path. This flexibility in NP computations complicates the construction of IVC for NP but also enables a wider range of applications.

A similar difficulty arises in the context of SNARGs. While we know how to build SNARGs for P from a variety of standard assumptions [CJJ22, KVZ21, WW22, CGJ⁺23], constructing SNARGs for NP has so far required the stronger machinery of indistinguishability obfuscation [SW14, WW24a, WW24c] or witness encryption [JKLM25].

On the Gentry-Wichs Barrier. In their seminal work [GW11], Gentry and Wichs show that SNARGs for NP cannot be proven adaptively sound via black-box reductions to falsifiable assumptions. The research community has viewed the impossibility result of [GW11] as presenting a formidable barrier to achieving IVC for NP computations. For instance, the 2023 work of [HAN23] which rules out IVC for NP in the random oracle model (under some additional assumptions) argues: “Since in particular the impossibility of Gentry-Wichs [GW11] for (adaptively sound) zk-SNARGs applies, one could not hope to construct non-deterministic IVC from “falsifiable assumptions”. (See also similar language in [BCMS20, CGSY23].) Indeed, note that even if we were to aim only for non-adaptively sound IVC for NP, where the configurations x_0 and x_T are chosen *before* the common reference string is published, this does not avoid the problem that the intermediate states $x_1, \dots, x_{T-1}$ can still be chosen *adaptively* after the CRS is published. As a result, some degree of adaptive soundness seems inherent in IVC for NP.

Recent works, however, have demonstrated that the Gentry-Wichs barrier can be overcome to construct adaptively sound SNARGs for NP from $i\mathcal{O}$ and OWFs [WW24a, MPV24, WW24c]. Motivated by these developments, in this work, we revisit the goal of realizing IVC for NP from falsifiable assumptions.

1.1 Our Results

We give two new constructions of IVC from standard assumptions. We first construct IVC for all NP with proof size that grows poly-logarithmically in the number of steps, but polynomially in the size of a configuration. This matches the efficiency of known constructions of IVC for P [PP22, DGKV22]. We next construct fully succinct IVC (i.e., with proof size independent of the configuration size) for trapdoor IVC languages – a generalization of a previously defined notion of trapdoor languages [CGKS23] (see more below).

1.1.1 Result I: IVC for NP.

We construct a zero-knowledge IVC scheme for NP in the common reference string model from standard assumptions. Our proof size, update time and verification time only grow polylogarithmically in the number of steps T .

Theorem 1.1 (Informal). *Consider an IVC language corresponding to a polynomial sized circuit $\mathcal{M}$ which takes as input a configuration of length $n := n(\lambda)$ and witness of length $m := m(\lambda)$, and outputs another configuration of length n . Assume the existence of subexponential versions of the following ingredients:*

- (Non-adaptive) succinct non-interactive arguments (SNARG) for NP.
- Rate-1 somewhere extractable batch arguments for NP.
- Statistically sound non-interactive zero knowledge proofs (NIZK) for NP.
- Perfectly correct public key encryption

Then, there exists a zero-knowledge IVC scheme for NP with

- proof size of $\text{poly}(\lambda, n, \log T)$.
- crs size $\text{poly}(\lambda, |\mathcal{M}|, \log T)$.
- prover and verifier runtime $\text{poly}(\lambda, |\mathcal{M}|, \log T)$.

where T is an upper bound on the number of time-steps of the IVC scheme.

Statistically sound NIZKs can be built from standard assumptions such as LWE [PS19] and bilinear maps [GOS06]. Moreover, rate-1 somewhere extractable BARGs can be instantiated from LWE [DGKV22], or from bilinear maps, assuming a combination of [BDSZ24, WW22]. Finally, (non-adaptive) SNARGs for NP can be instantiated from $i\mathcal{O}$ and one-way functions [SW14] with a non-transparent setup, or from subexponential $i\mathcal{O}$ and subexponential LWE with a transparent set-up [JKLM25].

Corollary 1.2. *Assuming subexponential $i\mathcal{O}$, and either subexponential LWE or subexponential bilinear maps, there exists an IVC scheme for NP with proof size $\text{poly}(\lambda, n, \log T)$.*

While $i\mathcal{O}$ is not a falsifiable assumption, we know of constructions of $i\mathcal{O}$ from standard, falsifiable assumptions [JLS21, JLS22]. We thus achieve IVC for NP from standard, falsifiable assumptions.

To obtain the above result, we build upon the approach of [PP22, DGKV22], who show how to build IVC for P via rate-1 BARGs. In fact, our construction is nearly identical to their schemes. Furthermore, the proof size of our scheme matches the proof size achieved by their constructions.

1.1.2 Result II: Fully-succinct IVC for Trapdoor Languages

While our first construction already achieves a non-trivial proof size that only grows polylogarithmically in the number of hops T , the proof size still grows polynomially with n - the size of each intermediate configuration. This begs the question:

Is it possible to construct IVC for NP with proof size $\text{poly}(\lambda, \log T)$?

We call IVC schemes achieving this level of succinctness *fully-succinct*.

We show a fully-succinct IVC scheme for a subclass of NP which we refer to as *trapdoor-IVC languages*. We say an IVC language defined by a machine $\mathcal{M}$ has a “trapdoor circuit” if there exists a circuit $\text{Trap}_{\mathcal{M}}$ such that

$$\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 1 \iff \exists(w_1, \dots, w_i) \text{ such that } x_0 \xrightarrow{w_1} x_1 \xrightarrow{w_2} \dots \xrightarrow{w_i} x_i.$$

where $x_i \xrightarrow{w_{i+1}} x_{i+1}$ denotes that $M(x_i, w_{i+1}) = x_{i+1}$. In other words, $\text{Trap}_{\mathcal{M}}$ is some non-uniform circuit that can *decide* the IVC language corresponding to the machine $\mathcal{M}$. This is the natural extension of trapdoor languages as defined in [CGKS23] to the IVC setting, since the IVC language make claims about both intermediate and final states.

We stress that a trapdoor circuit IVC language only requires that the trapdoor circuit *exists*, not that it is known to any party. Given this definition, we can now state our main theorem.

Theorem 1.3 (Informal). *Consider a trapdoor circuit IVC language corresponding to a polynomial-sized circuit $\mathcal{M}$ which takes as input a configuration of length $n := n(\lambda)$ and witness of length $m := m(\lambda)$, and outputs another configuration of length n , and which has a polynomial-sized trapdoor circuit of size $\gamma := \gamma(\lambda)$.*

Assuming subexponential $i\mathcal{O}$ and subexponential injective PRGs, there exists a zero-knowledge IVC scheme for this language where:

- *The proof size is $\text{poly}(\lambda, \log T)$.*
- *The crs size is $\text{poly}(\lambda, \gamma, |\mathcal{M}|, \log T)$.*
- *The prover and verifier runtime is $\text{poly}(\lambda, \gamma, |\mathcal{M}|, \log T)$.*

where T is an upper bound on the number of time-steps of the IVC scheme. We emphasize that it suffices to know an upper bound γ on the size of the trapdoor circuit; we do not need to know the actual trapdoor circuit to construct the scheme.

We note that the assumptions underlying our construction are incomparable to the assumptions in Theorem 1.1, and the works of [PP22, DGKV22]. In particular, somewhere-extractable BARGs are known to imply private information retrieval [BKP⁺24], and this cannot be built from $i\mathcal{O}$ and OWFs [AS15].

On Trapdoor IVC Languages. At first glance, this theorem statement might seem superfluous - if the language admits a trapdoor that allows one to decide it efficiently, why does one even need a SNARG or IVC for this language? To answer this, we stress that $\text{Trap}_{\mathcal{M}}$ only *appears in the security proof*. In other words, the person who sets up the IVC crs or produces any IVC proof *does not* need to know this trapdoor explicitly, but only needs to know that such a trapdoor *exists*. In fact, due to this reason, the above theorem also yields a candidate construction of fully succinct IVC for any NP language for which we might expect such a trapdoor function to exist.

We now give some examples of trapdoor IVC languages. Our first example involves a search languages with a unique witness. In fact, the example below can be generalized to all such search languages.

Example 1. Suppose Alice wants to determine the prime factors of some number N via a brute-force search. She delegates the following computation

$$0 \xrightarrow{\mathcal{M}_N} a \xrightarrow{\mathcal{M}_N} 2a \dots \xrightarrow{\mathcal{M}_N} ta.$$

where on each iteration i she uses a (possibly non-deterministic) computation $\mathcal{M}_N$ to determine if there are any factors of N in the range $[ia, (i+1)a]$. This lets her incrementally eliminate ranges that do not contain factors of N .

At each step, Alice (or a prover) attaches a succinct proof that the range $[0, ia]$ contains no factors of N . Using IVC, this proof can be updated incrementally, and we can maintain a short certificate to prove that a range has no factors without recomputing the entire history.

This computation does indeed have a trapdoor - the factorization of N . Indeed, given a factorization τ of N , one can efficiently decide if a given interval $[0, ia]$ contains a factor of N . Even though Alice does not know this trapdoor for $\mathcal{M}_N$, she knows that this trapdoor exists. She can therefore safely generate a crs using our IVC scheme, and delegate the computation.

In this example, the goal of the IVC scheme is to in fact help *find* this trapdoor factorization, but we can guarantee soundness knowing only that such a trapdoor exists.

Next, we give an example in the cryptographic setting, where trapdoors are in fact commonplace. For example, public key encryption schemes have a natural trapdoor - the secret key. We give an example in terms of the problem of “verifiable shuffling” [NSK04, CKLM12, CKLM13], inspired by [CGKS23]. We note that verifiable shuffling [NSK04] has many applications to mix-nets [Cha81], anonymized e-voting [SK95], and more.

Example 2. Suppose Bob has a list $\mathbf{x} = (x_1, \dots, x_k)$ that he would like to keep secret (e.g. a deck of cards). Bob is planning to use a “shuffled” version of this list in some application with his n friends. However, to ensure fairness of the shuffling, each of his n friends want to participate in the shuffling process. Additionally, he wants to be able to verify the shuffle at each step.

To facilitate this, we can follow the shuffling template of [NSK04] which uses a perfectly rerandomizable encryption scheme.^a Bob will use such a scheme to encrypt each element of x to get ciphertexts $\mathbf{c}_0 = (\text{ct}_1^{(0)}, \dots, \text{ct}_k^{(0)})$. Bob will pass these ciphertexts to his first friend, who will permute the set of ciphertexts according to a permutation σ_1 of their choice and then rerandomize each of these values to get ciphertexts $\mathbf{c}_1 = (\text{ct}_1^{(1)}, \dots, \text{ct}_k^{(1)})$. This friend will then pass these ciphertexts to the next friend, and so on:

$$\mathbf{c}_0 \xrightarrow{\sigma_1 + \text{ReRand}} \mathbf{c}_1 \xrightarrow{\sigma_2 + \text{ReRand}} \dots \xrightarrow{\sigma_n + \text{ReRand}} \mathbf{c}_n$$

One can view the above process as an incremental computation where at each step we take in a set of ciphertexts $\mathbf{c}_i$, and apply a “non-deterministic” transformation specified by a permutation σ_i and randomness r_i (for rerandomization). This process has a simple trapdoor: the secret key of the encryption scheme. Indeed, to verify that a set of ciphertexts $\mathbf{c}_i$ is the result of a valid sequence of shuffles on the original set of ciphertexts $\mathbf{c}_0$, one can use a simple trapdoor circuit $\text{Trap}_{\mathcal{M}}(\mathbf{c}_0, \mathbf{c}_i, i)$ which simply decrypts the two sets of ciphertexts with the corresponding secret key sk , and checks that the set of underlying messages are permutations of one another.

Bob can therefore delegate the shuffling process with the IVC scheme to implement a “multi-hop” verifiable shuffling. In particular, his friends can pass along their shuffled set of ciphertexts along with an updatable proof which ensures that everyone so far

has shuffled the ciphertexts honestly. As a bonus, note that the semantic security of the encryption scheme along with our zero-knowledge property guarantees that each person’s permutation remains computationally hidden.

^aIn a perfectly rerandomizable encryption scheme, there exists an algorithm ReRand such that for any message x , if $\text{sk} \leftarrow \text{Gen}(1^\lambda)$ and $\text{ct} \leftarrow \text{Enc}(\text{sk}, x)$, then $\text{ReRand}(\text{ct})$ is identically distributed to a fresh encryption $\text{ct}' \leftarrow \text{Enc}(\text{sk}, x)$. (Note that ReRand and Enc are probabilistic algorithms).

To achieve Theorem 1.3, we deviate from Valiant’s “proof-merging” template for IVC which has been the backbone of many constructions (we discuss barriers to using this approach to achieve full-succinctness in Section 2.3). We instead rely on a new “*chaining*” approach of computing proofs, inspired by the construction of fully-succinct BARGs of Garg et al [GSWW22]. We then show security of our construction by building on the ideas from a recent construction of adaptive SNARG for NP [WW24a, WW24c]. We also show a novel way to use the existence of trapdoors in our security proofs, and leverage it to achieve succinct proofs in the IVC setting. We give an overview of our construction and proof techniques in Section 2.3.

On the use of trapdoors. Trapdoor languages explicitly avoid the Gentry-Wichs barrier, as observed in [CGKS23]. However, as discussed in [CGKS23], it was not known how to leverage such a trapdoor for proving adaptive soundness. In this work, we demonstrate a novel way to leverage just the *existence* of such a trapdoor. We therefore think this aspect of our result is of independent interest (see for example Remark 2.3). We emphasize, however, that our main motivation for restricting to trapdoor languages is not to achieve adaptive soundness, but rather to achieve *full-succinctness* in the IVC setting (see Section 2.3 for a detailed discussion).

1.2 Organization of the paper

In Section 2, we give a technical overview of our work. In particular, we go over existing works, motivate our constructions and outline our proof of security. In Section 3, we introduce relevant notation and definitions. In Section 4, we present and analyze our construction of IVC for NP. In Section 5, we present and analyze our construction of fully-succinct IVC for trapdoor languages.

2 Technical Overview

In this section, we give an overview of our constructions and outline our security proofs. In Section 2.1, we review prior work on IVC for deterministic computations and describe some barriers to extending it to NP. In Section 2.2, we explain how to overcome these barriers and provide a construction of IVC for NP. In Section 2.3, we first describe the challenges in obtaining fully-succinct IVC, and then give our construction of fully-succinct IVC for trapdoor languages.

2.1 Prior Work

Valiant’s IVC template. In his seminal work that defined IVC, Valiant also proposed the following construction for IVC via “proof merging” assuming the existence of a “computational” proof of knowledge (i.e. a SNARG of knowledge, or a SNARK). The high level idea is as follows: Given a SNARK proof π_1 that x_0 reaches x_1 after t_1 (deterministic) steps, and a proof π_2 that x_1 reaches x_2 in t_2 steps, one can “merge” π_1 and π_2 to obtain a proof π that x_0 reaches x_2 in $t_1 + t_2$ steps. If the merged proof π has length roughly equal to one of π_1 and π_2 , one can repeatedly merge proofs

to obtain an IVC construction. To argue soundness, one can use the knowledge soundness of π to recursively extract the merged proofs π_1 and π_2 until we have extracted the base layer steps of the computation, which can be directly verified. In fact, assuming full knowledge extraction, we can extract the entirety of the prover’s IVC computational path

While this template has been utilized in several works to construct IVC (and more generally, proof carrying data) [BCCT13], there are known barriers to constructing SNARKs for all of NP from *falsifiable assumptions* [CGKS23]. Therefore, we need a different approach in order to construct IVC for NP (or even just IVC for P) from standard assumptions.

IVC via somewhere extractable BARGs. By adapting Valiant’s IVC template, the works of Devadas et al. [DGKV22] and Paneth and Pass [PP22] were able to construct IVC for P from rate-1 *somewhere extractable* batch arguments (seBARGs).

Recall that a BARG is a SNARG for Batch-NP-languages, i.e. languages of the form:

$$\{(x_1, \dots, x_k) \mid \exists (w_1, \dots, w_k) \text{ such that } R(x_i, w_i) = 1 \text{ for all } i\}.$$

A somewhere extractable BARG or seBARG has the following properties.

- **Somewhere extractability:** Given an index i , one can generate a crs along with a trapdoor td such that given a proof π corresponding to $(x_1, \dots, x_k)$, one can extract a witness $w_i \leftarrow \text{Ext}(\text{crs}, \text{td}, \pi)$.
- **Index hiding:** Given only the crs, the index i on which it is somewhere-sound is hidden.

The succinctness requirement for a BARG proof π for k instances is that $|\pi| \ll k \cdot |w|$. Since the succinctness requirement and the knowledge soundness property for a seBARG is milder than that of a full-fledged SNARK for NP, seBARGs can in fact be constructed from various falsifiable assumptions, including LWE [CJJ22, WW22, DGKV22, CGJ⁺23]. Moreover, the works of [PP22, DGKV22, BDSZ24] show how to achieve rate-1 BARGs, where the proof length is $|\pi| = |w| + \text{poly}(\lambda)$. Note that this is essentially the best succinctness one can hope to achieve from falsifiable assumptions while retaining the somewhere extractability property.

The observation in the works of [DGKV22, PP22] is that the knowledge soundness achieved by seBARGs is sufficient for the “merging” step in Valiant’s template. Suppose that a cheating proof π was created by merging π_1 and π_2 . Then, it must be the case that either π_1 or π_2 is a cheating proof. Since the reduction only needs to extract one of π_1 or π_2 , the reduction can pick the somewhere-extractable index of the seBARG to extract the one it wants. By the index-hiding property of the seBARG, one can assert that changing this index will not change the adversary’s cheating pattern.

The challenge in achieving IVC for NP. One of the key facts used in the soundness analysis of [DGKV22, PP22] was that the reduction can *decide* given a tuple (x_0, x_t, t) , if x_t was in fact reachable from x_0 in t steps. At a high level, since the seBARG only gives a “partial extraction” guarantee, we need to pick the somewhere extractable index based on the adversary’s cheating pattern. In the case of P, this can be decided in time $\text{poly}(|x|, t) \leq \text{poly}(|x|, T)$, where T was an upper bound on the number of steps. Therefore, one can rely on the $\text{poly}(|x|, T)$ security of the seBARG and change the extractable index of the seBARG, while ensuring that the adversary’s cheating pattern does not change.

However, if the computation path takes *non-deterministic* steps, the witness from x_0 to x_t has length $|w| \cdot t$, and a brute-force search over all such witness will take time $2^{O(|w| \cdot T)}$. If our reduction has to be $2^{O(|w| \cdot T)}$ -secure, even if we rely on subexponential assumptions, our security parameter has to grow polynomially with $\text{poly}(|w|, T)$. This efficiency can be trivially achieved by simply

maintaining all the intermediate witnesses. Therefore, it seems like IVC for NP with non-trivial succinctness is out of reach from standard assumptions.

This informal argument that the reduction needs to “decide” the language was in fact formalized by Gentry-Wichs for the case of adaptive SNARGs. Since IVC in particular implies an adaptive SNARG for the corresponding language, Gentry-Wichs has been often quoted as a reason why IVC for NP from standard assumptions is in fact impossible [BCMS20, HAN23].

2.2 IVC for NP

Overcoming Gentry-Wichs. However, our observation is that the special IVC language defined by

$$\mathcal{L}_{M,T} = \left\{ (x_0, x_t, t) : \begin{array}{l} M : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n, x_0, x_t \in \{0, 1\}^n, t \leq T, \\ \exists (x_1, \dots, x_{t-1}, w_1, \dots, w_t) \text{ such that } \forall i \in [t], M(x_i, w_i) = x_{i+1} \end{array} \right\}$$

has a non-uniform circuit of size $2^{O(|x|)} \cdot \text{poly}(T)$ which decides the language: the circuit that hard-codes the whole truth-table of $\mathcal{L}_{M,T}$. Therefore, it is in fact sufficient to rely on $2^{O(|x|)} \cdot \text{poly}(T, \lambda)$ -hardness of the underlying assumptions.¹ Assuming subexponential hardness, we can then hope to achieve proofs of size $\text{poly}(\lambda, |x|, \log T)$. While this proof size still grows with the size of each configuration, it does not grow polynomially in T ! Therefore, there still seems to be room to achieve a non-trivial proof length, and this is in fact what we leverage.

Our construction. Our observation is that with appropriate subexponential assumptions and a careful security analysis, a construction nearly identical to that of [PP22] can be used to construct IVC for NP.

We rely on the following ingredients:

- $2^{O(|x|)} \cdot \text{poly}(T, \lambda)$ -sound non-adaptive SNARGs for NP.
- $2^{O(|x|)} \cdot \text{poly}(T, \lambda)$ -sound rate-1 somewhere-extractable arguments for Batch-NP (seBARGs).

Note that although the BARG is rate-1, the proof size will incur an additive $\text{poly}(\lambda, |x|, \log T)$.

In contrast, the constructions of IVC for P relied on RAM delegation and polynomially secure rate-1 seBARGs.

Remark 2.1. For simplicity of exposition, in this technical overview, if we say that an assumption is 2^{λ^δ} -secure, we mean that all adversaries of size at most 2^{λ^δ} have advantage at most $2^{-\lambda^\delta}$. In our actual proof, it suffices to consider hardness against $\text{poly}(\lambda)$ -adversaries with advantage at most $2^{-\lambda^\delta}$.

We now describe our construction:

- **Base case:** For a single time step $x_0 \rightarrow x_1$, we compute a SNARG proof π for the claim that there exists w such that $M(x_0, w) = x_1$. We call π a “level 0” proof.
- **Merging proofs at level $k+1$:** Suppose we have two “level k ” proofs π_0 and π_1 for the claims that $x_0 \rightarrow x_1$ and $x_1 \rightarrow x_2$ in 2^k steps each. To merge them into a “level $k+1$ ” proof that x_0 reaches x_2 in 2^{k+1} steps, we first create a rate-1 seBARG proof π_{BARG} on the statements (x_0, x_1) , and (x_1, x_2) proving that there exists π_0 and π_1 such that $\text{IVC.Ver}_k(x_0, x_1, \pi_0) = 1$ and $\text{IVC.Ver}_k(x_1, x_2, \pi_1) = 1$. Then, we define

$$\text{IVC.Ver}_{k+1}(x_0, x_2, (x_1, \pi_{\text{BARG}})) = \text{seBARG.V}(((x_0, x_1), (x_1, x_2)), \pi_{\text{BARG}}).$$

¹This technique is also known as complexity-leveraging.

Note that since IVC.Ver_{k+1} needs the midpoint x_1 of the path between (x_0, x_2) to verify the proof, we set the new “level $k + 1$ ” proof to be (x_1, π) . This proof has length $|x_1| + |\pi_{\text{BARG}}| = |x_1| + |\pi_0| + \text{poly}(|x|, \log T, \lambda) \leq |\pi_0| + \text{poly}(|x|, \log T, \lambda)$.

Note that at time t , the prover maintains at most $O(\log t)$ proofs since he only needs to maintain at most one proof from each level (since two proofs of the same level could be merged together). Therefore, the total proof size grows with $\text{poly}(|x|, \log T, \lambda)$.

Soundness analysis. We can now establish the soundness of a “level k ” proof as follows (in a similar manner as was done in [PP22, DGKV22]). For the base-case, one can allude to the SNARG soundness to show that level 0 proofs are sound. Now, suppose there exists an adversary $\mathcal{A}$ that creates cheating level k proofs with probability ϵ . In other words, he produces configurations x_0, x_2 and proof (x_1, π_{BARG}) such that there $x_0 \not\rightarrow x_2$ in 2^k steps. Then, by the structure of the IVC language, it must therefore be the case that at least one of $x_0 \not\rightarrow x_1$ or $x_1 \not\rightarrow x_2$ in 2^{k-1} steps. We say $\mathcal{A}$ cheats on hop 0 if $x_0 \not\rightarrow x_1$, and hop 1 otherwise. Note that given x_0, x_2 and (x_1, π_{BARG}) , we can decide using a non-uniform circuit of size $2^{O(|x|)} \cdot \text{poly}(T)$ the probability that $\mathcal{A}$ is cheating on either hop.

Suppose that $\mathcal{A}$ cheats on hop b with probability $\frac{\epsilon}{2}$. Now, switch the seBARG crs to one that is extracting on hop b . Then, there are two possible cases:

- **$\mathcal{A}$ cheats on hop b with probability at least $\epsilon/2$.** In this case, we extract the level $k - 1$ proof from the seBARG corresponding to hop b . Now, one can allude to the soundness of the level $k - 1$ proof inductively. By doing this recursively, one can extract a cheating level 0 proof with probability ϵ/T .
- **$\mathcal{A}$ cheats on hop b with probability noticeably less than $\epsilon/2$.** In this case, since this fact can be checked by a $2^{O(|x|)} \cdot \text{poly}(T)$ circuit, we can use $\mathcal{A}$ to break the subexponential index-hiding property of the seBARG.

Therefore, one can then inductively argue that level k proofs are sound.

2.2.1 Adding zero-knowledge.

In the P setting, given x_0 , there is a unique, deterministic path. On the other hand, in the setting of NP, there could be multiple paths using different witnesses to go from a state x_0 to a state x_t . It is therefore natural to ask if our construction can be modified to additionally have a zero-knowledge property which hides the trace of the prover.

As a first attempt, one can imagine simply replacing the rate-1 seBARG and SNARG with ones that are zero-knowledge. However, this does not quite work because the zero-knowledge property only guarantees that the *witness* is hidden, not the statements.

To explain why this is an issue, consider a proof for a computation $x_0 \rightarrow x_2$. Ideally, one can argue that the IVC proof reveals nothing about the path from x_0 to x_2 . Looking back at our construction, however, we see that the corresponding IVC doesn’t just contain a BARG proof, but also contains the midpoint configuration x_1 . In other words, the midpoint x_1 is leaked, even if the seBARG proof is zero-knowledge. We therefore have to find a way to hide the intermediate states.

Our solution. To achieve our construction, we instead rely on the following additional tools:

- a statistically sound non-interactive zero knowledge argument (NIZK), which is additionally a proof of knowledge.
- a public-key encryption scheme with perfect correctness.

Now, along with the SNARG and BARG common reference strings, a single NIZK crs and the public key pk are published. Below, we highlight the main changes needed to make our construction zero knowledge:

- All configurations x_i are encrypted under pk. We denote by ct_i the encryption of x_i , and let r_i be the corresponding randomness.
- At step $i + 1$, the prover is additionally given ciphertexts ct_0 and ct_i and randomness r_0 and r_i in the clear. The prover is not provided with the randomness for any other intermediate ciphertexts. (Note that one can verify that the given randomness matches the ciphertexts by simply encrypting and checking equality.) The prover then encrypts the new state x_{i+1} with fresh randomness r_{i+1} . After the step, the randomness r_i is tossed and only the randomness r_{i+1} used to encrypt x_{i+1} is maintained.
- **Base case:** A level 0 proof at step $i + 1$ for $x_i \xrightarrow{w_{i+1}} x_{i+1}$ is now constructed as follows:
 - Sample randomness r_{i+1} and encrypt x_{i+1} to obtain ct_{i+1} .
 - Create a SNARG proof π_{SNARG} for the fact that there exists a witness w_{i+1} such that $M(x_i, w_{i+1}) = x_{i+1}$.
 - Create a NIZK proof π_{NIZK} for the instance $(\text{ct}_i, \text{ct}_{i+1})$ proving that there exist $x_i, x_{i+1}, r_i, r_{i+1}, \pi_{\text{SNARG}}$ such that $\text{Enc}_{\text{pk}}(x_i, r_i) = \text{ct}_i$, $\text{Enc}_{\text{pk}}(x_{i+1}, r_{i+1}) = \text{ct}_{i+1}$ and $\text{SNARG}.\mathcal{V}(x_i, x_{i+1}, \pi_{\text{SNARG}}) = 1$.

A level 0 proof is given by $(\text{ct}_i, \text{ct}_{i+1}, \pi_{\text{NIZK}})$.

- **Merging proofs at level $k + 1$:** Level $k + 1$ proofs for $k \geq 0$ are constructed just as before with respect to level k proofs, except the statements are defined by the ciphertexts $\text{ct}_k, \text{ct}_{k+2^j-1}, \text{ct}_{k+2^j}$ rather than their corresponding configurations $x_k, x_{k+2^j-1}, x_{k+2^j}$. Note that this merge can be done without knowing the randomness used to encrypt the configurations.

It suffices to check the soundness of the level 0 proofs since the rest of the construction is essentially identical. Since the NIZK is statistically sound and is a proof of knowledge, the only possible way to create a cheating level 0 proof is if $\text{SNARG}.\mathcal{V}(x_i, x_{i+1}, \pi_{\text{SNARG}}) = 1$ even though $x_i \not\rightarrow x_{i+1}$. In this case, we can extract a cheating SNARG proof, thereby reaching a contradiction.

Sketch of zero-knowledge. We sketch a sequence of hybrids to show that an IVC proof for the statement (x_0, x_i, i) is zero-knowledge, i.e. can be simulated given only (x_0, x_i, i) .

- H₁ Sample the common reference string, and construct the proof as an honest prover would based on an honest sequence $x_0 \xrightarrow{w_1} x_1 \xrightarrow{w_2} \dots \xrightarrow{w_i} x_i$.
- H₂ Compute each ciphertext $\text{ct}_i \leftarrow \text{Enc}_{\text{pk}}(x_i, r_i)$ honestly, but simulate the NIZK crs and all level 0 NIZK proofs using the zero-knowledge simulator for the NIZK. Compute all level j proofs as before for $j \geq 1$. Indistinguishability follows from the zero-knowledge simulation security of the NIZK.
- H₃ Sample randomness r_0 and r_i . Compute $\text{ct}_0 = \text{Enc}_{\text{pk}}(x_0; r_0)$ and $\text{ct}_i = \text{Enc}_{\text{pk}}(x_i; r_i)$, but replace $\text{ct}_k \leftarrow \text{Enc}_{\text{pk}}(0)$ for all $1 \leq k \leq i - 1$. Construct the rest of the proof as in $\mathcal{H}_1$.

Note that the randomness used to sample the ciphertexts ct_k for $1 \leq k \leq i - 1$ are hidden from the view in both $\mathcal{H}_1$ and $\mathcal{H}_2$. Therefore, indistinguishability follows from the semantic security of the public-key encryption scheme.

Note that everything in H_3 can be simulated given only x_0 and x_i . Therefore, H_3 defines a zero-knowledge simulator for the modified IVC protocol, as desired.

2.3 Fully succinct IVC for Trapdoor Languages

In this section, we describe our construction of fully succinct IVC for trapdoor languages. Recall that we say an IVC language defined by a machine $\mathcal{M}$ has a “trapdoor circuit” if there exists a circuit $\text{Trap}_{\mathcal{M}}$ such that

$$\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 1 \iff \exists(w_1, \dots, w_i) \text{ such that } x_0 \xrightarrow{w_1} x_1 \xrightarrow{w_2} \dots \xrightarrow{w_i} x_i.$$

In other words, $\text{Trap}_{\mathcal{M}}$ is some non-uniform circuit that can *decide* the IVC language corresponding to the machine $\mathcal{M}$.

We first begin by discussing why some immediate strategies to construct fully succinct IVC fail to give us *full* succinctness.

Issue with the proof merging template. We give an *informal* argument for why Valiant’s “proof merging” template does not seem to be a viable route towards fully succinct IVC for NP. Recall that the template is as follows: given a proof π_1 that x_0 reaches x_1 after t_1 (deterministic) steps, and a proof π_2 that x_1 reaches x_2 in t_2 steps, one can “merge” π_1 and π_2 to obtain a proof π that x_0 reaches x_2 in $t_1 + t_2$ steps. The soundness analysis then says that if x_0 does not in fact reach x_2 , then the prover must have cheated on either the first hop or second hop. This argument crucially relies on the ability to *extract* either π_1 or π_2 . Upon closer inspection, it is easy to see that one would also need to extract the midpoint x_1 . Therefore, π must information-theoretically determine both x_1 and π_b , which seems to imply that π has to be of length at least $|x_1| + |\pi_b|$. Therefore, an extraction-based security argument would result in IVC proofs which are not fully succinct.

A first attempt. A recent line of beautiful works of [WW24a, WZ24, WW24c] solves the long-standing open problem of constructing adaptive SNARGs for all of NP with proof size $\text{poly}(\lambda)$. More recently, the work of Devadas, Waters and Wu [DWW24] extend these techniques to the setting of BARGs for NP. They construct an adaptive BARG scheme where the size of the crs only grows polylogarithmically with the number of statements k , and the proof size is $\text{poly}(\lambda)$.

At first glance, this seems like exactly what we need. For an IVC computation path given by $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$, one just needs to prove the batch statement $((x_0, x_1), (x_1, x_2), \dots, (x_{T-1}, x_T))$ that for all i , there exists w_i such that $x_{i-1} \xrightarrow{w_i} x_i$. Therefore, it seems like we can directly apply the construction of [DWW24] to achieve a fully succinct protocol. However, looking closer at the construction, we notice two issues:

- The prover requires the whole statement $(x_1, \dots, x_k)$ before he generates the proof, and therefore the proof cannot be aggregated incrementally.
- The verifier needs the whole statement $(x_1, \dots, x_k)$, or at the very least, a *somewhere extractable hash* (SSB) of the statements to be able to verify the proof.

Therefore, while the BARG proof itself is short, the total amount of information that needs to be communicated to the verifier grows with at least the SSB hash, which scales with the size of $|x_i|$.

We remark that it might be possible to address the first issue by making the adaptive BARG of [DWW24] also *updatable* as was done in [GSWW22]. However, the second problem still persists and seems inevitable. In short, it seems necessary for the verification algorithm to receive at least a somewhere extractable hash of the instances so that it is *information-theoretically binding* on a wrong instance x_i . This seems crucial for the *iO*-based arguments to go through.

The challenge. To summarize, both the proof-merging approach and fully succinct BARG approach break down for a similar reason: if the proof size does not grow with the configuration size, the proof is no longer binding to *any intermediate point* of the trace of the IVC computation. This seems to be a huge bottle-neck with our known proof techniques. Therefore, there does not seem to be an immediate way to obtain succinctness beyond what was already achieved in our first construction.

2.3.1 Our construction

Nevertheless, we show that we can overcome this “configuration-length” barrier in the case of trapdoor languages.

We draw inspiration from the fully succinct BARG constructions from $i\mathcal{O}$ and OWF [GSWW22, DWW24], and adopt their “chaining approach” to constructing proofs. Concretely, the crs in our construction consists of programs $\tilde{P}$ and $\tilde{V}$ which are $i\mathcal{O}$ obfuscations of $P_K(\cdot)$ and $V_K(\cdot)$ as defined below:

- Sample a key K for a puncturable PRF F_K .
- $P_K((x_0, x_i, i), \pi_i, w_{i+1})$:
 - Given statement (x_0, x_i, i) , use the witness w_{i+1} to compute the next configuration $x_{i+1} = \mathcal{M}(x_i, w_{i+1})$.
 - **Verify previous proof:** If $i > 0$ and $\pi_i \neq F_K(x_0, x_i, i)$, output $\perp$.
 - **Generate new proof:** Output $F_K(x_0, x_{i+1}, i + 1)$.
- $V_K((x_0, x_i, i), \pi_i)$:
 - Given statement (x_0, x_i, i) , accept if $\pi_i = F_K(x_0, x_i, i)$.

Given the crs, the prover and verification algorithms can be defined straightforwardly as follows:

- **Prover algorithm:** Given an instance (x_0, x_T, T) along with a proof π_T , the prover can generate a new proof for $(x_0, x_{T+1}, T+1)$ with witness w_{T+1} by computing $\pi_{T+1} \leftarrow \tilde{P}((x_0, x_T, T), \pi_T, w_{T+1})$.
- **Verification algorithm:** Given an instance (x_0, x_T, T) and proof π_T , the verifier simply runs $\tilde{V}((x_0, x_T, T), \pi_T)$ and accepts if $\tilde{V}$ outputs 1.

We emphasize that our construction *does not* mention the trapdoor circuit $\text{Trap}_{\mathcal{M}}$. Looking forward, since the security reduction will change the obfuscated programs to invoke $\text{Trap}_{\mathcal{M}}$, the programs need to be padded to appropriate lengths which depend on $\text{Trap}_{\mathcal{M}}$.

The intuition behind the above construction is clear: one can retrieve a signature for $(x_0, x_{i+1}, i+1)$ if one has a signature for (x_0, x_i, i) , and a witness w_{i+1} such that $x_i \xrightarrow{w_{i+1}} x_{i+1}$. The main difference between our algorithm and that of GSWW [GSWW22] is that our verifier only needs to know the “end points” of the IVC path to be able to verify the proof.

We note that the above construction is clearly zero-knowledge, because one can compute all proofs given just the PRF K . Therefore, it suffices to argue soundness.

2.3.2 Soundness

To prove soundness of our construction, we draw inspiration from the proof strategies of GSWW [GSWW22] and WW [WW24a, WW24c]. We first describe the proof techniques from these prior works. Then, we show a warm-up soundness proof of our construction for $i = 1$ in Section 2.3.3, and describe the full proof in Section 2.3.4.

Proof strategy from GSWW [GSWW22]. While our construction is very natural, it is not immediately clear how to rely on $i\mathcal{O}$ security to prove soundness of this construction. In particular, for an intermediate point of the proof generation $(x_0, x_i, i) \notin L$, we only have the guarantee that a proof for $(x_0, x_{i-1}, i-1)$ is *hard to compute*. However, such a computational claim is not sufficient for us to invoke $i\mathcal{O}$ security. To address this, GSWW adopt the following proof strategy: in the non-adaptive setting, GSWW slowly puncture out the proofs iteratively for each time-step. By iteratively arguing that proofs for $(x_0, x_{i-1}, i-1) \notin L$ *do not exist*, we are able to remove the proof for (x_0, x_i, i) at the next time step i , and proceed forward through this iterative process. This is the general template that we will rely on in our proof.

Proving adaptive soundness. The second problem we have to address is that we need to prove *adaptive soundness*. Indeed, note that even if we were to aim only for non-adaptively sound IVC, where the configurations x_0 and x_T are chosen *before* the crs is published, a cheating prover can still choose the intermediate states $x_1, \dots, x_{T-1}$ adaptively. Since we want fully succinct proofs, we therefore need to show *adaptive soundness* of the scheme without complexity-leveraging. To do this, we draw inspiration from the SNARG literature.

The work of [WW24a, WW24c] showed *adaptive* soundness for SNARGs for NP. The main difference in the adaptive and non-adaptive settings is the following: in the non-adaptive setting, a cheating prover declares beforehand what $x^* \notin L$ he is cheating on. Therefore, one can imagine coming up with a “trapdoored crs” in the security proof which plants some challenge (e.g. one-way function, PRG, etc) at the point x^* . In fact, this was the strategy used by Sahai and Waters [SW14] to prove non-adaptive soundness of their SNARG setting. However, in the adaptive setting, we can no longer rely on this technique since we do not know which $x^* \notin L$ the adaptive adversary will cheat on. One could guess such an x^* (i.e. complexity-leverage), but this will result in long proofs if done naively.

The first idea from WW is simple: plant a “challenge” at every $x^* \notin L^2$. By doing this, one can ensure that any cheating proof can be used to break the security of the challenge. While this idea is reasonable, another problem arises: if we planted a different challenge at every x^* , we would need to invoke the one-way function or PRG security $2^{|x|}$ times. This would result in proofs that grow with $|x|$. Instead, Waters and Wu rely on the idea of “rerandomizing” the *same* challenge across all instances x^* , and only invoking the security once. This way, one can use any cheating proof to recover a solution to the challenge. WW implement this idea via rerandomizable one-way functions in [WW24a], but relax this requirement to only require injective one-way functions in [WW24c]. We will follow the strategy from [WW24c].

2.3.3 Warmup: Soundness for $i = 1$

As a warm-up, we first show the soundness of the following **simplified construction** which only creates proofs for one-hop statements. While this setting is way simpler, we gloss over the analysis

²Since WW do this for *all* of NP, they have to use an additional “two-challenge” approach. Since we are only considering trapdoor languages, we do not need to rely on this technique. Therefore, we gloss over this detail of WW in this overview. A curious reader can refer to Remark 2.2 for a comparison.

carefully to introduce the techniques from Waters-Wu [WW24a, WW24c] and [GSWW22] that will be relevant to our final security proof.

In this simplified construction, the crs consists of programs $\tilde{P}$ and $\tilde{V}$ which are $i\mathcal{O}$ obfuscations of $P_K(\cdot)$ and $V_K(\cdot)$ as defined below:

- Sample a key K for a puncturable PRF F_K .
- $P_K(x_0, w_1)$:
 - Given statement x_0 , use the witness w_1 to compute the next configuration $x_1 = \mathcal{M}(x_0, w_1)$.
 - Output $F_K(x_0, x_1, 1)$.
- $V_K((x_0, x_1, 1), \pi)$:
 - Given statement $(x_0, x_1, 1)$, accept if $\pi = F_K(x_0, x_1, 1)$. Else, reject.

We now describe our proof. Recall that we are only showing soundness for the case where the IVC language has a trapdoor, i.e. there exists a circuit $\text{Trap}_{\mathcal{M}}$ such that

$$\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 1 \iff \exists(w_1, \dots, w_i) \text{ such that } x_0 \xrightarrow{w_1} x_1 \xrightarrow{w_2} \dots \xrightarrow{w_i} x_i.$$

Plant challenge y . As a first step, we adopt the strategy of Waters, Wu [WW24c] in which we change the signature for all false $(x_0, x_1, 1)$ from $F_K(x_0, x_1, 1)$ to $F_{K_{\text{off}}}(x_0, x_1, 1) \oplus y$ where $F_{K_{\text{off}}}$ is an independent PRF serving as a one-time pad for a common value y . Looking forward, y will later correspond to a preimage for a “common challenge” y^* under an injective length-doubling PRG G , which will enable us to remove proofs of false statements.

To plant these signatures, we iterate over all **false** $(x_0, x_1, 1)$ and perform the following hybrids. Recall that we can efficiently determine which $(x_0, x_1, 1)$ tuples are false by checking if $\text{Trap}_{\mathcal{M}}(x_0, x_1, 1) = 0$.

$H_{x_0, x_1}^{(1)}$ Puncture the PRF keys K and K_{off} at $(x_0, x_1, 1)$ to get punctured keys $K\{x_0, x_1, 1\}$ and $K_{\text{off}}\{x_0, x_1, 1\}$, and use these punctured keys in the prover and verifier algorithms. Note that the prover algorithm never evaluates $F_K(x_0, x_1, 1)$ because $(x_0, x_1, 1)$ is false. Hard-code $z = F_K(x_0, x_1, 1)$ in the verification algorithm. Since these programs remain functionally equivalent, this is indistinguishable via $i\mathcal{O}$ security.

$H_{x_0, x_1}^{(2)}$ Sample a uniformly random string r , and set $z = r$. This follows from the punctured key security, i.e.

$$(K\{x_0, x_1, 1\}, F_K(x_0, x_1, 1)) \approx_c (K\{x_0, x_1, 1\}, r).$$

$H_{x_0, x_1}^{(3)}$ Replace z with $r \oplus y$. Since r is a uniformly random string, the distribution of z is identical to the previous hybrid.

$H_{x_0, x_1}^{(4)}$ Replace r with $F_{K_{\text{off}}}(x_0, x_1, 1)$. This follows from the punctured key security, i.e.

$$(K_{\text{off}}\{x_0, x_1, 1\}, F_{K_{\text{off}}}(x_0, x_1, 1)) \approx_c (K_{\text{off}}\{x_0, x_1, 1\}, r).$$

$H_{x_0, x_1}^{(5)}$ Unpuncture K and K_{off} .

This gives us the following verifier program (which is then obfuscated and placed in the crs).

- $V_K((x_0, x_1, 1), \pi)$:
 - If $\text{Trap}_{\mathcal{M}}(x_0, x_1, 1) = 0$,
 - * accept if $\pi = F_{K_{\text{off}}}(x_0, x_1, 1) \oplus y$.
 - Else, accept if $\pi = F_K(x_0, x_1, 1)$. Else, reject.

Note that the existence of a trapdoor $\text{Trap}_{\mathcal{M}}$ with small description size was important in ensuring that we only planted the PRG challenge at false statements.

Although we performed exponentially many hybrids, we have so far only relied on the security of $i\mathcal{O}$ and puncturable PRFs.

Remark 2.2. *The above hybrids follow the hybrids of [WW24c] very closely. The main difference is that Waters-Wu use a “two-challenge” approach, where for each x , one challenge is considered “on-path” and the other challenge is consider “off-path”. The proof that is “on-path” is decided by a “selector PRF” K_{sel} which outputs a special bit b on every x . They then plant a one-way function challenge in every “off-path” proof. This approach was important to achieving SNARGs for all of NP. In contrast, we only have one challenge for every instance, but we instead use the trapdoor circuit to switch the challenge to “off-path” mode for every instance not in the language.*

Remove proofs for false statements. At this point, we will implement the strategy of [GSWW22] to remove the proof at all false statements.

H₁ We make the following change to the verification algorithm: instead of checking the condition

$$\pi \stackrel{?}{=} F_{K_{\text{off}}}(x_0, x_1, 1) \oplus y,$$

we check

$$G(\pi \oplus F_{K_{\text{off}}}(x_0, x_1, 1)) \stackrel{?}{=} G(y),$$

where G is some injective PRG which is length-doubling. Since this change is functionally equivalent, we can invoke $i\mathcal{O}$ security to argue soundness.

H₂ Set $y^* = G(y)$, and change the check to $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_1, 1)) \stackrel{?}{=} y^*$.

H₃ Now, sample $y^* \leftarrow \{0, 1\}^{2\ell}$. This is indistinguishable from the previous hybrid by invoking PRG security.

H₄ Since G is length-expanding, with probability $2^{-\ell}$, y^* is not in the image of G . Then, via $i\mathcal{O}$ security, we can replace the verification algorithm with the following:

- $V_K((x_0, x_1, 1), \pi)$:
 - If $\text{Trap}_{\mathcal{M}}(x_0, x_1, 1) = 0$,
 - * Reject.
 - Else, accept if $\pi = F_K(x_0, x_1, 1)$. Else, reject.

In other words, the verification algorithm in this hybrid does not accept any proof for any false $(x_0, x_1, 1)$. We call this final hybrid H_{ideal} .

Soundness analysis. Suppose there exists a cheating adversary $\mathcal{A}$ for the crs in H_{real} . In other words,

$$\Pr_{\text{crs} \leftarrow H_{\text{real}}} [(x_0^*, x_1^*, \pi) \leftarrow \mathcal{A}(\text{crs}) \wedge (x_0^*, x_1^*, 1) \notin L \wedge V((x_0^*, x_1^*, 1), \pi) = 1] \geq \epsilon,$$

where ϵ is some function in λ . Then, by indistinguishability of H_{real} and H_{ideal} , we claim that:

$$\Pr_{\text{crs} \leftarrow H_{\text{ideal}}} [(x_0^*, x_1^*, \pi) \leftarrow \mathcal{A}(\text{crs}) \wedge (x_0^*, x_1^*, 1) \notin L \wedge V((x_0^*, x_1^*, 1), \pi) = 1] \geq \epsilon - \text{negl}(\lambda) \quad (1)$$

Note that for general (non-trapdoor) languages L , we might not be able to decide in polynomial time whether $(x_0^*, x_1^*, 1) \notin L$. Thus, for such languages, we would be unable to perform a reduction to the polynomial security of the PRG as we did in Hybrid H_3 . Instead, we rely on the existence of an efficient trapdoor circuit $\text{Trap}_{\mathcal{M}}$ such that $\text{Trap}_{\mathcal{M}}(x_0^*, x_1^*, 1) = 0 \iff (x_0^*, x_1^*, 1) \notin L$. Therefore, one could use $\mathcal{A}$ to *efficiently* distinguish H_{real} and H_{ideal} if (1) does not hold. On the other hand, note that

$$\Pr_{\text{crs} \leftarrow H_{\text{ideal}}} [(x_0^*, x_1^*, \pi) \leftarrow \mathcal{A}(\text{crs}) \wedge (x_0^*, x_1^*, 1) \notin L \wedge V((x_0^*, x_1^*, 1), \pi) = 1] = 0 \quad (2)$$

since there are no proofs for $(x_0^*, x_1^*, 1) \notin L$ such that $V((x_0^*, x_1^*, 1), \pi) = 1$. Therefore, we have that for all adversaries $\mathcal{A}$, $\mathcal{A}$'s advantage $\epsilon \leq \text{negl}(\lambda)$, i.e. the simplified construction is sound.

Remark 2.3. We believe that this warm-up is of independent interest for a few reasons. Firstly, one can adapt this proof to show that the original Sahai-Waters is adaptively sound for trapdoor languages. While it was shown by Mathialagan, Peters and Vaikuntanathan [MPV24] that the Sahai-Waters SNARG is adaptively sound in the designated-verifier setting, such a result was not known in the publicly verifiable setting. In fact, the construction of Waters-Wu [WW24a] had to modify the Sahai-Waters construction and introduce the “two-challenge” technique.

Secondly, as discussed in the work of [CGKS23], trapdoor languages are not subject to the Gentry-Wichs barrier [GW11], and hence it seems like a simpler target to show adaptive soundness. However, prior to this work, it was not clear how one could use such a trapdoor. We note that we still needed to rely on subexponential soundness of $i\mathcal{O}$ and puncturable PRFs since we had to puncture the PRF at every $x^* \notin L$.

2.3.4 Analysis of our construction

We now sketch the full soundness proof of our construction beyond the one-hop setting.

The challenge. In the multi-hop setting, we must handle one long, interconnected chain of computation. Because adjacent steps are related, we cannot treat each state as a separate, isolated instance of the language (as one would in a BARG for example). This greatly complicates the security proof.

As mentioned earlier, when using an $i\mathcal{O}$ -based chaining approach, arguing that it is computationally hard to find a false proof at step $i - 1$ is not enough to guarantee it is hard to find a false proof at step i . This is because $i\mathcal{O}$ only guarantees indistinguishability between programs with *identical* input/output behavior, rather than between programs with *computationally indistinguishable* input/output behavior. Looking forward, we will overcome this issue by using a trapdoor circuit which allows us to give information theoretic guarantees on the difficulty of generating false proofs.

Proof overview. Similar to the one-hop setting, our security proof can be broken down into three main steps. We first give an overview of each of the steps, and then describe each step in more detail.

1. **Plant the challenges:** Similar to the one-hop setting, we first perform exponentially many hybrids to plant injective PRG challenges at *all* false points. The main difference from the one-hop setting is as follows: we use a *different* PRG challenge y_i for each time-step i . In order to ensure the crs is short, we will compress all these challenges via a PRF $F_{K_{\text{chal}}}$ (i.e. $F_{K_{\text{chal}}}(i)$ is the challenge at the i th step).
2. **Remove proofs at false statements:** Similar to the one-hop setting, we follow the techniques of [GSWW22] to remove proofs of false statements using a length-doubling PRG. In the multi-hop setting, we inductively remove the proofs for each time step $i = 1, 2, \dots$. Crucially, once we remove all proofs for all false statements (x_0, x_i, i) for $i \leq k$, we use the following fact about the structure of trapdoor IVC languages:

$$\begin{aligned} &\text{if } \mathcal{M}(x_k, w_{k+1}) = x_{k+1} \text{ and } \text{Trap}_{\mathcal{M}}(x_0, x_{k+1}, k+1) = 0, \\ &\text{it must be the case that } \text{Trap}_{\mathcal{M}}(x_0, x_k, k) = 0. \end{aligned}$$

Since we have already removed proofs for all instances (x_0, x_k, k) satisfying $\text{Trap}_{\mathcal{M}}(x_0, x_k, k) = 0$, we can inductively remove the proof at $i = k + 1$.

Since we will only be performing $\text{poly}(T)$ hybrids, we can rely on $\text{poly}(\lambda, T)$ assumptions. Therefore, we can pick an injective $G : \{0, 1\}^\ell \rightarrow \{0, 1\}^{2\ell}$ where $\ell = \text{poly}(\lambda, \log T)$. This is crucial in ensuring that the proof size ℓ in our IVC scheme remains small.

We call this resulting hybrid H_{ideal} .

3. **Analyzing soundness:** At this point, the crs in H_{ideal} contains programs that do not accept any proofs for false statements. We can then argue just as in the one-hop setting that the construction is sound.

In other words, our proof relies on the existence of a trapdoor circuit in each step in crucial ways:

1. We used the succinct description of the circuit in Step 1 to plant the PRG challenges at only false statements.
2. In step 2, we needed the trapdoor circuit to information theoretically assert the inductive step.
3. We use the trapdoor circuit to overcome the so-called ‘‘Gentry-Wichs issue’’ in Step 3 by using it to efficiently decide if $(x_0^*, x_i^*, i) \notin L$ in the reduction.

Remark 2.4. In the SNARG setting (which is equivalent to one-hop IVC), Waters-Wu show how to overcome the first and third problem using a selector PRF. The main reason why we are restricting ourselves to trapdoor languages is to address the second problem, which seems unique to the IVC setting.

Now, we describe each step in more detail.

Step 1: Plant the challenges. Our security proof starts from the real distribution, H_{real} . In this step, for each time-step i , we will ‘‘plant’’ the same PRG challenge $F_{K_{\text{chal}}}(i)$ across *all* false instances (x_0, x_i, i) . To do this, we change the signature of every false statement (x_0, x_i, i) from $F_K(x_0, x_i, i)$ to a signature of the form $F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$.

To plant our challenges, we iterate over all instances (α, β, T) such that $\text{Trap}_{\mathcal{M}}(\alpha, \beta, T) = 0$ (similar to the one-hop setting) and conduct a similar puncturing argument. At the end of this exponential sequence of hybrids, the crs consists of the following two obfuscated programs.

- Sample three puncturable PRF keys, $K, K_{\text{off}}, K_{\text{chal}}$.
- $P_{K, K_{\text{off}}, K_{\text{chal}}}((x_0, x_i, i), \pi_i, w_{i+1})$:
 - Given statement (x_0, x_i, i) , use the witness w_{i+1} to compute the next configuration $x_{i+1} = \mathcal{M}(x_i, w_{i+1})$.
 - **Verify previous proof:** If $i > 0$,
 - * if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$,
 - if $\pi_i \neq F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$, output $\perp$.
 - * else,
 - if $\pi_i \neq F_K(x_0, x_i, i)$, output $\perp$.
 - **Generate new proof:**
 - * if $\text{Trap}_{\mathcal{M}}(x_0, x_{i+1}, i+1) = 0$,
 - output $F_{K_{\text{off}}}(x_0, x_{i+1}, i+1) \oplus F_{K_{\text{chal}}}(i+1)$.
 - * else, output $F_K(x_0, x_{i+1}, i+1)$.
- $V_{K, K_{\text{off}}, K_{\text{chal}}}((x_0, x_i, i), \pi)$:
 - if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$,
 - * if $\pi_i = F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$, accept. Else, reject.
 - else,
 - * if $\pi_i = F_K(x_0, x_i, i)$, accept. Else, reject.

Step 2: Remove proofs from false statements. Now, we take inspiration from the proof strategy of [GSWW22], and remove all proofs of false statements iteratively for $i = 1, 2, \dots$

As before, we make the following functionally equivalent change to the programs in the crs: instead of checking the condition

$$\pi_i \stackrel{?}{=} F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$$

in the prover and verification algorithm, we check

$$G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) \stackrel{?}{=} G(F_{K_{\text{chal}}}(i)),$$

where G is some injective PRG. Since this change preserves functionality, we can invoke $i\mathcal{O}$ to argue that the crs remains indistinguishable. From this point, we will only be using $\text{poly}(\lambda, T)$ -hardness assumptions.

Let $G : \{0, 1\}^\ell \rightarrow \{0, 1\}^{2\ell}$, for some $\ell = \text{poly}(\lambda, \log n, \log m, \log T)$.

Base case: For the case of $i = 1$, note that the prover algorithm never generates a proof for false statements $(x_0, x_1, 1)$. Therefore, $F_{K_{\text{chal}}}(1)$ is only used in the “verify previous proof” step, and not the “generate new proof” step of the prover algorithm.

- First, we puncture K_{chal} at $i = 1$, and hardcode $y = F_{K_{\text{chal}}}(1)$ via $i\mathcal{O}$ security.
- Then, we rely on punctured key security to replace y with random. Now, we can perform hybrids just as we did in the one-hop setting to remove all one-hop proofs.

Now that we have replaced the challenge $F_{K_{\text{chal}}}(1)$ with a random string, we can proceed as we did in the one-hop setting to remove all one-hop proofs and reach the following hybrid:

- Sample three puncturable PRF keys, $K, K_{\text{off}}, K_{\text{chal}}$.
- $P_{K, K_{\text{off}}, K_{\text{chal}}\{1\}, z}((x_0, x_i, i), \pi_i, w_{i+1})$:
 - Given statement (x_0, x_i, i) , use the witness w_{i+1} to compute the next configuration $x_{i+1} = \mathcal{M}(x_i, w_{i+1})$.
 - **Verify previous proof:** If $i > 0$,
 - * if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$ and $i = 1$:
 - Output $\perp$.
 - * if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$,
 - if $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) \neq G(F_{K_{\text{chal}}}(i))$, output $\perp$.
 - * else,
 - if $\pi_i \neq F_K(x_0, x_i, i)$, output $\perp$.
 - **Generate new proof:**
 - * if $\text{Trap}_{\mathcal{M}}(x_0, x_{i+1}, i+1) = 0$ and $i > 1$,
 - output $F_{K_{\text{off}}}(x_0, x_{i+1}, i+1) \oplus F_{K_{\text{chal}}}(i+1)$.
 - * else, output $F_K(x_0, x_{i+1}, i+1)$.
- $V_{K, K_{\text{off}}, K_{\text{chal}}}((x_0, x_i, i), \pi)$:
 - if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$ and $i = 1$,
 - * reject.
 - else if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$ and $i > 1$,
 - * if $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) = G(F_{K_{\text{chal}}}(i))$, accept. Else, reject.
 - else,
 - * if $\pi_i = F_K(x_0, x_i, i)$, accept. Else, reject.

Therefore, we have successfully removed the proofs for all $(\alpha, \beta, 1) \notin L$.

Inductive step: At the inductive step, suppose we have removed all proofs for statements (α, β, i) for $i \leq T$. Then, we use the **structure** of IVC languages to assert the following:

if $\mathcal{M}(x_T, w_{T+1}) = x_{T+1}$ and $\text{Trap}_{\mathcal{M}}(x_0, x_{T+1}, T+1) = 0$,
it must be the case that $\text{Trap}_{\mathcal{M}}(x_0, x_T, T) = 0$.

In other words, the prover never reaches the “Generate new proof” step of the prover algorithm on inputs $(\alpha, \beta, T+1)$ such that $\text{Trap}_{\mathcal{M}}(\alpha, \beta, T+1) = 0$.

With this in mind, we can change the prover algorithm as follows via $i\mathcal{O}$ security:

- $P_{K, K_{\text{off}}, K_{\text{chal}}\{1\}, z}((x_0, x_i, i), \pi_i, w_{i+1})$:
 - Given statement (x_0, x_i, i) , use the witness w_{i+1} to compute the next configuration $x_{i+1} = \mathcal{M}(x_i, w_{i+1})$.
 - **Verify previous proof:** If $i > 0$,
 - * if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$ and $i \leq T$:
 - Output $\perp$.

- * if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$,
 - if $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) \neq G(F_{K_{\text{chal}}}(i))$, output $\perp$.
- * else,
 - if $\pi_i \neq F_K(x_0, x_i, i)$, output $\perp$.
- **Generate new proof:**
 - * if $\text{Trap}_{\mathcal{M}}(x_0, x_{i+1}, i+1) = 0$ and $i > T+1$,
 - output $F_{K_{\text{off}}}(x_0, x_{i+1}, i+1) \oplus F_{K_{\text{chal}}}(i+1)$.
 - * else, output $F_K(x_0, x_{i+1}, i+1)$.

Since $F_{K_{\text{chal}}}(T+1)$ is no longer called to generate proofs, we can now freely puncture $F_{K_{\text{chal}}}$ at $T+1$, and proceed with hybrids just as in the base case (hybrids H_1 to H_4) to remove all proofs at step $T+1$.

Final hybrid: After iterating through all time-steps i , we reach a hybrid H_{ideal} where the crs consists of obfuscations of the following programs:

- Sample three puncturable PRF key K .
- $P_K((x_0, x_i, i), \pi_i, w_{i+1})$:
 - Given statement (x_0, x_i, i) , use the witness w_{i+1} to compute the next configuration $x_{i+1} = \mathcal{M}(x_i, w_{i+1})$.
 - **Verify previous proof:** If $i > 0$,
 - * if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$: output $\perp$
 - * else,
 - if $\pi_i \neq F_K(x_0, x_i, i)$, output $\perp$.
 - **Generate new proof:**
 - * Output $F_K(x_0, x_{i+1}, i+1)$.
- $V_{K, K_{\text{off}}, K_{\text{chal}}}((x_0, x_i, i), \pi)$:
 - if $\text{Trap}_{\mathcal{M}}(x_0, x_i, i) = 0$, output $\perp$.
 - else,
 - * if $\pi_i = F_K(x_0, x_i, i)$, accept. Else, reject.

Since we relied on polynomial hardness in these hybrids, we have that for any $\text{poly}(\lambda, T, n, m)$ adversary $\mathcal{A}$,

$$\left| \Pr_{\text{crs} \leftarrow H_{\text{real}}} [\mathcal{A}(\text{crs}) = 1] - \Pr_{\text{crs} \leftarrow H_{\text{real}}} [\mathcal{A}(\text{crs}) = 1] \right| \leq \text{negl}(\lambda).$$

Step 3: Analyzing soundness. Since we have now reached a hybrid H_{ideal} that does not contain any proofs for instances $x^* \notin L$, we can now analyze soundness just as we did in the one-hop case.

Remark 2.5. A reader familiar with [DWW24] might ask why their selector PRF technique to obtain adaptively sound fully succinct BARGs for NP does not help us in achieving fully succinct IVC for all of NP.

First, as mentioned earlier, the verifier in their BARG construction receives the full batch statement $x = (x_1, \dots, x_k)$ in addition to the proof π and thus can compute a somewhere extractable hash on x (which grows with the size of one instance x_i). In their security analysis, it was important for them to extract this hash to determine if the prover is giving an “off-path” proof in order to implement their “selector PRF”

strategy. However, while this hash is very important for soundness, it is not counted towards the proof size of the BARG because the verifier can compute it themselves from x .

In contrast, in the IVC setting, the verifier only receives the endpoints of the computation and thus cannot compute this hash over all the intermediate states. Thus, such a hash would have to be included in the proof, making the proof no longer fully succinct.

However, we do not believe there is any inherent barrier to achieving fully succinct IVC for NP. We leave this as a fascinating open problem, and hope that our construction and techniques pave a pathway towards such a construction.

3 Preliminaries

Throughout, we will use λ to denote the security parameter.

- We say that a function $f(\lambda)$ is negligible in λ if $f(\lambda) = \lambda^{-\omega(1)}$, and we denote it by $f(\lambda) = \text{negl}(\lambda)$.
- We say that a function $g(\lambda)$ is polynomial in λ if $g(\lambda) = p(\lambda)$ for some fixed polynomial p , and we denote it by $g(\lambda) = \text{poly}(\lambda)$.
- For $n \in \mathbb{N}$, we use $[n]$ to denote $\{1, \dots, n\}$.
- If R is a random variable, then $r \leftarrow R$ denotes sampling r from R . If T is a set, then $i \leftarrow T$ denotes sampling i uniformly at random from T .

Security Notions. We will use the parameters (s, ϵ) to describe the security parameters of our primitive. In particular, we say that a primitive is (s, ϵ) -secure if for all sufficiently large λ and all adversaries $\mathcal{A}$ of size $s(\lambda)$, the advantage of $\mathcal{A}$ is $\epsilon(\lambda)$.

- **Polynomial Security:** We say that a primitive is polynomially secure if it is $(s, \text{negl}(\lambda))$ -secure for all polynomials $s = s(\lambda)$. We denote this by saying it is $(\text{poly}(\lambda), \text{negl}(\lambda))$ -secure.
- **Sub-exponential Security:** We say that a primitive is sub-exponentially secure³ if it is $(s, 2^{-\lambda^c})$ -secure for some $0 < c < 1$ and for all polynomials $s = s(\lambda)$. We denote this by saying it is $(\text{poly}(\lambda), 2^{-\lambda^c})$ -secure.

3.1 Incrementally Verifiable Computation (IVC)

Definition 3.1 (IVC Language). We define the IVC language $\mathcal{L}_{\text{IVC}}$ to be

$$\mathcal{L}_{\text{IVC}} = \left\{ (\mathcal{C}, x_0, x_t, t) : \begin{array}{l} C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n, x_0, x_t \in \{0, 1\}^n \\ \exists (x_1, \dots, x_{t-1}, w_1, \dots, w_t) \text{ such that } \forall i \in [t], C(x_i, w_i) = x_{i+1} \end{array} \right\}$$

We define $\mathcal{L}_C$ to be the restriction of $\mathcal{L}_{\text{IVC}}$ to circuit $C = C$, i.e.

$$\mathcal{L}_C = \{(x_0, x_i, i) : (C, x_0, x_i, i) \in \mathcal{L}_{\text{IVC}}\}$$

We define $\mathcal{L}_{C,i}$ to be the restriction of $\mathcal{L}_{\text{IVC}}$ to circuit $C = C$ and iteration bound $t = i$, i.e.

$$\mathcal{L}_{C,i} = \{(x_0, x_i) : (C, x_0, x_i, i) \in \mathcal{L}_{\text{IVC}}\}$$

³Note that in some settings, sub-exponential security refers to the case where the primitive is $(2^{\lambda^c}, 2^{-\lambda^c})$ -secure for some $0 < c < 1$. We do not need this stronger notion in our work.

Definition 3.2 (IVC for $\mathcal{L}_{\text{IVC}}$ (adapted from [HAN23])). An incrementally verifiable computation (IVC) scheme for $\mathcal{L}_{\text{IVC}}$ is a tuple of PPT algorithms $\text{IVC} = (\text{IVC.Gen}, \text{IVC.P}, \text{IVC.V})$ where

- $\text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C)$: takes as input the security parameter λ , an iteration bound T in binary, an input length n , a witness length m , and a circuit $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, and outputs a common reference strings crs .
- $\text{IVC.P}(\text{crs}, (x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$: takes as input the common reference string crs , inputs $x_0, x_{i-1} \in \{0, 1\}^n$, an index $i-1 \in [T-1]$, a proof π_{i-1} , and a witness $w_i \in \{0, 1\}^m$, and outputs a proof π_i .
- $\text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t)$: takes as input the common reference string crs , inputs $x_0, x_t \in \{0, 1\}^n$, an index $t \in [T]$, and a proof π_t , and outputs a value in $\{0, 1\}$.

The scheme is additionally required to satisfy completeness and efficiency as defined below.

Completeness: For all $\lambda \in \mathbb{N}$, all iteration bounds $T \leq 2^\lambda$, all $t \leq T$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, all $x_0 \in \{0, 1\}^n$, and all $w_1, \dots, w_t \in \{0, 1\}^m$,

$$\Pr \left[\begin{array}{l} \text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C) \\ \forall i \in [t] : x_i = C(x_{i-1}, w_i) \\ \forall i \in [t] : \\ \pi_i \leftarrow \text{IVC.P}(\text{crs}, (x_0, x_{i-1}, i-1), \pi_{i-1}, w_i) \end{array} : \text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t) = 1 \right] = 1.$$

Efficiency: There exists polynomials p_1 and p_2 such that on setup parameters $(1^\lambda, T, 1^n, 1^m, C)$ where C is a polynomial-sized circuit $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, all algorithms run in time $p_1(\lambda, \log T, |C|)$ and the proof size is $p_2(\lambda, n, m, \log T, \log |C|)$.

We say that the scheme satisfies (adaptive) (s, ϵ) -soundness if the following holds:

(s, ϵ) -**Soundness:** For all sufficiently large λ , all iteration bounds $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, and all $s = s(\lambda)$ -sized adversaries $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} \text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t) = 1 \\ (C, x_0, x_t, t) \notin \mathcal{L}_{\text{IVC}} \\ t \leq T \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C) \\ (x_0, x_t, t, \pi_t) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \epsilon(\lambda)$$

Additionally, we say that the scheme satisfies (adaptive) (s, ϵ) -zero knowledge if the following holds:

(s, ϵ) -**(Adaptive) Zero Knowledge:** There exists a PPT simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ such that for all sufficiently large λ , all iteration bounds $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, and all $s = s(\lambda)$ -sized adversaries $\mathcal{A}$,

$$\left| \Pr \left[\begin{array}{l} \mathcal{A}_2(\text{crs}, \pi_t, \text{st}_{\mathcal{A}}) = 1 \\ t \leq T \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C) \\ (x_0, w_1, w_2, \dots, w_t, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(\text{crs}) \\ \forall i \in [t], x_i = C(x_{i-1}, w_i), \\ \forall i \in [t], \pi_i \leftarrow \text{IVC.P}(\text{crs}, (x_0, x_{i-1}, i-1), \pi_{i-1}, w_i) \end{array} \right] - \Pr \left[\begin{array}{l} \mathcal{A}_2(\text{crs}, \pi_t, \text{st}_{\mathcal{A}}) = 1 \\ t \leq T \end{array} : \begin{array}{l} (\text{crs}, \text{st}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^\lambda, T, 1^n, 1^m, C) \\ (x_0, w_1, w_2, \dots, w_t, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(\text{crs}) \\ \forall i \in [t], x_i = C(x_{i-1}, w_i), \\ \pi_t \leftarrow \text{Sim}_2(x_0, x_t, 1^t, \text{st}_{\text{Sim}}) \end{array} \right] \right| \leq \epsilon(\lambda)$$

Remark 3.3. We may use the phrase IVC for NP to refer to IVC for $\mathcal{L}_{\text{IVC}}$

Definition 3.4 (IVC for NP with Witness-Independent Efficiency.). We say that an IVC for NP has witness-independent efficiency if on setup parameters $(1^\lambda, T, 1^n, 1^m, C)$, the proofs are of size $|\pi_i| = \text{poly}(\lambda, n, \log T, \log |C|)$.

Definition 3.5 (IVC for NP with Full Succinctness.). We say that an IVC for NP is fully succinct if on setup parameters $(1^\lambda, T, 1^n, 1^m, C)$, the proofs are of size $|\pi_i| = \text{poly}(\lambda, \log T, \log |C|)$.

Remark 3.6 (IVC for Trapdoor Languages). We define IVC for trapdoor languages in Section 5.1.

3.2 Succinct Non-Interactive Arguments (SNARGs) and Batch Arguments (BARGs)

Definition 3.7 (Boolean Circuit Satisfiability). We define the Boolean circuit satisfiability language $\mathcal{L}_{\text{SAT}}$ as

$$\mathcal{L}_{\text{SAT}} = \left\{ (C, x) : \begin{array}{l} C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}, x \in \{0, 1\}^n, \\ \exists w \in \{0, 1\}^m, C(x, w) = 1 \end{array} \right\}$$

Definition 3.8 (SNARG for $\mathcal{L}_{\text{SAT}}$). A succinct non-interactive argument (SNARG) for $\mathcal{L}_{\text{SAT}}$ is a tuple of PPT algorithms $\text{SNARG} = (\text{SNARG.Gen}, \text{SNARG.P}, \text{SNARG.V})$ where

- $\text{SNARG.Gen}(1^\lambda, 1^n, 1^m, \mathcal{R})$: takes as input the security parameter λ , an input length n , a witness length m , and a circuit $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, and outputs a pair of common reference strings $\text{crs} = (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}})$.
- $\text{SNARG.P}(\text{crs}_{\mathcal{P}}, x, w)$: takes as input the prover common reference string $\text{crs}_{\mathcal{P}}$, an input $x \in \{0, 1\}^n$, and a witness $w \in \{0, 1\}^m$, and outputs a proof π .
- $\text{SNARG.V}(\text{crs}_{\mathcal{V}}, x, \pi)$: takes as input the verifier common reference string $\text{crs}_{\mathcal{V}}$, an input $x \in \{0, 1\}^n$, and a proof π , and outputs a value in $\{0, 1\}$.

The scheme is additionally required to satisfy completeness and efficiency as defined below.

Completeness: For all $\lambda \in \mathbb{N}$, all polynomials $n = n(\lambda), m = m(\lambda)$, all polynomial-sized circuits $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, all $x \in \{0, 1\}^n$, and all $w \in \{0, 1\}^m$ such that $\mathcal{R}(x, w) = 1$,

$$\Pr \left[\text{SNARG.V}(\text{crs}_{\mathcal{V}}, x, \pi) = 1 : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \text{SNARG.Gen}(1^\lambda, 1^n, 1^m, \mathcal{R}), \\ \pi \leftarrow \mathcal{P}(\text{crs}_{\mathcal{P}}, x, w) \end{array} \right] = 1.$$

Efficiency: There exists a polynomial p such that on setup parameters $(1^\lambda, 1^n, 1^m, \mathcal{R})$ where $\mathcal{R}$ is a polynomial-sized Boolean circuit $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, the proof has size $|\pi| = p(\lambda, \log |R|)$. Moreover, the verifier runtime is $\text{poly}(\lambda, n, \log |R|)$.⁴

A SNARG may satisfy either adaptive or non-adaptive soundness. We define both below:

(s, ϵ) -Non-Adaptive Soundness: For all sufficiently large λ and all $s = s(\lambda)$ -sized adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,

$$\Pr \left[\begin{array}{l} (R, x^*) \notin \mathcal{L}_{\text{SAT}} \\ \wedge \text{V}(\text{crs}_{\mathcal{V}}, x^*, \pi^*) = 1 \end{array} : \begin{array}{l} (1^n, 1^m, \mathcal{R}, x^*, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda), \\ (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \text{SNARG.Gen}(1^\lambda, 1^n, 1^m, \mathcal{R}), \\ \pi^* \leftarrow \mathcal{A}_2(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}, x^*, \text{st}) \end{array} \right] \leq \epsilon(\lambda).$$

⁴Note that this verifier efficiency can be generically obtained via RAM delegation techniques [CJJ22, KLVW23]. See Remark 2.7 of [WW24b].

(s, ϵ) -Adaptive Soundness: For all sufficiently large λ and all $s = s(\lambda)$ -sized adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,

$$\Pr \left[\begin{array}{l} (R, x) \notin \mathcal{L}_{\text{SAT}} \\ \wedge \mathcal{V}(\text{crs}_{\mathcal{V}}, x^*, \pi^*) = 1 \end{array} : \begin{array}{l} (1^n, 1^m, \mathcal{R}, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda) \\ (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \text{SNARG.Gen}(1^\lambda, 1^n, 1^m, \mathcal{R}), \\ (x^*, \pi^*) \leftarrow \mathcal{A}_2(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}, \text{st}) \end{array} \right] \leq \epsilon(\lambda).$$

Theorem 3.9 ([SW14, WW24d]). Assuming the subexponential $i\mathcal{O}$ and one-way functions, there exist adaptively sound SNARGs with crs size $\text{poly}(\lambda, |R|)$ and proof size $\text{poly}(\lambda)$.

Definition 3.10 (seBARG for $\mathcal{L}_{\text{SAT}}$). A (publicly verifiable and non-interactive) somewhere extractable batch argument (seBARG) for $\mathcal{L}_{\text{SAT}}$ is a tuple of PPT algorithms

$$\text{seBARG} = (\text{seBARG.Gen}, \text{seBARG.P}, \text{seBARG.V}, \text{seBARG.TDGen}, \text{seBARG.Ext})$$

where

- $\text{seBARG.Gen}(1^\lambda, k, 1^n, 1^m, R)$: takes as input the security parameter λ , a number of instances k in binary, an input length n , a witness length m , and a circuit $R : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, and outputs a common reference strings crs .
- $\text{seBARG.P}(\text{crs}, \{x_i\}_{i \in [k]}, \{w_i\}_{i \in [k]})$: takes as input a common reference string crs , statements $\{x_i\}_{i \in [k]}$ where each $x_i \in \{0, 1\}^n$, and witnesses $\{w_i\}_{i \in [k]}$ where each $w_i \in \{0, 1\}^m$, and outputs a proof π .
- $\text{seBARG.V}(\text{crs}, \{x_i\}_{i \in [k]}, \pi)$: takes as input a common reference string crs , statements $\{x_i\}_{i \in [k]}$ where each $x_i \in \{0, 1\}^n$, and a proof π , and outputs a value in $\{0, 1\}$.
- $\text{seBARG.TDGen}(1^\lambda, k, 1^n, 1^m, \mathcal{R}, i^*)$: takes as input the security parameter λ , a number of instances k in binary, an input length n , a witness length m , an NP-relation circuit $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, and an index $i^* \in [k]$, and outputs a common reference strings crs and a trapdoor td .
- $\text{seBARG.Ext}(\text{td}, \{x_i\}_{i \in [k]}, \pi)$: takes as input a trapdoor td , statements $\{x_i\}_{i \in [k]}$ where each $x_i \in \{0, 1\}^n$, and a proof π , and outputs a witness $w \in \{0, 1\}^m$.

The scheme is additionally required to satisfy completeness and efficiency as defined below.

Completeness: For all $\lambda \in \mathbb{N}$, all $k \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, all polynomial-sized Boolean circuits $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, all $x_1, \dots, x_k \in \{0, 1\}^n$, all $w_1, \dots, w_k \in \{0, 1\}^m$ where for each $i \in [k]$, $C(x_i, w_i) = 1$,

$$\Pr \left[\text{seBARG.V}(\text{crs}, \{x_i\}_{i \in [k]}, \pi) = 1 : \begin{array}{l} \text{crs} \leftarrow \text{seBARG.Gen}(1^\lambda, k, 1^n, 1^m, \mathcal{R}), \\ \pi \leftarrow \mathcal{P}(\text{crs}, \{x_i\}_{i \in [k]}, \{w_i\}_{i \in [k]}) \end{array} \right] = 1.$$

Efficiency: There exists a polynomial p such that on setup parameters $(1^\lambda, k, 1^n, 1^m, \mathcal{R})$ where $\mathcal{R}$ is a polynomial-sized Boolean circuit $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, the proof has size $|\pi| = m \cdot p(\lambda, \log k, \log |R|)$.

We say that an seBARG is (s, ϵ) -secure if it satisfies (s, ϵ) -trapdoor indistinguishability and (s, ϵ) -somewhere argument of knowledge as defined below:

(s, ϵ) -Index Hiding For all sufficiently large λ , for all $s = s(\lambda)$ -sized adversaries $\mathcal{A}$, for all $k \leq 2^\lambda$, and for all $i_0, i_1 \in [k]$,

$$\Pr \left[\mathcal{A}(\text{crs}, \text{st}) = b : \begin{array}{l} b \leftarrow \{0, 1\} \\ (\text{crs}, \text{td}) \leftarrow \text{seBARG.TDGen}(1^\lambda, k, 1^n, 1^m, C, i_b) \end{array} \right] \leq \frac{1}{2} + \epsilon(\lambda)$$

(s, ϵ) -Somewhere Argument of Knowledge. For all sufficiently large λ , all $s = s(\lambda)$ -sized adversaries $\mathcal{A}$, all $k \leq 2^\lambda$, and all $i^* \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{seBARG.V}(\text{crs}, \{x_i\}_{i \in [k]}, \pi) = 1 \\ \mathcal{R}(x_{i^*}, w_{i^*}) = 0 \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \text{Setup}(\lambda, k, 1^n, 1^m, C, i^*) \\ (\{x_i\}_{i \in [k]}, \pi) \leftarrow \mathcal{A}(\text{crs}, \text{st}) \\ w_{i^*} \leftarrow \text{seBARG.Ext}(\text{td}, \{x_i\}_{i \in [k]}, \pi) \end{array} \right] \leq \epsilon(\lambda)$$

Theorem 3.11. Constructions of seBARGs for $\mathcal{L}_{\text{SAT}}$ can be constructed from:

- **LWE** [CJJ21].
- **Bilinear maps** [WW22, KLVW23].
- **sub-exponential DDH** [CGG⁺21].
- **sub-exponential DDH and quadratic residuosity** [HJKS22, KLVW23].

Definition 3.12 (Rate-1 seBARG for $\mathcal{L}_{\text{SAT}}$). We say that an seBARG for NP is rate-1 if there exists a polynomial p such that on setup parameters $(1^\lambda, k, 1^n, 1^m, \mathcal{R})$ where $\mathcal{R}$ is a polynomial-sized Boolean circuit $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, the proof has size $|\pi| = m + p(\lambda, \log k, \log |R|)$.

Theorem 3.13. Rate-1 seBARGs for $\mathcal{L}_{\text{SAT}}$ can be constructed from

- **polynomial LWE** [DGKV22]
- **polynomial DDH and any seBARG** [BDSZ24]. Combined with Theorem 3.11, this can therefore be instantiated from either bilinear maps, or subexponential DDH.

Remark 3.14. Since we need subexponentially secure seBARGs, we need to rely on subexponential underlying assumptions.

3.3 Non-interactive Zero-Knowledge (NIZK)

Definition 3.15 (NIZK Proof of Knowledge for NP). A non-interactive zero-knowledge argument system (NIZK) proof of knowledge for NP is a tuple of PPT algorithms $\text{NIZK} = (\text{NIZK.Gen}, \text{NIZK.P}, \text{NIZK.V})$

- $\text{NIZK.Gen}(1^\lambda, 1^n, 1^m, \mathcal{R})$: takes as input the security parameter λ , an input length n , a witness length m , and an NP-relation circuit $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, and outputs a common reference string crs .
- $\text{NIZK.P}(\text{crs}, x, w)$: takes as input the common reference string crs , an input $x \in \{0, 1\}^n$, and a witness $w \in \{0, 1\}^m$, and outputs a proof π .
- $\text{NIZK.V}(\text{crs}, x, \pi)$: takes as input the common reference string crs , an input $x \in \{0, 1\}^n$, and a proof π , and outputs a value in $\{0, 1\}$.

We additionally require the following properties.

(Perfect) Completeness: For all $\lambda \in \mathbb{N}$, all polynomials $n = n(\lambda), m = m(\lambda)$, all polynomial-sized circuits $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}$, all $x \in \{0, 1\}^n$ and $w \in \{0, 1\}^m$ such that $\mathcal{R}(x, w) = 1$,

$$\Pr \left[\text{NIZK.V}(\text{crs}, x, \pi) = 1 : \begin{array}{l} \text{crs} \leftarrow \text{NIZK.Gen}(1^\lambda, 1^n, 1^m, \mathcal{R}), \\ \pi \leftarrow \text{NIZK.P}(\text{crs}, x, w) \end{array} \right] = 1$$

(s, ϵ) -Knowledge Extraction: *There exists a PPT extractor Ext such that for all sufficiently large λ , all $s = s(\lambda)$ -sized adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,*

$$\Pr \left[\begin{array}{l} \text{NIZK.V}(\text{crs}, x, \pi) = 1 \\ \wedge \mathcal{R}(x, w^*) = 0 \end{array} : \begin{array}{l} (1^n, 1^m, \mathcal{R}, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda) \\ (\text{crs}, \text{td}) \leftarrow \text{NIZK.TDGen}(1^\lambda, 1^n, 1^m, \mathcal{R}) \\ (x, \pi) \leftarrow \mathcal{A}_2(\text{crs}, \text{st}) \\ w^* \leftarrow \text{NIZK.Ext}(\text{td}, x, \pi) \end{array} \right] \leq \mu(\lambda)$$

(s, ϵ) -(Adaptive) Zero-Knowledge: *There exists a PPT simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2, \text{Sim}_3)$ such that for all sufficiently large λ , all polynomials $n = n(\lambda), m = m(\lambda)$, all $x \in \{0, 1\}^n$ and $w \in \{0, 1\}^m$ such that $\mathcal{R}(x, w) = 1$, all $s = s(\lambda)$ -sized adversaries $\mathcal{A}$,*

$$\left| \Pr \left[\mathcal{A}^{\mathcal{P}(\text{crs}, \cdot, \cdot)}(\text{crs}) : \text{crs} \leftarrow \text{NIZK.Gen}(1^\lambda, 1^n, 1^m, \mathcal{R}), \right] - \Pr \left[\mathcal{A}^{\text{Sim}_2(\text{st}, \cdot, \cdot)} : (\text{crs}, \text{st}) \leftarrow \text{Sim}_1(1^\lambda, 1^n, 1^m, \mathcal{R}) \right] \right| \leq \epsilon(\lambda)$$

where

- $\mathcal{P}(\text{crs}, x, w)$: If $\mathcal{R}(x, w) = 0$, output $\perp$. Else, output $\pi \leftarrow \text{NIZK.P}(\text{crs}, x, w)$.
- $\text{Sim}_2(\text{st}, x, w)$: If $\mathcal{R}(x, w) = 0$, output $\perp$. Else, output $\pi \leftarrow \text{Sim}_3(\text{st}, x, w)$.

Theorem 3.16. *There exists statistical sound NIZK proof of knowledge for NP with adaptive zero-knowledge assuming either*

- bilinear maps [GOS06].
- LWE [PS19].

3.4 Public-Key Encryption (PKE)

Definition 3.17 (Public-Key Encryption (PKE)). *A public-key encryption scheme with ciphertext size $m(\cdot)$ is a tuple of PPT algorithms $\text{PKE} = (\text{PKE.Gen}, \text{PKE.Enc}, \text{PKE.Dec})$ where*

- $\text{PKE.Setup}(1^\lambda, 1^n)$ is a randomized algorithm that takes as input the security parameter λ and an input length n and outputs a public key pk and a secret key sk .
- $\text{PKE.Enc}(\text{pk}, x)$ is a randomized algorithm that takes as input a public key pk and a message $x \in \{0, 1\}^n$ and outputs an encryption $\text{ct} \in \{0, 1\}^{m(\lambda, n)}$ of x .
- $\text{PKE.Dec}(\text{sk}, \text{ct})$ is a deterministic algorithm that takes as input a secret key sk and a ciphertext $\text{ct} \in \{0, 1\}^{m(\lambda, n)}$ and outputs a value $y \in \{0, 1\}^n$.

(Perfect) Correctness: *For all $\lambda, n \in \mathbb{N}$ and every $x \in \{0, 1\}^n$,*

$$\Pr \left[\text{PKE.Dec}(\text{sk}, \text{PKE.Enc}(\text{pk}, x)) = x : (\text{pk}, \text{sk}) \leftarrow \text{PKE.Setup}(1^\lambda, 1^n) \right] = 1$$

(s, ϵ) -Security: *For all sufficiently large $\lambda \in \mathbb{N}$ and all $s = s(\lambda)$ -sized adversaries $\mathcal{A}$,*

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{PKE}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{PKE}}(1^\lambda, 1) = 1] \right| \leq \epsilon(\lambda)$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, we define

$\text{Expt}_{\mathcal{A}}^{\text{PKE}}(1^\lambda, b)$

1. **Parameters:** $\mathcal{A}$ takes as input 1^λ and outputs an input size 1^n .
2. **Setup:** $(\text{pk}, \text{sk}) \leftarrow \text{PKE.Setup}(1^\lambda, 1^n)$
3. **Challenge Message Query:**
 - (a) $\mathcal{A}$ outputs a challenge message pair (x_0, x_1) where $x_0, x_1 \in \{0, 1\}^n$.
 - (b) $\text{ct}_b \leftarrow \text{PKE.Enc}(\text{pk}, x_b)$
 - (c) Sent ct_b to $\mathcal{A}$.
4. **Experiment Outcome:** $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

3.5 Indistinguishability Obfuscation

Definition 3.18 (Indistinguishability obfuscation ($i\mathcal{O}$), [BGI⁺01]). An indistinguishability obfuscator $i\mathcal{O}$ for a circuit class $\mathcal{C} = \{\mathcal{C}_\lambda\}_\lambda$ is a PPT algorithm that satisfies the following conditions:

- **Correctness:** For all security parameters $\lambda \in \mathbb{N}$, for all $C \in \mathcal{C}_\lambda$, for all inputs x , we have that

$$\Pr[C' \leftarrow i\mathcal{O}(1^\lambda, C) : C'(x) = C(x)] = 1.$$

- **(s, ϵ) -Security:** For all pairs of circuits $C_0, C_1 \in \mathcal{C}_\lambda$ such that $C_0(x) = C_1(x)$ for all x , for all $s = s(\lambda)$ -sized adversaries $\mathcal{A}$,

$$\left| \Pr[\mathcal{A}(i\mathcal{O}(1^\lambda, C_0)) = 1] - \Pr[\mathcal{A}(i\mathcal{O}(1^\lambda, C_1)) = 1] \right| \leq \epsilon(\lambda)$$

3.6 Puncturable Pseudorandom Functions

Definition 3.19 (Puncturable PRF). A puncturable PRF family on key space $\mathcal{K} = \{\mathcal{K}_\lambda\}_\lambda$, domain $\mathcal{X} = \{\mathcal{X}_\lambda\}_\lambda$, output space $\mathcal{Y} = \{\mathcal{Y}_\lambda\}_\lambda$ and polynomial $t(\lambda)$ is a tuple of PPT algorithms $(\text{PPRF.Gen}, \text{PPRF.Punc}, \text{PPRF.Eval})$ with the following interface and properties.

- $\text{PPRF.Gen}(1^\lambda)$: On input the security parameter 1^λ , outputs a key $k \in \mathcal{K}_\lambda$.
- $\text{PPRF.Punc}(k, T)$: On input a key $k \in \mathcal{K}_\lambda$ and a set $T \subseteq \mathcal{X}_\lambda$ such that $|T| \leq t(\lambda)$, outputs a new key $k(\{T\}) \in \mathcal{K}_\lambda$.
- $\text{PPRF.Eval}(k, x)$: On input $k \in \mathcal{K}_\lambda$ and $x \in \mathcal{X}_\lambda$, outputs some value $y \in \mathcal{Y}_\lambda$.
- **Functionality preserving:** For all sets $T \subseteq \mathcal{X}$ of size $|T| \leq t(\lambda)$, for all $x \notin T$,

$$\Pr[k \leftarrow \text{Gen}(1^\lambda), k\{T\} \leftarrow \text{Punc}(k, T) : \text{Eval}(k\{T\}, x) = \text{Eval}(k, x)] = 1.$$

- **(s, ϵ) -pseudorandomness at punctured points:** Consider the following experiment $\text{Expt}_{\mathcal{A}}^{\text{PPRF}}(1^\lambda, b)$ between a challenger $\mathcal{C}$ and adversary $\mathcal{A}$:

- First, $\mathcal{A}$ chooses a set $T \subseteq \mathcal{X}_\lambda$ such that $|T| \leq t(\lambda)$, and sends it to $\mathcal{C}$.
- $\mathcal{C}$ samples $k \leftarrow \text{PPRF.Gen}(1^\lambda)$, and computes $k\{T\} \leftarrow \text{PPRF.Punc}(k, T)$.
 - * If $b = 0$, compute $L_0 = \{x_i, \text{PPRF.Eval}(k, x_i)\}_{x_i \in T}$.
 - * If $b = 1$, compute $L_1 = \{x_i, y_i\}_{x_i \in T}$ where each $y_i \leftarrow \mathcal{Y}$.

Give $k\{T\}, L_b$ to $\mathcal{A}$.

- $\mathcal{A}$ produces a bit b' .

We say that the PPRF is pseudorandom at punctured points if for all size $s = s(\lambda)$ adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Expt}_{\mathcal{A}}^{\text{PKE}}(1^\lambda, 0) = 1] - \Pr[\text{Expt}_{\mathcal{A}}^{\text{PKE}}(1^\lambda, 1) = 1] \right| \leq \epsilon(\lambda)$$

Theorem 3.20 ([GGM84, BW13, BGI14, KPTZ13]). Assuming the existence of polynomially secure (resp. sub-exponentially secure) one-way functions, then there exists polynomially secure (resp. sub-exponentially secure) puncturable PRFs.

4 IVC for NP

In this section, we present our construction of an IVC for NP with *witness independent efficiency* from a subexponentially secure PKE, a subexponentially secure SNARG, a subexponentially secure NIZK proof of knowledge, and a subexponentially secure rate-1 seBARG.

Theorem 4.1. Assume the following ingredients:

- $(\text{poly}(\lambda_{\text{PKE}}), 2^{-\lambda_{\text{PKE}}^{\delta_{\text{PKE}}}})$ -secure perfectly correct PKE,
- $(\text{poly}(\lambda_{\text{SNARG}}), 2^{-\lambda_{\text{SNARG}}^{\delta_{\text{SNARG}}}})$ -adaptively sound SNARG for NP,
- NIZK Proof of Knowledge for NP with perfect completeness, $(\text{poly}(\lambda_{\text{NIZK}}), 2^{-\lambda_{\text{NIZK}}^{\delta_{\text{NIZK}}}})$ -knowledge extraction and adaptive zero knowledge,
- Rate-1 seBARG for NP with $(\text{poly}(\lambda_{\text{seBARG}}), 2^{-\lambda_{\text{seBARG}}^{\delta_{\text{seBARG}}}})$ -index hiding and somewhere argument of knowledge,

where $0 < \delta_{\text{PKE}}, \delta_{\text{SNARG}}, \delta_{\text{NIZK}}, \delta_{\text{seBARG}} < 1$ are any constants and $\lambda_{\text{PKE}}, \lambda_{\text{SNARG}}, \lambda_{\text{NIZK}}, \lambda_{\text{seBARG}} = \text{poly}(\lambda, \log T, |C|)$ are as defined in Section 4.1. Then, there exists an IVC for NP for circuits of size $|C|$, inputs of size n , and time bound T with

- $(\text{poly}(\lambda), \text{negl}(\lambda))$ -soundness,
- $(\text{poly}(\lambda), \text{negl}(\lambda))$ -adaptive zero knowledge,
- crs of size $\text{poly}(\lambda, \log T, |C|)$,
- proofs of size $\text{poly}(\lambda, n, \log T, \log |C|)$ (i.e. witness-independent efficiency).

Remark 4.2. We remark that in Theorem 4.1, if we replace the adaptively sound SNARG for NP with the “trivial” proof system which simply sends the witness w as its proof, then we get an IVC for NP with the same properties as above except that the proof sizes will additionally depend on the witness size m .

4.1 Parameters

We will build our scheme using the primitives specified in Theorem 4.1.

On security parameter λ , iteration bound T , input size n , witness size m , and circuit C , we will instantiate our primitives with the following parameters. Observe that we will create $\lceil \log T \rceil$ instantiations of our seBARG scheme which will correspond to different “levels” of our proof.

	Security Parameter	Number of Instances	Input size (per instance)	Witness Size	Circuit
PKE	λ_{PKE}		n		
SNARG	λ_{SNARG}		$2n$	m	$\mathcal{R}_{C,1}$
NIZK (level 0)	λ_{NIZK}		$n_0 = 2 \text{ct} $	$m_0 = 2n + 2\lambda_{\text{PKE}} + \pi_{\text{SNARG}} $	$\mathcal{R}_{0,\text{pk},\text{crs}_{\text{SNARG}},\nu}$
seBARG at level 1	λ_{seBARG}	$k_1 = 2$	$n_1 = 2 \text{ct} $	$m_1 = \pi^{(0)} $	$\mathcal{R}_{1,\text{crs}_{\text{NIZK}}}$
seBARG at level ℓ for $2 \leq \ell \leq \lceil \log T \rceil$	λ_{seBARG}	$k_\ell = 2$	$n_\ell = 2 \text{ct} $	$m_\ell = \pi^{(\ell-1)} + \text{ct} $	$\mathcal{R}_{\ell,\text{crs}_{\ell-1}}$

where $\lambda_{\text{PKE}} = \lambda_{\text{SNARG}} = \lambda_{\text{NIZK}} = \lambda_{\text{seBARG}} = (3n + 2\lambda)^{1/\delta'}$ for some constant $0 < \delta' < \min(\delta_{\text{PKE}}, \delta_{\text{SNARG}}, \delta_{\text{NIZK}}, \delta_{\text{seBARG}})$, and where we define:

	Definition
$ \text{ct} $	size of PKE ciphertexts under the above parameters
$ \pi_{\text{SNARG}} $	size of SNARG proofs under the above parameters
$ \pi^{(0)} $	size of NIZK proofs under the above parameters
$ \pi^{(\ell)} $ for $1 \leq \ell \leq \lceil \log T \rceil$	size of seBARG proofs at level ℓ under the above parameters
$\mathcal{R}_{C,1}$	NP-relation circuit for the language $\mathcal{L}_{C,1}$ defined in Definition 3.1
$\mathcal{R}_{0,\text{pk},\text{crs}_{\text{SNARG}},\nu}$	NP-relation circuit for the language $\mathcal{L}_{0,\text{pk},\text{crs}_{\text{SNARG}},\nu}$ defined in Equation (3) in the construction below where pk is a public key of the PKE scheme and $\text{crs}_{\text{SNARG},\nu}$ is the verifier’s common reference string for the SNARG scheme
$\mathcal{R}_{1,\text{crs}_{\text{NIZK}}}$	NP-relation circuit for the language $\mathcal{L}_{1,\text{crs}_{\text{NIZK}}}$ defined in Equation (4) in the construction below where crs_{NIZK} is the common reference string of the NIZK scheme
$\mathcal{R}_{\ell,\text{crs}_{\ell-1}}$ for $2 \leq \ell \leq \lceil \log T \rceil$	NP-relation circuit for the language $\mathcal{L}_{\ell,\text{crs}_{\ell-1}}$ defined in Equation (5) in the construction below where $\text{crs}_{\ell-1}$ is a common reference string of the level $\ell - 1$ seBARG scheme

We can compute these sizes as follows:

- $|\text{ct}| = \text{poly}(\lambda_{\text{PKE}}, n) = \text{poly}(\lambda, n)$
- $|\mathcal{R}_{C,1}| = \text{poly}(|C|)$
- $|\pi_{\text{SNARG}}| = \text{poly}(\lambda_{\text{SNARG}}, \log |\mathcal{R}_{C,1}|) = \text{poly}(\lambda, \log |C|)$
- We compute

$$\begin{aligned}
|\mathcal{R}_{0,\text{pk},\text{crs}_{\text{SNARG}},\nu}| &= \text{poly}(n_0, m_0, |\text{PKE.Enc}(\text{pk}, \cdot)|, |\text{SNARG.V}(\text{crs}_{\text{SNARG},\nu}, \cdot, \cdot)|) \\
&= \text{poly}(\lambda_{\text{PKE}}, \lambda_{\text{SNARG}}, n, \log |C|) \\
&= \text{poly}(\lambda, n, \log |C|)
\end{aligned}$$

- $|\pi^{(0)}| = \text{poly}(\lambda_{\text{NIZK}}, |\mathcal{R}_{0,\text{pk},\text{crs}_{\text{SNARG}},\nu}|) = \text{poly}(\lambda, n, \log |C|)$

- We compute

$$\begin{aligned}
|\mathcal{R}_{1, \text{crs}_{\text{NIZK}}}| &= \text{poly}(n_1, m_1, |\text{NIZK}.\mathcal{V}(\text{crs}_{\text{NIZK}}, \cdot, \cdot)|) \\
&= \text{poly}(|\text{ct}|, |\pi^{(0)}|, \lambda_{\text{NIZK}}, |\mathcal{R}_{0, \text{pk}, \text{crs}_{\text{SNARG}, \mathcal{V}}}|) \\
&= \text{poly}(\lambda, n, \log |C|)
\end{aligned}$$

- Since the seBARG is rate-1,

$$\begin{aligned}
|\pi^{(1)}| &= m_1 + \text{poly}(\lambda_{\text{seBARG}}, \log k_1, \log |\mathcal{R}_{1, \text{crs}_{\text{NIZK}}}|) \\
&= |\pi^{(0)}| + \text{poly}(\lambda, \log |\mathcal{R}_{1, \text{crs}_{\text{NIZK}}}|) \\
&= \text{poly}(\lambda, n, \log |C|)
\end{aligned}$$

- For $2 \leq \ell \leq \lceil \log T \rceil$,

$$\begin{aligned}
|\mathcal{R}_{\ell, \text{crs}_{\ell-1}}| &= \text{poly}(n_\ell, m_\ell, |\text{seBARG}.\mathcal{V}(\text{crs}_{\ell-1}, \{\cdot, \cdot\}, \cdot)|) \\
&= \text{poly}(|\text{ct}|, |\pi^{(\ell-1)}|, \lambda_{\text{seBARG}}, k_{\ell-1}, |\mathcal{R}_{\ell-1, \text{crs}_{\ell-2}}|) \\
&= \text{poly}(\lambda, n, |\pi^{(\ell-1)}|, |\mathcal{R}_{\ell-1, \text{crs}_{\ell-2}}|)
\end{aligned}$$

- Since the seBARG is rate-1, for $2 \leq \ell \leq \lceil \log T \rceil$,

$$\begin{aligned}
|\pi^{(\ell)}| &= m_\ell + \text{poly}(\lambda_{\text{seBARG}}, \log k_\ell, \log |\mathcal{R}_{\ell, \text{crs}_{\ell-1}}|) \\
&= |\pi^{(\ell-1)}| + |\text{ct}| + \text{poly}(\lambda, \log |\mathcal{R}_{\ell, \text{crs}_{\ell-1}}|) \\
&= |\pi^{(1)}| + (\ell - 1) \cdot |\text{ct}| + \sum_{k=2}^{\ell} \text{poly}(\lambda, \log |\mathcal{R}_{k, \text{crs}_{k-1}}|) \\
&= \text{poly}(\lambda, n, \ell, \log |C|)
\end{aligned}$$

We additionally assume that on security parameter λ_{PKE} , the PKE encryption algorithm uses randomness of length λ_{PKE} .

Remark 4.3. If we replace our SNARG with the “trivial” proof system which consists of simply sending the witness w , then our proof sizes will additionally depend on the witness size m .

4.2 Construction

We now construct our IVC for NP using the parameters defined in the previous section.

- $\text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C)$:

1. Define $\ell_{\max} = \lceil \log T \rceil$.

2. **Setup SNARG to verify one hop:**

- (a) Let $\mathcal{R}_{C,1}$ be the NP-relation circuit for the language $\mathcal{L}_{C,1}$ defined in Definition 3.1.
- (b) $(\text{crs}_{\text{SNARG}, \mathcal{P}}, \text{crs}_{\text{SNARG}, \mathcal{V}}) \leftarrow \text{SNARG.Gen}(1^{\lambda_{\text{SNARG}}}, 1^{2n}, 1^m, \mathcal{R}_{C,1})$.

3. **Setup NIZK at level 0 to prove knowledge of a SNARG for one hop:**

- (a) $(\text{pk}, \text{sk}) \leftarrow \text{PKE.Gen}(1^{\lambda_{\text{PKE}}}, 1^n)$.

(b) Define

$$\mathcal{L}_{0,\text{pk},\text{crs}_{\text{SNARG},\mathcal{V}}} = \left\{ (\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}) : \begin{array}{l} \exists (x_{\text{init}}, x_{\text{fin}}, r_{\text{init}}, r_{\text{fin}}, \pi_{\text{SNARG}}) \text{ such that} \\ \text{SNARG}.\mathcal{V}(\text{crs}_{\text{SNARG},\mathcal{V}}, (x_{\text{init}}, x_{\text{fin}}), \pi_{\text{SNARG}}) = 1 \\ \wedge \text{ct}_{\text{init}} = \text{PKE}.\text{Enc}(\text{pk}, x_{\text{init}}; r_{\text{init}}) \\ \wedge \text{ct}_{\text{fin}} = \text{PKE}.\text{Enc}(\text{pk}, x_{\text{fin}}; r_{\text{fin}}) \end{array} \right\} \quad (3)$$

Let $\mathcal{R}_{0,\text{pk},\text{crs}_{\text{SNARG},\mathcal{V}}}$ be the NP-relation circuit for the language $\mathcal{L}_{0,\text{pk},\text{crs}_{\text{SNARG},\mathcal{V}}}$.

(c) $(\text{crs}_{\text{NIZK}}, \text{td}_{\text{NIZK}}) \leftarrow \text{NIZK}.\text{Gen}(1^{\lambda_{\text{NIZK}}}, 1^{n_0}, 1^{m_0}, \mathcal{R}_{0,\text{pk},\text{crs}_{\text{SNARG},\mathcal{V}}})$.

4. **Setup seBARG at level 1 to verify a batch of two NIZK proofs:**

(a) Define

$$\mathcal{L}_{1,\text{crs}_{\text{NIZK}}} = \left\{ (\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}) : \begin{array}{l} \exists \pi^{(0)} \text{ such that} \\ \text{NIZK}.\mathcal{V}(\text{crs}_{\text{NIZK}}, (\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}), \pi^{(0)}) = 1 \end{array} \right\}. \quad (4)$$

Let $\mathcal{R}_{1,\text{crs}_{\text{NIZK}}}$ be the NP-relation circuit for the language $\mathcal{L}_{1,\text{crs}_{\text{NIZK}}}$.

(b) $\text{crs}_1 \leftarrow \text{seBARG}.\text{Gen}(1^{\lambda_{\text{seBARG}}}, 2, 1^{n_1}, 1^{m_1}, \mathcal{R}_{1,\text{crs}_{\text{NIZK}}})$.

5. **Setup seBARG at level $\ell > 1$ to verify a batch of two level $\ell - 1$ seBARG proofs:**

For $\ell \in \{2, \dots, \ell_{\max}\}$,

(a) Define

$$\mathcal{L}_{\ell,\text{crs}_{\ell-1}} = \left\{ (\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}) : \begin{array}{l} \exists (\text{ct}_{\text{mid}}, \pi^{(\ell-1)}) \text{ such that} \\ \text{seBARG}.\mathcal{V}(\text{crs}_{\ell-1}, \{(\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}), (\text{ct}_{\text{mid}}, \text{ct}_{\text{fin}})\}, \pi^{(\ell-1)}) = 1 \end{array} \right\}. \quad (5)$$

Let $\mathcal{R}_{\ell,\text{crs}_{\ell-1}}$ be the NP-relation circuit for the language $\mathcal{L}_{\ell,\text{crs}_{\ell-1}}$.

(b) $\text{crs}_{\ell} \leftarrow \text{seBARG}.\text{Gen}(1^{\lambda_{\text{seBARG}}}, 2, 1^{n_{\ell}}, 1^{m_{\ell}}, \mathcal{R}_{\ell,\text{crs}_{\ell-1}})$.

6. **Output $\text{crs} := (\ell_{\max}, C, \text{pk}, \text{crs}_{\text{SNARG},\mathcal{P}}, \text{crs}_{\text{SNARG},\mathcal{V}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_{\ell}\}_{\ell \in [\ell_{\max}]})$.**

• **IVC. $\mathcal{P}(\text{crs}, (x_0, x_{i-1}, i-1), \Pi_{i-1}, w_i)$:**

1. **Parse** $\text{crs} = (\ell_{\max}, C, \text{pk}, \text{crs}_{\text{SNARG},\mathcal{P}}, \text{crs}_{\text{SNARG},\mathcal{V}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_{\ell}\}_{\ell \in [\ell_{\max}]})$.

2. **Compute base case values:** If $i = 1$,

(a) Sample $r_0 \leftarrow \{0, 1\}^{\lambda_{\text{PKE}}}$.

(b) $\text{ct}_0 \leftarrow \text{PKE}.\text{Enc}(\text{pk}, x_0; r_0)$.

(c) For $\ell \in \{0, \dots, \ell_{\max}\}$, define $S^{(\ell)} = \emptyset$.

3. **Verify proof of previous iteration:** If $i > 1$,

(a) Parse $\Pi_{i-1} = (\text{ct}_0, \text{ct}_{i-1}, r_0, r_{i-1}, \{S^{(\ell)}\}_{\ell \in \{0, \dots, \ell_{\max}\}})$.

(b) If $\text{IVC}.\mathcal{V}(\text{crs}, (x_0, x_{i-1}, i-1), \Pi_{i-1}) = 0$, output $\perp$.

4. **Compute NIZK proof $\pi_i^{(0)}$ for iteration i :**

(a) $x_i = C(x_{i-1}, w_i)$.

(b) $\pi_{\text{SNARG}} \leftarrow \text{SNARG}.\mathcal{P}(\text{crs}_{\text{SNARG},\mathcal{P}}, (x_{i-1}, x_i), w_i)$.

(c) Sample $r_i \leftarrow \{0, 1\}^{\lambda_{\text{PKE}}}$.

(d) $\text{ct}_i \leftarrow \text{PKE}.\text{Enc}(\text{pk}, x_i; r_i)$.

(e) $\pi_i^{(0)} \leftarrow \text{NIZK}.\mathcal{P}(\text{crs}_{\text{NIZK}}, (\text{ct}_{i-1}, \text{ct}_i), (x_{i-1}, x_i, r_{i-1}, r_i, \pi_{\text{SNARG}}))$.

(f) Add $(i-1, \text{ct}_{i-1}, \perp, \text{ct}_i, \pi_i^{(0)})$ to $S^{(0)}$.

5. **Merge proofs:** For $\ell \in \{0, \dots, \ell_{\max} - 1\}$,

(a) **Check if there are two proofs at level ℓ :**

- i. If $|S^{(\ell)}| < 2$, continue to the next iteration. If $|S^{(\ell)}| > 2$, output $\perp$.
- ii. If $|S^{(\ell)}| = 2$, parse

$$S^{(\ell)} = \{(i_{\text{init}}, \text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}, \pi^{(\ell)}), (i_{\widetilde{\text{init}}}, \text{ct}_{\widetilde{\text{init}}}, \text{ct}_{\widetilde{\text{mid}}}, \text{ct}_{\widetilde{\text{fin}}}, \widetilde{\pi}^{(\ell)})\},$$

and set $S^{(\ell)} = \perp$.

(b) **Verify adjacency of the two proofs:** If $\text{ct}_{\text{fin}} \neq \text{ct}_{\widetilde{\text{fin}}}$ or $i_{\text{init}} + 2^\ell \neq i_{\widetilde{\text{init}}}$, output $\perp$.

(c) **Create a level $\ell + 1$ seBARG proof for the two level ℓ proofs:**

- i. If $\ell = 0$, compute

$$\pi^{(1)} \leftarrow \text{seBARG}.\mathcal{P}(\text{crs}_1, \{(\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}), (\text{ct}_{\widetilde{\text{init}}}, \text{ct}_{\widetilde{\text{fin}}})\}, \{\pi^{(0)}, \widetilde{\pi}^{(0)}\})$$

and add $(i_{\text{init}}, \text{ct}_{\text{init}}, \text{ct}_{\text{fin}}, \text{ct}_{\widetilde{\text{fin}}}, \pi^{(1)})$ to $S^{(1)}$.

- ii. If $\ell \geq 1$, compute

$$\pi^{(\ell+1)} \leftarrow \text{seBARG}.\mathcal{P}(\text{crs}_{\ell+1}, \{(\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}), (\text{ct}_{\widetilde{\text{init}}}, \text{ct}_{\widetilde{\text{fin}}})\}, \{(\text{ct}_{\text{mid}}, \pi^{(\ell)}), (\text{ct}_{\widetilde{\text{mid}}}, \widetilde{\pi}^{(\ell)})\})$$

and add $(i_{\text{init}}, \text{ct}_{\text{init}}, \text{ct}_{\text{fin}}, \text{ct}_{\widetilde{\text{fin}}}, \pi^{(\ell+1)})$ to $S^{(\ell+1)}$.

6. Output $\Pi_i = (\text{ct}_0, \text{ct}_i, r_0, r_i, \{S^{(\ell)}\}_{\ell \in \{0, \dots, \ell_{\max}\}})$.

• IVC. $\mathcal{V}(\text{crs}, (x_0, x_t, t), \Pi_t)$:

1. Parse $\text{crs} = (\ell_{\max}, C, \text{pk}, \text{crs}_{\text{SNARG}, \mathcal{P}}, \text{crs}_{\text{SNARG}, \mathcal{V}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_\ell\}_{\ell \in [\ell_{\max}]})$
and $\Pi_t = (\text{ct}_0, \text{ct}_t, r_0, r_t, \{S^{(\ell)}\}_{\ell \in \{0, \dots, \ell_{\max}\}})$.

2. Let $i_{\text{curr}} = 0$ and $\text{ct}_{\text{curr}} = \text{ct}_0$.

3. **Verify proofs at each level:** For $\ell \in \{\ell_{\max}, \ell_{\max} - 1, \dots, 0\}$,

(a) **Check if there is a proof at level ℓ :**

- i. Let s_ℓ be the ℓ^{th} least significant bit in the binary representation of t .
That is, we define $s_{\ell_{\max}} \dots s_1 s_0$ to be the $(\ell_{\max} + 1)$ -bit binary number representing t (with leading zeros added as necessary).
- ii. If $s_\ell = 0$, continue to the next iteration.
- iii. If $s_\ell = 1$, parse $S^{(\ell)} = \{(i_{\text{init}}, \text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}, \pi^{(\ell)})\}$.
Output 0 if $S^{(\ell)}$ is not of this form.

(b) **Verify adjacency with previous proof:**

- i. If $i_{\text{curr}} \neq i_{\text{init}}$ or $\text{ct}_{\text{curr}} \neq \text{ct}_{\text{init}}$, output 0.
- ii. Set $i_{\text{curr}} = i_{\text{curr}} + 2^\ell$ and $\text{ct}_{\text{curr}} = \text{ct}_{\text{fin}}$.

(c) **Verify proof at level ℓ :**

- i. If $\ell = 0$ and $\text{NIZK}.\mathcal{V}(\text{crs}_{\text{NIZK}}, (\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}), \pi^{(0)}) = 0$, output 0.
- ii. If $\ell \geq 1$ and $\text{seBARG}.\mathcal{V}(\text{crs}_\ell, \{(\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}), (\text{ct}_{\text{mid}}, \text{ct}_{\text{fin}})\}, \pi^{(\ell)}) = 0$, output 0.

4. **Verify the ciphertexts:** If $\text{ct}_{\text{curr}} \neq \text{ct}_t$, $\text{PKE}.\text{Enc}(\text{pk}, x_0; r_0) \neq \text{ct}_0$, or $\text{PKE}.\text{Enc}(\text{pk}, x_t; r_t) \neq \text{ct}_t$, output 0.

5. Output 1.

4.3 Correctness and Efficiency

Correctness. The correctness of our scheme follows in a straightforward manner from the correctness of the PKE, SNARG, NIZK, and seBARG.

The proof for (x_0, x_t, t) is $\Pi_t = (\text{ct}_0, \text{ct}_t, r_0, r_t, \{S^{(\ell)}\}_{\ell \in \{0, \dots, \ell_{\max}\}})$. Under an honest prover, ct_0 and ct_t will indeed be encryptions of x_0 and x_t using randomness r_0 and r_t . Additionally, for each $\ell \in \{0, \dots, \ell_{\max}\}$, except with negligible probability, each NIZK or seBARG proof contained in $S^{(\ell)}$ will correctly verify. This is because for each level ℓ , an honest prover will generate a NIZK or seBARG proof of a *true* statement (except with some negligible probability corresponding to the probability that at some previous step, some SNARG/NIZK/seBARG proof of a true statement does not verify).

Efficiency. We now argue that our scheme is efficient. That is, on setup parameters $(1^\lambda, T, 1^n, 1^m, C)$ where C is a polynomial-sized circuit $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$,

- The proof size is $\text{poly}(\lambda, n, \log T, \log |C|)$.
- All algorithms run in time $\text{poly}(\lambda, n, \log T, |C|)$.
- The crs size is $\text{poly}(\lambda, n, \log T, |C|)$.

We begin by analyzing the proof size. For an honest prover and for $t \leq T$, the proof for (x_0, x_t, t) is

$$\Pi_t = (\text{ct}_0, \text{ct}_t, r_0, r_t, \{S^{(\ell)}\}_{\ell \in \{0, \dots, \ell_{\max}\}})$$

where $\ell_{\max} = \lceil \log T \rceil$ and each $S^{(\ell)}$ contains at most one tuple of the form $(i_{\text{init}}, \text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}, \pi^{(\ell)})$. Thus based on the parameters in Section 4.1,

$$\begin{aligned} |\Pi_t| &\leq 2|\text{ct}| + 2\lambda_{\text{PKE}} + \sum_{\ell=0}^{\ell_{\max}} (\ell_{\max} + 3|\text{ct}| + |\pi^{(\ell)}|) \\ &\leq 2|\text{ct}| + 2\lambda_{\text{PKE}} + (\ell_{\max} + 1)(\ell_{\max} + 3|\text{ct}| + \text{poly}(\lambda, \ell_{\max}, n, \log |C|)) \\ &= \text{poly}(\lambda, n, \ell_{\max}, \log |C|) \\ &= \text{poly}(\lambda, n, \log T, \log |C|) \end{aligned}$$

It is then easy to see that our algorithms all run in time $\text{poly}(\lambda, \log T, |C|)$, which also implies that the crs is of size $\text{poly}(\lambda, \log T, |C|)$.

4.4 Soundness

In this section, we will prove the $(\text{poly}(\lambda), \text{negl}(\lambda))$ -soundness of our IVC for NP construction.

An honest proof for (x_0, x_t, t) is $\Pi_t = (\text{ct}_0, \text{ct}_t, r_0, r_t, \{S^{(\ell)}\}_{\ell \in \{0, \dots, \ell_{\max}\}})$ where each $S^{(\ell)}$ contains at most one tuple of the form $(i_{\text{init}}, \text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}, \pi^{(\ell)})$. The verifier will check each proof $\pi^{(\ell)}$ using the following process:

IVC. $\mathcal{V}^{(\ell)}$ (crs, $(\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}})$, $\pi^{(\ell)}$) :

1. Parse $\text{crs} = (\text{pk}, \text{crs}_{\text{SNARG}, \mathcal{P}}, \text{crs}_{\text{SNARG}, \mathcal{V}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_\ell\}_{\ell \in [\lambda]})$.
2. If $\ell = 0$ and $\text{NIZK}.\mathcal{V}(\text{crs}_{\text{NIZK}}, (\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}), \pi^{(0)}) = 0$, output 0.
3. If $\ell \geq 1$ and $\text{seBARG}.\mathcal{V}(\text{crs}_\ell, \{(\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}), (\text{ct}_{\text{mid}}, \text{ct}_{\text{fin}})\}, \pi^{(\ell)}) = 0$, output 0.

4. Output 1.

In addition, the verifier will check that these proofs correspond to adjacent steps of computation and that the first and last ciphertexts are correctly computed.

4.4.1 Proving soundness of proofs $\pi^{(\ell)}$ at every level

To prove soundness of our scheme, we will first prove the soundness of each of the individual proofs $\pi^{(\ell)}$ that make up the final proof Π_t for (x_0, x_t, t) . In particular, we will show that for every $\ell \in \{0, \dots, \ell_{\max}\}$, an adversary's advantage in outputting a level- ℓ proof $\pi^{(\ell)}$ for any “false” statement $(\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}})$ is negligible. Intuitively, a statement $(\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}})$ at level ℓ is false if $(x_{\text{init}}, x_{\text{fin}}, 2^\ell) \notin \mathcal{L}_C$ where x_{init} and x_{fin} are the values encrypted in ct_{init} and ct_{fin} respectively. Specifically, we show

Lemma 4.4. *For all sufficiently large λ , all iteration bounds $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, all polynomial-sized adversaries $\mathcal{A}$, and all $\ell \leq \{0, \dots, \lceil \log T \rceil\}$,*

$$\text{Adv}_{\mathcal{A}}^{(\ell)}(\lambda, T, n, m, C) \leq \text{negl}(\lambda)$$

where we define

$$\text{Adv}_{\mathcal{A}}^{(\ell)}(\lambda, T, n, m, C) = \Pr \left[\begin{array}{l} \text{IVC.V}^{(\ell)}(\text{crs}, (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) = 1 \\ \wedge (\exists (x_{\text{init}}, x_{\text{fin}}, r_{\text{init}}, r_{\text{fin}}) \\ \text{s.t. } (x_{\text{init}}, x_{\text{fin}}, 2^\ell) \notin \mathcal{L}_M \\ \wedge \text{ct}_{\text{init}} = \text{PKE.Enc}(\text{pk}, x_{\text{init}}; r_{\text{init}}) \\ \wedge \text{ct}_{\text{fin}} = \text{PKE.Enc}(\text{pk}, x_{\text{fin}}; r_{\text{fin}})) \end{array} : \begin{array}{l} (\text{crs}, \text{sk}) \leftarrow \text{IVC.TDGen}(1^\lambda, T, 1^n, 1^m, C, \perp, \perp), \\ ((\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) \leftarrow \mathcal{A}(\text{crs}, \text{sk}) \end{array} \right].$$

To determine whether a proof $\pi^{(\ell)}$ corresponds to a “false” statement $(\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}})$ at level ℓ , we will use the following trapdoor generation procedure IVC.TDGen . This is identical to IVC.Gen except that

- It additionally outputs the secret key sk of the PKE scheme. This lets us check which values are encrypted in the ciphertexts.
- It takes additional input (ℓ, b) and, if these values are not $\perp$, it generates $(\text{crs}_\ell, \text{td}_\ell)$ using seBARG 's trapdoor generation to extract on the b^{th} statement. It then outputs td_ℓ . This feature will be used in our proof.

$\text{IVC.TDGen}(1^\lambda, T, 1^n, 1^m, C, \ell, b) :$

1. **If $\ell = \perp$, $\text{td}_\ell = \perp$.**
2. $(\text{pk}, \text{sk}) \leftarrow \text{PKE.Gen}(1^\lambda)$.
3. $\text{crs}_{\text{SNARG}} \leftarrow \text{SNARG.Gen}(1^{\lambda_{\text{SNARG}}}, \mathcal{R}_{M,1}, 1^n, 1^m)$.
4. $(\text{crs}_{\text{NIZK}}, \text{td}_{\text{NIZK}}) \leftarrow \text{NIZK.Gen}(1^\lambda, \mathcal{R}_{0,\text{pk},\text{crs}_{\text{SNARG}},\mathcal{V}})$.
5. **If $\ell = 1$, $(\text{crs}_1, \text{td}_1) \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, \mathcal{R}_{1,\text{crs}_{\text{NIZK}}}, 1^{2n}, 1^{m_1}, 2, b)$.**
6. **Else**, $\text{crs}_1 \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, \mathcal{R}_{1,\text{crs}_{\text{NIZK}}}, 1^{2n}, 1^{m_1}, 2, \perp)$.

7. For $j \in \{2, \dots, \lambda\}$,
 - (a) If $\ell = j$, $(\text{crs}_\ell, \text{td}_\ell) \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, \mathcal{R}_{\ell, \text{crs}_{\ell-1}}, 1^{2n}, 1^{m_\ell}, 2, b)$.
 - (b) Else, $\text{crs}_j \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, \mathcal{R}_{j, \text{crs}_{j-1}}, 1^{2n}, 1^{m_j}, 2, \perp)$.
8. Let $\text{crs} := (\text{pk}, \text{crs}_{\text{SNARG}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_j\}_{j \in [\lambda]})$.
9. Output $(\text{crs}, \text{td}_\ell, \text{sk})$.

Proof of Lemma 4.4. We will prove this lemma via induction on ℓ starting from level $\ell = 0$. We show the following two lemmas:

Lemma 4.5. For all sufficiently large λ , all iteration bounds $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, and all polynomial-sized adversaries $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}}^{(0)}(\lambda, T, n, m, C) \leq 2^{-\lambda_{\text{NIZK}} \epsilon_{\text{NIZK}}} + 2^{-\lambda_{\text{SNARG}} \epsilon_{\text{SNARG}}}.$$

Lemma 4.6. For all sufficiently large λ , all iteration bounds $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, and all polynomial-sized adversaries $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}}^{(\ell)}(\lambda, T, n, m, C) \leq 2 \cdot \left(\text{Adv}_{\mathcal{A}}^{(\ell-1)}(\lambda, T, n, m, C) + (2^{2n} + 1) \cdot 2^{-\lambda_{\text{seBARG}} \epsilon_{\text{seBARG}}} \right).$$

These two lemmas then imply the following corollary.

Corollary 4.7. For all sufficiently large λ , all iteration bounds $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, and all polynomial-sized adversaries $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}}^{(\ell)}(\lambda, T, n, m, C) \leq 2^\ell \cdot 2^{-\lambda_{\text{NIZK}} \epsilon_{\text{NIZK}}} + 2^\ell \cdot 2^{-\lambda_{\text{SNARG}} \epsilon_{\text{SNARG}}} + 2^\ell \cdot (2^{2n} + 1) \cdot 2^{-\lambda_{\text{seBARG}} \epsilon_{\text{seBARG}}}.$$

We now prove each of the claims.

Proof of Lemma 4.5. Suppose for sake of contradiction that there exists a polynomial-sized adversary $\mathcal{A}$ such that for infinitely many λ (and for some $T \leq 2^\lambda$, polynomials $n = n(\lambda)$, $m = m(\lambda)$, and polynomial-sized circuit $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$),

$$\text{Adv}_{\mathcal{A}}^{(0)}(\lambda, T, n, m, C) \geq 2^{-\lambda_{\text{NIZK}} \epsilon_{\text{NIZK}}} + 2^{-\lambda_{\text{SNARG}} \epsilon_{\text{SNARG}}} \quad (6)$$

We build a polynomial-sized adversary $\mathcal{B}$ that breaks the soundness of our SNARG.

$\mathcal{B}(\text{crs}_{\text{SNARG}, \mathcal{P}}, \text{crs}_{\text{SNARG}, \mathcal{V}})$:

1. **Generate the rest of the IVC crs:**

- (a) $(\text{pk}, \text{sk}) \leftarrow \text{PKE.Gen}(\lambda_{\text{PKE}}, 1^n)$.
- (b) $(\text{crs}_{\text{NIZK}}, \text{td}_{\text{NIZK}}) \leftarrow \text{NIZK.Gen}(\lambda_{\text{NIZK}}, 1^{n_0}, 1^{m_0}, \mathcal{R}_{0, \text{pk}, \text{crs}_{\text{SNARG}, \mathcal{V}}})$.
- (c) $\text{crs}_1 \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, 2, 1^{n_1}, 1^{m_1}, \mathcal{R}_{1, \text{crs}_{\text{NIZK}}})$.
- (d) For $\ell \in \{2, \dots, \lambda\}$, $\text{crs}_\ell \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, 2, 1^{n_\ell}, 1^{m_\ell}, \mathcal{R}_{\ell, \text{crs}_{\ell-1}})$.
- (e) $\text{crs} = (\text{pk}, \text{crs}_{\text{SNARG}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_\ell\}_{\ell \in [\lambda]})$.

2. **Get level 0 proof from $\mathcal{A}$:**

- (a) $((\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(0)}) \leftarrow \mathcal{A}(\text{crs}, \text{sk})$.

(b) If $\text{IVC.V}^{(0)}(\text{crs}, (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(0)}) = 0$, output $\perp$.

3. **Extract SNARG proof from NIZK proof:**

(a) $(x_{\text{init}}, x_{\text{fin}}, r_{\text{init}}, r_{\text{fin}}, \pi_{\text{SNARG}}) \leftarrow \text{NIZK.Ext}(\text{crs}_{\text{NIZK}}, \text{td}_{\text{NIZK}}, (\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}), \pi^{(0)})$.

4. **Output** $((x_{\text{init}}, x_{\text{fin}}), \pi_{\text{SNARG}})$.

We claim that $\mathcal{B}$ breaks the soundness of SNARG. That is, for infinitely many λ ,

$$\Pr \left[\begin{array}{l} \text{SNARG.V}(\text{crs}_{\text{SNARG}, \mathcal{V}}, (x_{\text{init}}, x_{\text{fin}}), \pi_{\text{SNARG}}) = 1 \\ \wedge (x_{\text{init}}, x_{\text{fin}}) \notin \mathcal{L}_{C,1} \end{array} : \begin{array}{l} (\text{crs}_{\text{SNARG}, \mathcal{P}}, \text{crs}_{\text{SNARG}, \mathcal{V}}) \leftarrow \\ \text{SNARG.Gen}(1^{\lambda_{\text{SNARG}}}, 1^{2n}, 1^m, \mathcal{R}_{C,1}) \\ ((x_{\text{init}}, x_{\text{fin}}), \pi_{\text{SNARG}}) \leftarrow \mathcal{B}(\text{crs}_{\text{SNARG}, \mathcal{P}}, \text{crs}_{\text{SNARG}, \mathcal{V}}) \end{array} \right] \geq 2^{-\lambda_{\text{SNARG}} \epsilon_{\text{SNARG}}}$$

By definition of $\text{Adv}_{\mathcal{A}}^{(0)}$, with probability at least $\text{Adv}_{\mathcal{A}}^{(0)}(\lambda, T, n, m, C)$, $\mathcal{A}$ outputs values $((\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(0)})$ to $\mathcal{B}$ such that $\exists (x_{\text{init}}, x_{\text{fin}}, r_{\text{init}}, r_{\text{fin}})$ where

- $\text{NIZK.V}(\text{crs}_{\text{NIZK}}, (\text{ct}_{\text{init}}, \text{ct}_{\text{fin}}), \pi^{(0)}) = 1$
- $(x_{\text{init}}, x_{\text{fin}}) \notin \mathcal{L}_{C,1}$
- $\text{ct}_{\text{init}} = \text{PKE.Enc}(\text{pk}, x_{\text{init}}; r_{\text{init}})$ and $\text{ct}_{\text{fin}} = \text{PKE.Enc}(\text{pk}, x_{\text{fin}}; r_{\text{fin}})$.

By Eq. (6) and by the $2^{-\lambda_{\text{NIZK}} \epsilon_{\text{NIZK}}}$ extractable soundness of the NIZK, then this means that with probability at least

$$\text{Adv}_{\mathcal{A}}^{(0)}(\lambda, T, n, m, C) - 2^{-\lambda_{\text{NIZK}} \epsilon_{\text{NIZK}}} \geq 2^{-\lambda_{\text{SNARG}} \epsilon_{\text{SNARG}}},$$

NIZK.Ext extracts a valid witness $(x'_{\text{init}}, x'_{\text{fin}}, r'_{\text{init}}, r'_{\text{fin}}, \pi_{\text{SNARG}})$ for $(\text{ct}_{\text{init}}, \text{ct}_{\text{fin}})$. But if this is a valid witness, then since PKE is perfectly correct (i.e. two distinct messages will not encrypt to the same ciphertext), we must have

$$\text{SNARG.V}(\text{crs}_{\text{SNARG}, \mathcal{V}}, (x_{\text{init}}, x_{\text{fin}}), \pi_{\text{SNARG}}) = 1.$$

Since $(x_{\text{init}}, x_{\text{fin}}) \notin \mathcal{L}_{C,1}$, with probability at least $2^{-\lambda_{\text{SNARG}} \epsilon_{\text{SNARG}}}$, $\mathcal{B}$ breaks the security of the SNARG. $\square$

Proof of Lemma 4.6. We prove this via a hybrid argument. Fix any $\ell \in \{1, \dots, \lceil \log T \rceil\}$. We define $\text{Hybrid}_{\mathcal{A},0}^{(\ell)}$ to be the hybrid such that

$$\text{Adv}_{\mathcal{A}}^{(\ell)}(\lambda, T, n, m, C) = \Pr[\text{Hybrid}_{\mathcal{A},0}^{(\ell)}(1^\lambda, T, 1^n, 1^m, C) = 1]$$

Hybrid $_{\mathcal{A},0}^{(\ell)}(1^\lambda, T, 1^n, 1^m, C)$:

1. $(\text{crs}, \text{sk}) \leftarrow \text{IVC.TDGen}(1^\lambda, T, 1^n, 1^m, C, \perp, \perp)$.
2. $((\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) \leftarrow \mathcal{A}(\text{crs}, \text{sk})$.
3. If both of the following are true, output 1.
 - **Proof Verifies:** $\text{IVC.V}^{(\ell)}(\text{crs}, (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) = 1$.
 - **Statement is False:** There exists $(x_{\text{init}}, x_{\text{fin}}, r_{\text{init}}, r_{\text{fin}})$ s.t. $(x_{\text{init}}, x_{\text{fin}}, 2^\ell) \notin \mathcal{L}_M$, $\text{ct}_{\text{init}} = \text{PKE.Enc}(\text{pk}, x_{\text{init}}; r_{\text{init}})$, and $\text{ct}_{\text{fin}} = \text{PKE.Enc}(\text{pk}, x_{\text{fin}}; r_{\text{fin}})$

Else, output 0.

Now, to check whether the statement is false, we use the secret key sk of our PKE scheme to decrypt ct_{init} and ct_{fin} .

Hybrid $_{\mathcal{A},1}^{(\ell)}(1^\lambda, T, 1^n, 1^m, C)$:

1. $(crs, sk) \leftarrow \text{IVC.TDGen}(1^\lambda, T, 1^n, 1^m, C, \perp, \perp)$.
2. $((ct_{init}, ct_{mid}, ct_{fin}), \pi^{(\ell)}) \leftarrow \mathcal{A}(crs, sk)$.
3. If both of the following are true, output 1.
 - **Proof Verifies:** $\text{IVC.V}^{(\ell)}(crs, (ct_{init}, ct_{mid}, ct_{fin}), \pi^{(\ell)}) = 1$.
 - **Statement is False:** $(x_{init}, x_{fin}, 2^\ell) \notin \mathcal{L}_M$ where $x_{init} = \text{PKE.Dec}(sk, ct_{init})$ and $x_{fin} = \text{PKE.Dec}(sk, ct_{fin})$.

Else, output 0.

Lemma 4.8. For all sufficiently large λ , all iteration bounds $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, all polynomial-sized adversaries $\mathcal{A}$, and all $\ell \in [\lceil \log T \rceil]$,

$$\Pr[\text{Hybrid}_{\mathcal{A},1}^{(\ell)}(1^\lambda) = 1] = \Pr[\text{Hybrid}_{\mathcal{A},0}^{(\ell)}(1^\lambda) = 1]$$

Proof. Since the PKE is perfectly correct, the hybrids have identical behavior. $\square$

Making the seBARG extractable on one half of the statement. Recall that $\pi^{(\ell)}$ is a level ℓ seBARG proof.

We change the verification statement. Rather than checking if the full hop is incorrect (i.e. if $(x_{init}, x_{fin}, 2^\ell) \notin \mathcal{L}_M$), we check if either the first half of the hop is wrong (i.e. $(x_{init}, x_{mid}, 2^{\ell-1}) \notin \mathcal{L}_M$), or if the second half of the hop is wrong (i.e. $(x_{mid}, x_{fin}, 2^{\ell-1}) \notin \mathcal{L}_M$). We check the first half if $b = 0$ and the second half if $b = 1$.

Hybrid $_{\mathcal{A},2,b}^{(\ell)}(1^\lambda)$:

1. $(crs, sk) \leftarrow \text{IVC.TDGen}(1^\lambda, \mathcal{M}, 1^n, 1^m)$.
2. $((ct_{init}, ct_{mid}, ct_{fin}), \pi^{(\ell)}) \leftarrow \mathcal{A}(1^\lambda, (crs, sk))$.
3. Let $x_{init} = \text{PKE.Dec}(sk, ct_{init})$, $x_{mid} = \text{PKE.Dec}(sk, ct_{mid})$ and $x_{fin} = \text{PKE.Dec}(sk, ct_{fin})$.
4. If both of the following are true, output 1.
 - **Proof verifies:** $\text{IVC.V}^{(\ell)}(crs, (ct_{init}, ct_{mid}, ct_{fin}), \pi^{(\ell)}) = 1$.
 - **First or second half of statement is false:** Either
 - $b = 0$ and $(x_{init}, x_{mid}, 2^{\ell-1}) \notin \mathcal{L}_C$,
 - or $b = 1$ and $(x_{mid}, x_{fin}, 2^{\ell-1}) \notin \mathcal{L}_C$.

Else, output 0.

Lemma 4.9. For all $\ell \in \mathbb{N}$, all polynomial-sized adversaries $\mathcal{A}$, there exists a $b \in \{0, 1\}$, such that for all sufficiently large λ ,

$$\Pr[\mathbf{Hybrid}_{\mathcal{A},2,b}^{(\ell)}(1^\lambda) = 1] \geq \frac{1}{2} \cdot \Pr[\mathbf{Hybrid}_{\mathcal{A},1}^{(\ell)}(1^\lambda) = 1]$$

where $\mathbf{Hybrid}_{\mathcal{A},2,b}^{(\ell)}$ is defined above.

Proof. If $(x_{\text{init}}, x_{\text{fin}}, 2^\ell) \notin \mathcal{L}_M$, then for any x_{mid} , at least one of $(x_{\text{init}}, x_{\text{mid}}, 2^{\ell-1}) \notin \mathcal{L}_M$ or $(x_{\text{mid}}, x_{\text{fin}}, 2^{\ell-1}) \notin \mathcal{L}_M$. Thus, since PKE is perfectly correct (i.e. any ciphertext ct can only decrypt to one value), the lemma follows from the pigeonhole principle. $\square$

Hybrid $_{\mathcal{A},3,b}^{(\ell)}$ (1^λ): We now generate crs_ℓ so that we can extract on the b^{th} statement of the ℓ^{th} seBARG.

1. $(\text{crs}, \text{td}_\ell, \text{sk}) \leftarrow \text{IVC.TDGen}(1^\lambda, \mathcal{M}, 1^n, 1^m, \ell, b)$.
2. $((\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) \leftarrow \mathcal{A}(1^\lambda, (\text{crs}, \text{sk}))$.
3. Let $x_{\text{init}} = \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{init}})$, $x_{\text{mid}} = \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{mid}})$ and $x_{\text{fin}} = \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{fin}})$.
4. If both of the following are true, output 1.
 - **Proof verifies:** $\text{IVC.V}^{(\ell)}(\text{crs}, (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) = 1$.
 - **First or second half of statement is false:** Either
 - $b = 0$ and $(x_{\text{init}}, x_{\text{mid}}, 2^{\ell-1}) \notin \mathcal{L}_M$,
 - or $b = 1$ and $(x_{\text{mid}}, x_{\text{fin}}, 2^{\ell-1}) \notin \mathcal{L}_M$.

Else, output 0.

Lemma 4.10. For all $\ell \in \mathbb{N}$, all $b \in \{0, 1\}$, all polynomial-sized adversaries $\mathcal{A}$, and all sufficiently large λ ,

$$\Pr[\mathbf{Hybrid}_{\mathcal{A},3,b}^{(\ell)}(1^\lambda) = 1] \leq \Pr[\mathbf{Hybrid}_{\mathcal{A},2,b}^{(\ell)}(1^\lambda) = 1] + 2^{2n} \cdot 2^{\lambda_{\text{seBARG}}} \cdot 2^{\epsilon_{\text{seBARG}}}.$$

Proof. If we were to try proving this lemma directly via a reduction to the crs indistinguishability of our seBARG, then our reduction would need to decide the language $\mathcal{L}_M$. This would likely require either nondeterministic advice of size 2^{2n} (which means we need seBARG security against 2^{2n} -sized adversaries), or an prohibitively large runtime.

To avoid this, rather than checking if $(x_{\alpha'}, x_{\beta'}, 2^{\ell-1}) \notin \mathcal{L}_M$ for some $(x_{\alpha'}, x_{\beta'})$, we will simply check if $(x_{\alpha'}, x_{\beta'}) = (x_\alpha, x_\beta)$ for some hardcoded (x_α, x_β) . We will then argue crs indistinguishability with respect to every choice of $(x_\alpha, x_\beta) \in \{0, 1\}^{2n}$. This will incur a 2^{2n} loss in the security advantage, but only a polynomial loss in the size of our adversaries.

In more detail, for $b' \in \{0, 1\}$, we define $\mathbf{Hybrid}_{\mathcal{A},i,b,(x_\alpha,x_\beta)}^{(\ell)}$ to be the same as $\mathbf{Hybrid}_{\mathcal{A},i,b}^{(\ell)}$ for $i \in \{2, 3\}$ except that we change Step 4 to the following:

4. If both of the following are true, output 1.
 - **Proof verifies:** $\text{IVC.V}^{(\ell)}(\text{crs}, (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) = 1$.
 - **First or second half of statement is (x_α, x_β) :** Either
 - $b = 0$ and $(x_{\text{init}}, x_{\text{mid}}) = (x_\alpha, x_\beta)$,
 - or $b = 1$ and $(x_{\text{mid}}, x_{\text{fin}}) = (x_\alpha, x_\beta)$.

Else, output 0.

We now argue crs indistinguishability with respect to each choice of $(x_\alpha, x_\beta) \in \{0, 1\}^{2n}$.

Proposition 4.11. *For all $\ell \in \mathbb{N}$, $b \in \{0, 1\}$, $(x_\alpha, x_\beta) \in \{0, 1\}^{2n}$, all polynomial-sized adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, and all sufficiently large λ .*

$$\left| \Pr[\mathbf{Hybrid}_{\mathcal{A}, 3, b, (x_\alpha, x_\beta)}^{(\ell)}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{\mathcal{A}, 2, b, (x_\alpha, x_\beta)}^{(\ell)}(1^\lambda) = 1] \right| < 2^{-\lambda_{\text{seBARG}} \epsilon_{\text{seBARG}}}$$

Proof. Suppose for contradiction that there exists $\ell \in \mathbb{N}$, $b \in \{0, 1\}$, $(x_\alpha, x_\beta) \in \{0, 1\}^{2n}$, and a polynomial-sized adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ such that for infinitely many λ ,

$$\left| \Pr[\mathbf{Hybrid}_{\mathcal{A}, 3, b, (x_\alpha, x_\beta)}^{(\ell)}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{\mathcal{A}, 2, b, (x_\alpha, x_\beta)}^{(\ell)}(1^\lambda) = 1] \right| \geq 2^{-\lambda_{\text{seBARG}} \epsilon_{\text{seBARG}}}$$

We now build a polynomial-sized⁵ adversary $\mathcal{B} = (\mathcal{B}_{1, b, (x_\alpha, x_\beta)}^{(\ell)}, \mathcal{B}_{2, b, (x_\alpha, x_\beta)}^{(\ell)})$ that breaks the crs indistinguishability of the ℓ^{th} seBARG.

$\mathcal{B}_{1, b, (x_\alpha, x_\beta)}^{(\ell)}(1^{\lambda_{\text{seBARG}}})$:

1. Generate initial part of crs:

- (a) $(\text{pk}, \text{sk}) \leftarrow \text{PKE.Gen}(1^\lambda)$.
- (b) $\text{crs}_{\text{SNARG}} \leftarrow \text{SNARG.Gen}(1^{\lambda_{\text{SNARG}}}, \mathcal{R}_{M, 1}, 1^n, 1^m)$.
- (c) $(\text{crs}_{\text{NIZK}}, \text{td}_{\text{NIZK}}) \leftarrow \text{NIZK.Gen}(1^\lambda, \mathcal{R}_{0, \text{pk}, \text{crs}_{\text{SNARG}}, \nu})$.
- (d) **If $\ell > 1$** , $\text{crs}_1 \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, \mathcal{R}_{1, \text{crs}_{\text{NIZK}}}, 1^{2n}, 1^{m_1}, 2, \perp)$.
- (e) **For $j \in \{2, \dots, \ell - 1\}$** ,

$$\text{crs}_j \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, \mathcal{R}_{j, \text{crs}_{j-1}}, 1^{2n}, 1^{m_j}, 2, \perp).$$

2. $\text{st}_\mathcal{B} = (1^{\lambda_{\text{seBARG}}}, \mathcal{M}, 1^n, 1^m, \text{st}_\mathcal{A}, \text{pk}, \text{sk}, \text{crs}_{\text{SNARG}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_j\}_{j \in [\ell-1]})$.

3. **If $\ell = 1$** , output $((\mathcal{R}_{1, \text{crs}_{\text{NIZK}}}, 1^{2n}, 1^{m_1}, 2, b), \text{st}_\mathcal{B})$

4. **If $\ell > 1$** , output $((\mathcal{R}_{\ell, \text{crs}_{\text{NIZK}}}, 1^{2n}, 1^{m_\ell}, 2, b), \text{st}_\mathcal{B})$

$\mathcal{B}_{2, b, (x_\alpha, x_\beta)}^{(\ell)}(\text{crs}^*, \text{st}_\mathcal{B})$:

- 1. **$\text{crs}_\ell := \text{crs}^*$.**

2. Generate remainder of crs:

- (a) **For $j \in \{\ell + 1, \dots, \lambda\}$** ,

$$\text{crs}_j \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, \mathcal{R}_{j, \text{crs}_{j-1}}, 1^{2n}, 1^{m_j}, 2, \perp).$$

⁵We only consider the infinitely many λ such that $\lambda \geq \ell$.

3. Define $\text{crs} := (\text{pk}, \text{crs}_{\text{SNARG}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_\ell\}_{\ell \in [\lambda]}).$
4. $((\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) \leftarrow \mathcal{A}(1^\lambda, (\text{crs}, \text{sk})).$
5. Let $x_{\text{init}} = \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{init}})$, $x_{\text{mid}} = \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{mid}})$, and $x_{\text{fin}} = \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{fin}}).$
6. If both of the following are true, output 1.
 - **Proof verifies:** $\text{IVC.V}^{(\ell)}(\text{crs}, (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) = 1.$
 - **First or second half of statement is false:** Either
 - $b = 0$ and $(x_{\text{init}}, x_{\text{mid}}) = (x_\alpha, x_\beta),$
 - or $b = 1$ and $(x_{\text{mid}}, x_{\text{fin}}) = (x_\alpha, x_\beta).$

Else, output 0.

Note that $\mathcal{B}_{b, x_\alpha, x_\beta}^{(\ell)}$ as defined in above does *not* need to decide the language $\mathcal{L}_M$.

Observe that if $\mathcal{B}_{b, x_\alpha, x_\beta}^{(\ell)}$ receives a non-trapdoor crs^* , then it perfectly emulates $\text{Hybrid}_{\mathcal{A}, 2, b, (x_\alpha, x_\beta)}^{(\ell)}$, and if $\mathcal{B}_{b, x_\alpha, x_\beta}^{(\ell)}$ receives a crs^* generated with a trapdoor on the b^{th} statement, then $\mathcal{B}$ perfectly emulates $\text{Hybrid}_{\mathcal{A}, 3, b, (x_\alpha, x_\beta)}^{(\ell)}$. Thus, $\mathcal{B}_{b, x_\alpha, x_\beta}^{(\ell)}$ distinguishes between the two types of crs with probability at least $2^{-\lambda_{\text{seBARG}} \epsilon_{\text{seBARG}}}$, breaking the crs indistinguishability of the seBARG . $\square$

Therefore, by the claim and by definition of our hybrids, we have that

$$\begin{aligned}
& \left| \Pr[\text{Hybrid}_{\mathcal{A}, 3, b}^{(\ell)}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{\mathcal{A}, 2, b}^{(\ell)}(1^\lambda) = 1] \right| \\
& \leq \left| \sum_{\substack{(x_\alpha, x_\beta) \in \{0, 1\}^{2n}: \\ (x_\alpha, x_\beta, 2^{\ell-1}) \notin \mathcal{L}_M}} \left(\Pr[\text{Hybrid}_{\mathcal{A}, 3, b, (x_\alpha, x_\beta)}^{(\ell)}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{\mathcal{A}, 2, b, (x_\alpha, x_\beta)}^{(\ell)}(1^\lambda) = 1] \right) \right| \\
& \leq \sum_{\substack{(x_\alpha, x_\beta) \in \{0, 1\}^{2n}: \\ (x_\alpha, x_\beta, 2^{\ell-1}) \notin \mathcal{L}_M}} \left| \Pr[\text{Hybrid}_{\mathcal{A}, 3, b, (x_\alpha, x_\beta)}^{(\ell)}(1^\lambda) = 1] - \Pr[\text{Hybrid}_{\mathcal{A}, 2, b, (x_\alpha, x_\beta)}^{(\ell)}(1^\lambda) = 1] \right| \\
& \leq 2^{2n} \cdot 2^{-\lambda_{\text{seBARG}} \epsilon_{\text{seBARG}}}
\end{aligned}$$

$\square$

Lemma 4.12. For all $\ell \in \mathbb{N}$, all $b \in \{0, 1\}$, if there exists a polynomial-sized adversary $\mathcal{A}$ and a function μ such that for all sufficiently large λ ,

$$\Pr[\text{Hybrid}_{\mathcal{A}, 3, b}^{(\ell)}(1^\lambda) = 1] \geq \mu(\lambda)$$

then there exists a polynomial-sized adversary $\mathcal{B}$ such that for all sufficiently large λ ,

$$\Pr[\text{Hybrid}_{\mathcal{B}}^{(\ell-1)}(1^\lambda) = 1] \geq \mu(\lambda) - 2^{-\lambda_{\text{seBARG}} \delta_{\text{seBARG}}}$$

Proof. Consider such an adversary $\mathcal{A}$, and construct $\mathcal{B}$:

$\mathcal{B}^{(\ell-1)}(\text{crs}^*, \text{sk})$:

1. Parse $\text{crs}^* = (\ell_{\max}, C, \text{pk}, \text{crs}_{\text{SNARG}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_j\}_{j \in [\ell_{\max}]})$
2. **Discard and regenerate crs_j for all $j \geq \ell$:**
 - Discard crs_j for $j \geq \ell$.
 - Compute
$$(\text{crs}'_{\ell}, \text{td}) \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, \mathcal{R}_{1, \text{crs}_{\ell-1}, 1^{2n}, 1^{m_{\ell}}, 2, b}).$$
 - For $j \geq \ell + 1$, generate
$$\text{crs}'_j \leftarrow \text{seBARG.Gen}(1^{\lambda_{\text{seBARG}}}, \mathcal{R}_{j, \text{crs}_{j-1}, 1^{2n}, 1^{m_j}, 2, \perp}).$$
3. Let $\text{crs}' = (\ell_{\max}, C, \text{pk}, \text{crs}_{\text{SNARG}}, \text{crs}_{\text{NIZK}}, \{\text{crs}_j\}_{j \in [\ell-1]} \cup \{\text{crs}'_j\}_{j \in \{\ell, \dots, \ell_{\max}\}})$.
4. $((\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) \leftarrow \mathcal{A}(1^{\lambda}, (\text{crs}, \text{sk}))$.
5. If proof verifies, i.e. $\text{IVC.V}^{(\ell)}(\text{crs}', (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) = 1$,
 - Let $(\text{ct}, \pi) \leftarrow \text{seBARG.Ext}(\text{td}, (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)})$.
 - If $b = 0$, output $(\text{ct}_{\text{init}}, \text{ct}, \text{ct}_{\text{mid}}, \pi)$. If $b = 1$, output $(\text{ct}_{\text{mid}}, \text{ct}, \text{ct}_{\text{fin}}, \pi)$.

We argue that

$$\Pr[\mathbf{Hybrid}_B^{(\ell-1)}(1^{\lambda}) = 1] \geq \mu(\lambda) - 2^{-\lambda_{\text{seBARG}} \delta_{\text{seBARG}}}.$$

First, note that crs' given to $\mathcal{A}$ by $\mathcal{B}$ is distributed identically to $\mathbf{Hybrid}_{\mathcal{A}, 3, b}^{(\ell)}(1^{\lambda})$. Therefore, by assumption, with probability at least $\mu(\lambda)$, $\mathcal{A}$ outputs a proof $((\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)})$ such that

- $\text{IVC.V}^{(\ell)}(\text{crs}', (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)}) = 1$.
- Let $x_{\text{init}} = \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{init}})$, $x_{\text{mid}} = \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{mid}})$, and $x_{\text{fin}} = \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{fin}})$. If $b = 0$, then $(x_{\text{init}}, x_{\text{mid}})$ is false. If $b = 1$, $(x_{\text{mid}}, x_{\text{fin}})$ is false.

If $\text{IVC.V}^{(\ell)}$ accepts, by the somewhere soundness property of the seBARG, except with probability $2^{-\lambda_{\text{seBARG}} \delta_{\text{seBARG}}}$,

$$(\text{ct}, \pi) \leftarrow \text{seBARG.Ext}(\text{td}, (\text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}), \pi^{(\ell)})$$

satisfies:

- If $b = 0$, $\text{seBARG.V}(\text{crs}_{\ell-1}, \{(\text{ct}_{\text{init}}, \text{ct}), (\text{ct}, \text{ct}_{\text{mid}})\}, \pi) = 1$.
- If $b = 1$, $\text{seBARG.V}(\text{crs}_{\ell-1}, \{(\text{ct}_{\text{mid}}, \text{ct}), (\text{ct}, \text{ct}_{\text{fin}})\}, \pi) = 1$.

Therefore, with probability at least $\mu(\lambda) - 2^{-\lambda_{\text{seBARG}} \delta_{\text{seBARG}}}$, the extracted proof is accepting while the corresponding instance $((x_{\text{init}}, x_{\text{mid}})$ if $b = 0$ and $(x_{\text{mid}}, x_{\text{fin}})$ if $b = 1$) is false. In other words,

$$\Pr[\mathbf{Hybrid}_B^{(\ell-1)}(1^{\lambda}) = 1] \geq \mu(\lambda) - 2^{-\lambda_{\text{seBARG}} \delta_{\text{seBARG}}}$$

as desired. □

Therefore, combining Lemma 4.9, Lemma 4.10, Lemma 4.12, we have completed the proof of Lemma 4.6. □

□

4.4.2 Proving soundness of our IVC scheme.

We are now ready to prove the soundness of the construction in Section 4.2.

Suppose there exists a cheating prover $\mathcal{P}^*$ that outputs an instance (x_0, x_t, t) and a proof Π_t such that IVC.V accepts with probability $p(\lambda) \geq \frac{1}{\text{poly}(\lambda)}$ for some poly. Then, we show that for some $i \in [\ell_{\max}]$, one can create an adversary $\mathcal{B}$ such that

$$\text{Adv}_{\mathcal{B}}^{(\ell)}(\lambda) \geq \frac{p(\lambda)}{\ell_{\max}}.$$

In fact, note that when $\mathcal{P}^*$ creates outputs a false statement (x, x_i, i) and an *accepting* proof $\Pi_i = (\text{ct}_0, \text{ct}_i, r_0, r_1, \{S^{(\ell)}\}_{\ell \in [i]})$. Recall that we check that ct_0 and ct_i are consistent with x_0, r_0, x_i, r_i , the fact that we verify the “adjacency” of the proofs (i.e. the end point states of ct are consistent). Therefore, there must exist a proof $S^{(j)} = \{(j, \text{ct}_{\text{init}}, \text{ct}_{\text{mid}}, \text{ct}_{\text{fin}}, \pi^{(j)})\}$ where $x_a := \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{init}})$ and $x_b := \text{PKE.Dec}(\text{sk}, \text{ct}_{\text{fin}})$ satisfies that $(x_0, x_a, a) \in \mathcal{L}_{\mathcal{M}}$ but $(x_a, x_b, b - a) \notin \mathcal{L}_{\mathcal{M}}$ (this must be true since x_0 is a valid configuration, and x_i is not reachable from x_0 in i steps, so we can consider the smallest j such that this happens). However, since the proof accepts, it must be the case that $\text{IVC.V}^{(j)}$ accepted the proof $\pi^{(j)}$. We say that $\mathcal{P}^*$ successfully cheats on index j if this holds.

By pigeonhole principle, there must exist some $j \in [\ell_{\max}]$ such $\mathcal{P}^*$ cheats on index j with probability $p(\lambda)/\ell_{\max}$. Now, consider the reduction $\mathcal{B}$:

$\mathcal{B}(\text{crs}, \text{sk})$:

1. Call $\Pi_i \leftarrow \mathcal{A}(\text{crs}, \text{sk})$.
2. Parse $\Pi_i = (\text{ct}_0, \text{ct}_i, r_0, r_1, \{S^{(\ell)}\}_{\ell \in [i]})$.
3. Output $S^{(j)}$.

By our earlier discussion, $\Pr[\text{Hybrid}_{\mathcal{B}}^{(j)} = 1] = p(\lambda)/\ell_{\max}$, as desired. However, this contradicts Corollary 4.7. Therefore, we have that no such $\mathcal{P}^*$ can exist, therefore showing $(\text{poly}(\lambda), \text{negl}(\lambda))$ -soundness in Theorem 4.1.

4.5 Zero Knowledge

In this section, we prove the following:

Theorem 4.13. *The IVC construction in Section 4.2 is adaptively, computational zero-knowledge (see Definition 3.2 for the formal definition of this property).*

Proof. Consider the following hybrids.

Hybrid_{A,0}: In this experiment, proofs are generated honestly (as in the adaptive zero-knowledge IVC game Definition 3.15). Recall that the proof has the form

$$\Pi_i = (\text{ct}_0, \text{ct}_i, r_0, r_i, \{S^{(\ell)}\}_{\ell \in \{0, \dots, \ell_{\max}\}}).$$

Hybrid_{A,1}: Let $\text{Sim} = (\text{Sim}_1, \text{Sim}_2, \text{Sim}_3)$ be the simulator guaranteed by the NIZK’s adaptive, multi-theorem zero-knowledge property (see Definition 3.15). In this hybrid, IVC.Gen is modified so that

$$\text{st}_{\text{NIZK}}, \text{crs}_{\text{NIZK}} \leftarrow \text{Sim}_1(1^{\lambda_{\text{NIZK}}}, 1^{n_0}, 1^{m_0}, \mathcal{R}_{0, \text{pk}, \text{crs}_{\text{SNARG}}})$$

in Step 3c. Now, given $(x_0, w_1, \dots, w_t, x_t)$ such that for all i , $x_i = C(x_{i-1}, w_i)$, proceed as before except level 0 proofs are generated as follows:

1. $x_i = C(x_{i-1}, w_i)$.
2. $\pi_{\text{SNARG}} \leftarrow \text{SNARG}.\mathcal{P}(\text{crs}_{\text{SNARG}}, (x_{i-1}, x_i), w_i)$.
3. $r_i \leftarrow \{0, 1\}^{\lambda_{\text{PKE}}}$.
4. $\text{ct}_i \leftarrow \text{PKE}.\text{Enc}(\text{pk}, x_i; r_i)$.
5. $\pi_i^{(0)} \leftarrow \text{Sim}_2(\text{st}_{\text{NIZK}}, \text{crs}_{\text{NIZK}}, (\text{ct}_{i-1}, \text{ct}_i), (x_{i-1}, x_i, r_{i-1}, r_i, \pi_{\text{SNARG}}))$, which by definition of the simulator is equal to $\text{Sim}_3(\text{st}_{\text{NIZK}}, \text{crs}_{\text{NIZK}}, (\text{ct}_{i-1}, \text{ct}_i))$, i.e. does not explicitly depend on the values $x_{i-1}, x_i, r_{i-1}, r_i, \pi_{\text{SNARG}}$.
6. Add $(i-1, \text{ct}_{i-1}, \perp, \text{ct}_i, \pi_i^{(0)})$ to $S^{(0)}$.

Hybrid_{A,2}: In this hybrid, call Sim_3 directly instead of Sim_2 . No longer compute π_{SNARG} . In other words, level 0 proofs are generated as follows:

1. $x_i = C(x_{i-1}, w_i)$.
2. $\pi_{\text{SNARG}} \leftarrow \text{SNARG}.\mathcal{P}(\text{crs}_{\text{SNARG}}, (x_{i-1}, x_i), w_i)$.
3. $r_i \leftarrow \{0, 1\}^{\lambda_{\text{PKE}}}$.
4. $\text{ct}_i \leftarrow \text{PKE}.\text{Enc}(\text{pk}, x_i; r_i)$.
5. $\pi_i^{(0)} \leftarrow \text{Sim}_3(\text{st}_{\text{NIZK}}, \text{crs}_{\text{NIZK}}, (\text{ct}_{i-1}, \text{ct}_i))$.
6. Add $(i-1, \text{ct}_{i-1}, \perp, \text{ct}_i, \pi_i^{(0)})$ to $S^{(0)}$.

Hybrid_{A,3}: In this hybrid, level 0 proofs are instead sampled as follows:

1. $x_i = C(x_{i-1}, w_i)$.
2. For $i = 0$ and $i = T$, sample $r_i \leftarrow \{0, 1\}^{\lambda_{\text{PKE}}}$ and set $\text{ct}_i \leftarrow \text{PKE}.\text{Enc}(\text{pk}, x_i; r_i)$.
3. For $1 \leq i \leq T-1$, $\text{ct}_i \leftarrow \text{PKE}.\text{Enc}(\text{pk}, 0)$.
4. $\pi_i^{(0)} \leftarrow \text{Sim}_3(\text{st}_{\text{NIZK}}, \text{crs}_{\text{NIZK}}, (\text{ct}_{i-1}, \text{ct}_i))$.
5. Add $(i-1, \text{ct}_{i-1}, \perp, \text{ct}_i, \pi_i^{(0)})$ to $S^{(0)}$.

Hybrid_{A,4}: In this hybrid, we make the following syntactic changes in the construction of level 0 proofs.

1. ~~$x_i = C(x_{i-1}, w_i)$~~ .
2. For $i = 0$ and $i = T$, sample $r_i \leftarrow \{0, 1\}^{\lambda_{\text{PKE}}}$ and set $\text{ct}_i \leftarrow \text{PKE}.\text{Enc}(\text{pk}, x_i; r_i)$.
3. For $1 \leq i \leq T-1$, $\text{ct}_i \leftarrow \text{PKE}.\text{Enc}(\text{pk}, 0)$.
4. $\pi_i^{(0)} \leftarrow \text{Sim}_3(\text{st}_{\text{NIZK}}, \text{crs}_{\text{NIZK}}, (\text{ct}_{i-1}, \text{ct}_i))$.
5. Add $(i-1, \text{ct}_{i-1}, \perp, \text{ct}_i, \pi_i^{(0)})$ to $S^{(0)}$.

Note that x_i 's and w_i 's are no longer used in this hybrid. The view of h

We can define the simulator for the IVC protocol Sim_{IVC} to generate proofs just as described in **Hybrid_{A,4}**.

It is easy to see that the views of $\mathcal{A}$ in **Hybrid_{A,1}** and **Hybrid_{A,2}** are identical, and the views in **Hybrid_{A,3}** and **Hybrid_{A,4}** are identical.

Lemma 4.14. For all $\ell \in \mathbb{N}$, $b \in \{0, 1\}$, $(x_\alpha, x_\beta) \in \{0, 1\}^{2n}$, all polynomial-sized adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, and all sufficiently large λ .

$$\left| \Pr[\mathbf{Hybrid}_{\mathcal{A},1}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{\mathcal{A},0}(1^\lambda) = 1] \right| < 2^{-\lambda_{\text{NIZK}}^{\delta_{\text{NIZK}}}}.$$

Proof. This follows directly from the adaptive, zero-knowledge security of the NIZK scheme. $\square$

Lemma 4.15. For all $\ell \in \mathbb{N}$, $b \in \{0, 1\}$, $(x_\alpha, x_\beta) \in \{0, 1\}^{2n}$, all polynomial-sized adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, and all sufficiently large λ .

$$\left| \Pr[\mathbf{Hybrid}_{\mathcal{A},3}(1^\lambda) = 1] - \Pr[\mathbf{Hybrid}_{\mathcal{A},2}(1^\lambda) = 1] \right| < 2^{-\lambda_{\text{PKE}}^{\delta_{\text{PKE}}}}.$$

Proof. This reduces to the semantic security of $\text{ct}_1, \dots, \text{ct}_{T-1}$. Note that the randomness used to generate ct_i for $1 \leq i \leq T-1$ is hidden from the adversary in both $\mathbf{Hybrid}_{\mathcal{A},2}$ and $\mathbf{Hybrid}_{\mathcal{A},3}$ (only the randomness corresponding to ct_0 and ct_T are in the clear). Therefore, this follows from the semantic security. $\square$

Therefore, the simulator Sim_{IVC} in fact satisfies the adaptive zero-knowledge property in Definition 3.2, as desired. Therefore, since $\mathbf{Hybrid}_{\mathcal{A},0}$ (the normal view), and $\mathbf{Hybrid}_{\mathcal{A},4}$ (the simulated view) are indistinguishable, we have that the IVC scheme is in fact adaptively zero-knowledge. $\square$

5 Fully Succinct IVC for Trapdoor Languages

In this section, we present our construction of a *fully succinct* IVC for trapdoor languages from subexponentially secure $i\mathcal{O}$ and subexponentially secure length-doubling injective PRGs.⁶

Theorem 5.1. Assume the following ingredients:

- $(\text{poly}(\lambda_{i\mathcal{O}}), 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}})$ -secure $i\mathcal{O}$,
- $(\text{poly}(\lambda_{\text{PRG}}), 2^{-\lambda_{\text{PRG}}^{\delta_{\text{PRG}}}})$ -secure length-doubling injective PRG,
- $(\text{poly}(\lambda_{\text{PPRF}}), 2^{-\lambda_{\text{PPRF}}^{\delta_{\text{PPRF}}}})$ -secure PPRF,

where $0 < \delta_{i\mathcal{O}}, \delta_{\text{PPRF}}, \delta_{\text{PRG}} < 1$ are any constants and $\lambda_{i\mathcal{O}}, \lambda_{\text{PPRF}}, \lambda_{\text{PRG}} = \text{poly}(\lambda, \log T, n, m, |C|, \gamma)$ are as defined in Section 5.2. Then, there exists an IVC for (γ, T) -Trapdoor IVC Languages with circuits of size $|C|$ with⁷

- $(\text{poly}(\lambda), \text{negl}(\lambda))$ -soundness,
- perfect zero knowledge,
- crs of size $\text{poly}(\lambda, \log T, |C|, \gamma)$,
- proofs of size $\text{poly}(\lambda, \log T, \log |C|)$ (i.e. full succinctness).

⁶Our construction also uses subexponentially secure puncturable PRFs which can be constructed from subexponentially secure injective PRGs.

⁷We use $\text{poly}(\lambda)$ as shorthand notation. More formally, to build an IVC scheme with $(p(\lambda), \text{negl}(\lambda))$ -soundness for some polynomial p , we require our underlying primitives to be secure against q -sized adversaries for some polynomial $q = q(p, \lambda, \log T, n, m, |C|, \gamma)$.

5.1 Trapdoor Language Definitions

IVC for Trapdoor Languages is defined similarly to IVC for NP (Definition 3.2) except that

- The scheme is designed for a special class of $\mathcal{L}_{\text{IVC}}$ (Definition 3.1) called Trapdoor IVC Languages (Definition 5.2). A (γ, T) -Trapdoor IVC Language is the restriction of the IVC language $\mathcal{L}_{\text{IVC}}$ to a specific circuit C for which there exists a γ -sized trapdoor circuit Trap_C which can decide whether any tuple (C, x_0, x_i, i) with $i \leq T$ is in $\mathcal{L}_{\text{IVC}}$.
- We allow IVC.Gen to take an additional input 1^γ indicating the maximum size of the trapdoor circuit. Note that we do not require the trapdoor circuit Trap_C itself to be known.

We now formally define these notions.

Definition 5.2 ((γ, T) -Trapdoor IVC Language). *Let $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ be a circuit. We say that the restricted IVC language*

$$\mathcal{L}_C = \{(x_0, x_i, i) : (C, x_0, x_i, i) \in \mathcal{L}_{\text{IVC}}\}$$

is a (γ, T) -Trapdoor IVC Language if there exists a trapdoor circuit Trap_C of size at most γ such that for all (x_0, x_t, t) where $x_0, x_t \in \{0, 1\}^n$ and $t \leq T$,

$$\text{Trap}_C(x_0, x_t, t) = 1 \text{ iff } (C, x_0, x_t, t) \in \mathcal{L}_{\text{IVC}}$$

We say that C has a (γ, T) -trapdoor circuit Trap_C if the above situation holds.

Definition 5.3 (IVC for Trapdoor Languages). *An incrementally verifiable computation (IVC) scheme for trapdoor languages is a tuple of PPT algorithms $\text{IVC} = (\text{IVC.Gen}, \text{IVC.P}, \text{IVC.V})$ where*

- $\text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$: *takes as input the security parameter λ , an iteration bound T in binary, an input length n , a witness length m , a circuit $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$, and a trapdoor circuit size γ , and outputs a common reference strings crs .*
- $\text{IVC.P}(\text{crs}, (x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$: *takes as input the common reference string crs , inputs $x_0, x_{i-1} \in \{0, 1\}^n$, an index $i-1 \in [T-1]$, a proof π_{i-1} , and a witness $w_i \in \{0, 1\}^m$, and outputs a proof π_i .*
- $\text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t)$: *takes as input the common reference string crs , inputs $x_0, x_t \in \{0, 1\}^n$, an index $i \in [T]$, and a proof π_t , and outputs a value in $\{0, 1\}$.*

The scheme is additionally required to satisfy completeness and efficiency as defined below.

Completeness: *For all $\lambda \in \mathbb{N}$, all iteration bounds $T \leq 2^\lambda$, all $t \leq T$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all $x_0 \in \{0, 1\}^n$, and all $w_1, \dots, w_t \in \{0, 1\}^m$,*

$$\Pr \left[\begin{array}{l} \text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma) \\ \forall i \in [t] : x_i = C(x_{i-1}, w_i) \\ \forall i \in [t] : \\ \pi_i \leftarrow \text{IVC.P}(\text{crs}, (x_0, x_{i-1}, i-1), \pi_{i-1}, w_i) \end{array} : \text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t) = 1 \right] = 1.$$

Efficiency: *There exists polynomials p_1 and p_2 such that on setup parameters $(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$ where C is a polynomial-sized circuit $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which has a (γ, T) -trapdoor circuit Trap_C , all algorithms run in time $p_1(\lambda, \log T, |C|, \gamma)$ and the proof size is $p_2(\lambda, n, m, \log T, \log |C|, \log \gamma)$.*

We say that the scheme satisfies (adaptive) (s, ϵ) -soundness if the following holds:

(s, ϵ) -Soundness: For all sufficiently large λ , all iteration bounds $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , and all $s = s(\lambda)$ -sized adversaries $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} \text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t) = 1 \\ (C, x_0, x_t, t) \notin \mathcal{L}_{\text{IVC}} \\ t \leq T \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma) \\ (x_0, x_t, t, \pi_t) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \epsilon(\lambda)$$

Additionally, we say that the scheme satisfies (adaptive) (s, ϵ) -zero knowledge if the following holds:

(s, ϵ) -(Adaptive) Zero Knowledge: There exists a PPT simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ such that for all sufficiently large λ , all iteration bounds $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$ where $t \leq T$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , and all $s = s(\lambda)$ -sized adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,

$$\left| \Pr \left[\begin{array}{l} \mathcal{A}_2(\text{crs}, \pi_t, \text{st}_{\mathcal{A}}) = 1 \\ t \leq T \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma) \\ (x_0, w_1, w_2, \dots, w_t, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(\text{crs}) \\ \forall i \in [t], x_i = C(x_{i-1}, w_i), \\ \forall i \in [t], \pi_i \leftarrow \text{IVC.P}(\text{crs}, (x_0, x_{i-1}, i-1), \pi_{i-1}, w_i) \end{array} \right] \right. \\ \left. - \Pr \left[\begin{array}{l} \mathcal{A}_2(\text{crs}, \pi_t, \text{st}_{\mathcal{A}}) = 1 \\ t \leq T \end{array} : \begin{array}{l} (\text{crs}, \text{st}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^\lambda, T, 1^n, 1^m, C, 1^\gamma) \\ (x_0, w_1, w_2, \dots, w_t, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_1(\text{crs}) \\ \forall i \in [t], x_i = C(x_{i-1}, w_i), \\ \pi_t \leftarrow \text{Sim}_2(x_0, x_t, 1^t, \text{st}_{\text{Sim}}) \end{array} \right] \right| \leq \epsilon(\lambda)$$

Definition 5.4 (Perfect Zero Knowledge). We say that a scheme satisfies perfect zero-knowledge if the real and simulated zero knowledge distributions are identical (i.e. if the scheme is $(s, 0)$ -adaptive zero knowledge for every $s = s(\lambda)$).

Definition 5.5 (IVC for Trapdoor Languages with Full Succinctness.). We say that an IVC for Trapdoor Languages is fully succinct if on setup parameters $(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$, the proofs are of size $|\pi_i| = \text{poly}(\lambda, \log T, \log |C|, \log \gamma)$.

5.2 Parameters

We will build our scheme using the primitives specified in Theorem 5.1.

On security parameter λ , iteration bound T , input size n , witness size m , circuit C , and trapdoor circuit size γ we instantiate our primitives with the parameters shown below. Note that we will create three separate instantiations of our PPRF scheme, which can be differentiated based on the name of the PPRF key being used.

	Security Parameter	Input Size	Output Size	Circuit Size
PRG	λ_{PRG}	λ_{PRG}	$2\lambda_{\text{PRG}}$	
PPRF scheme for key K	λ_{PPRF}	$2n + \log T$	λ_{PRG}	
PPRF scheme for key K_{off}	λ_{PPRF}	$2n + \log T$	λ_{PRG}	
PPRF scheme for key K_{chal}	λ_{PRF}	$\log T$	λ_{PRG}	
$i\mathcal{O}$	$\lambda_{i\mathcal{O}}$			$\ell_{i\mathcal{O}}$

where

- $\lambda_{\text{PRG}} = (\lambda + \log T)^{1/\delta_{\text{PRG}}}.$
- $\lambda_{\text{PPRF}} = (\lambda + 2n + \log T)^{1/\delta_{\text{PPRF}}}.$
- $\lambda_{i\mathcal{O}} = (\lambda + 2n + \log T)^{1/\delta_{i\mathcal{O}}}.$
- $\ell_{i\mathcal{O}} = \text{poly}(\lambda, \log T, |C|, \gamma)$ is the maximum size of any of the circuits named P or V used in either our construction in Section 5.3 or in the proof of soundness in Section 5.5.

5.3 Construction

We now construct our IVC for Trapdoor Languages using the parameters and ingredients defined in the previous section.

Notation. For readability, we will use $F_K(x)$ to denote $\text{PPRF.Eval}(K, x)$ (and similarly for K_{off} and K_{chal}) and may omit some parameters from the setup algorithms of our primitives.

- $\text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma):$
 1. $K \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}}).$
 2. $\tilde{P} \leftarrow i\mathcal{O}(P[C, K])$ where P is padded to length $\ell_{i\mathcal{O}}$.

$P[C, K]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i):$

- (a) $x_i = C(x_{i-1}, w_i).$
- (b) if $i > 0$ and $\pi_{i-1} \neq F_K(x_0, x_{i-1}, i-1)$, output $\perp$.
- (c) Output $\pi_i = F_K(x_0, x_i, i).$

3. $\tilde{V} \leftarrow i\mathcal{O}(V[K])$ where V is padded to length $\ell_{i\mathcal{O}}$.

$V[K]((x_0, x_i, i), \pi_i):$

- (a) if $\pi_i \neq F_K(x_0, x_i, i)$, output 0.
- (b) Output 1.

4. Output $\text{crs} = (\tilde{P}, \tilde{V}).$
- $\text{IVC.P}(\text{crs}, (x_0, x_{i-1}, i-1), \pi_{i-1}, w_i):$
 1. Parse $\text{crs} = (\tilde{P}, \tilde{V}).$
 2. Output $\pi_i = \tilde{P}((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i).$
 - $\text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t):$
 1. Parse $\text{crs} = (\tilde{P}, \tilde{V}).$
 2. Output $\tilde{V}((x_0, x_t, t), \pi_t).$

5.4 Correctness, Efficiency, and Zero-Knowledge

Correctness. The correctness of our scheme follows in a straightforward manner from the correctness of the $i\mathcal{O}$ and PPRF.

Efficiency. We now argue that our scheme is *fully succinct*. That is, on setup parameters $(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$ where C is a polynomial-sized circuit $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which has a (γ, T) -trapdoor circuit Trap_C ,

- The proof size is $\text{poly}(\lambda, \log T)$.
The proof consists of a single PPRF output of size $\lambda_{\text{PRG}} = (\lambda + \log T)^{1/\delta_{\text{PRG}}} = \text{poly}(\lambda, \log T)$.
- All algorithms run in time $\text{poly}(\lambda, \log T, |C|, \gamma)$.
The crs consists of obfuscations of two circuits P and V of size $\ell_{i\mathcal{O}} = \text{poly}(\lambda, \log(T), |C|, \gamma)$. Thus, the crs has size $2 \text{poly}(\ell_{i\mathcal{O}}) = \text{poly}(\lambda, \log(T), |C|, \gamma)$. It is then easy to see that all of our algorithms run in time $\text{poly}(\lambda, \log T, |C|, \gamma)$.
- The crs size is $\text{poly}(\lambda, \log T, |C|, \gamma)$.
This follows since all algorithms run in time $\text{poly}(\lambda, \log T, |C|, \gamma)$.

Perfect (Adaptive) Zero-Knowledge. We construct a PPT simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ as follows:

- $\text{Sim}_1(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$:
 1. Compute $\text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$. (This is computed honestly.)
 2. Define $\text{st}_{\text{Sim}} = K$, where K is the PPRF key that was used in IVC.Gen .
 3. Output $(\text{crs}, \text{st}_{\text{Sim}})$.
- $\text{Sim}_2(x_0, x_t, 1^t, \text{st}_{\text{Sim}})$:
 1. Parse $\text{st}_{\text{Sim}} = K$.
 2. Output $F_K(x_0, x_t, t)$.

Note that the distribution of the crs is identical to the crs of an honest set-up, and an honestly generated proof for a true statement (x_0, x_t, t) is in fact equal to $F_K(x_0, x_t, t)$. Therefore, the simulated world is identical to the real world, and perfect zero-knowledge follows.

5.5 Soundness

We prove soundness via a hybrid argument.

Hybrid₀^A $(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$: This is the real world experiment.

1. $\text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$.
2. $(x_0, x_t, t, \pi_t) \leftarrow \mathcal{A}(\text{crs})$.
3. Output 1 if $\text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t) = 1$, $(C, x_0, x_t, t) \notin \mathcal{L}_{\text{IVC}}$, and $t \leq T^*$. Otherwise, output 0.

Making our hybrid efficient. We now use our trapdoor circuit Trap_C (given as non-uniform advice) to check whether $(C, x_0, x_t, t) \in \mathcal{L}_{\text{IVC}}$. This modification enables our hybrid (and all future hybrids) to be run by a polynomial-sized non-uniform adversary. For all remaining hybrids, we will continue to provide the reduction with the trapdoor circuit Trap_C as non-uniform advice.

Hybrid₁^A($1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C$):

1. $\text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$.
2. $(x_0, x_t, t, \pi_t) \leftarrow \mathcal{A}(\text{crs})$.
3. Output 1 if $\text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t) = 1$, $\text{Trap}_C(x_0, x_t, t) = 1$, and $t \leq T$. Otherwise, output 0.

Lemma 5.6. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , and all polynomial-sized adversaries $\mathcal{A}$,

$$\left| \Pr[\text{Hybrid}_1^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\text{Hybrid}_0^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma) = 1] \right| = 0$$

Proof. By the correctness of Trap_C , the hybrids have identical behavior. $\square$

Planting a challenge value in the proofs of false statements. For each $(\alpha, \beta, T^*) \notin \mathcal{L}_C$, we proceed through a sequence of subhybrids. The goal is to change the proof for each $(\alpha, \beta, T^*) \notin \mathcal{L}_C$ from $\pi = F_K(\alpha, \beta, T^*)$ to an alternate “off path” proof $\pi_{\text{off}} = F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$ which has a challenge value $y_{T^*} = F_{K_{\text{chal}}}(T^*)$ embedded into it. We briefly sketch the hybrids here before formally defining them below.

- **Hybrid_{2,(\alpha,\beta,T^*),0}^A**: We prove and verify tuples $(x_0, x_i, i) < (\alpha, \beta, T^*)$ where $(x_0, x_i, i) \notin \mathcal{L}_C$ using the off path proof $F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$.
- **Hybrid_{2,(\alpha,\beta,T^*),1}^A**: We puncture the PPRF keys K and K_{off} at (α, β, T^*) . Since P and V still need to know $z = F_K(\alpha, \beta, T^*)$, we precompute and hardwire this value into P and V . Since we are not changing the input/output behavior of P and V , we can rely on the security of $i\mathcal{O}$.
- **Hybrid_{2,(\alpha,\beta,T^*),2}^A**: We replace z with a uniformly random value using the security of the PPRF for K at the punctured point.
- **Hybrid_{2,(\alpha,\beta,T^*),3}^A**: We set $z = r \oplus F_{K_{\text{chal}}}(T^*)$ where r is a uniformly random value. This does not affect the distribution of z .
- **Hybrid_{2,(\alpha,\beta,T^*),4}^A**: We replace r with $F_{K_{\text{off}}}(\alpha, \beta, T^*)$ using the security of the PPRF for key K_{off} at the punctured point. This is equivalent to setting $z = F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$.
- **Hybrid_{2,(\alpha,\beta,T^*),5}^A**: We remove the puncturing of K and K_{off} and the hardwiring of z . Since we are not changing the input/output behavior of P and V , we can rely on the security of $i\mathcal{O}$.

Note that the final hybrid for (α, β, T^*) has the same behavior as the first hybrid for the next tuple $\text{succ}(\alpha, \beta, T^*) = (\alpha_{\text{succ}}, \beta_{\text{succ}}, T_{\text{succ}}^*) \notin \mathcal{L}_C$ after (α, β, T^*) .

Hybrid $_{2,(\alpha,\beta,T^*),0}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid** $_1^A$ except that we change IVC.Gen as follows:

- $\text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$:
 1. $K \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 2. $K_{\text{off}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 3. $K_{\text{chal}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 4. $\tilde{P} \leftarrow i\mathcal{O}(P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}])$ where P is padded to length $\ell_{i\mathcal{O}}$ and Trap_C is the trapdoor function for C .

$P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$:

(a) $x_i = C(x_{i-1}, w_i)$.

(b) **Verify** π_{i-1} : If $i > 0$,

- i. if $(x_0, x_{i-1}, i-1) < (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 A. if $\pi_{i-1} \neq F_{K_{\text{off}}}(x_0, x_{i-1}, i-1) \oplus F_{K_{\text{chal}}}(i-1)$, output $\perp$.
- ii. Else,
 A. if $\pi_{i-1} \neq F_K(x_0, x_{i-1}, i-1)$, output $\perp$.

(c) **Generate** π_i :

- i. if $(x_0, x_i, i) < (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 A. Output $\pi_i = F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$.
- ii. Else, output $\pi_i = F_K(x_0, x_i, i)$.

5. $\tilde{V} \leftarrow i\mathcal{O}(V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}])$ where V is padded to length $\ell_{i\mathcal{O}}$ and Trap_C is the trapdoor function for C .

$V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}]((x_0, x_i, i), \pi_i)$:

(a) if $(x_0, x_i, i) < (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 i. if $\pi_i \neq F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$, output 0.

(b) Else,
 i. if $\pi_i \neq F_K(x_0, x_i, i)$, output 0.

(c) Output 1.

6. Output $\text{crs} = (\tilde{P}, \tilde{V})$.

Lemma 5.7. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , and all polynomial-sized adversaries $\mathcal{A}$,

$$\left| \Pr[\mathbf{Hybrid}_{2,(\alpha_0, \beta_0, T_0^*), 0}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_1^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_{i\mathcal{O}}^\delta}.$$

where $(\alpha_0, \beta_0, T_0^*)$ is the lexicographically first tuple such that $\text{Trap}_C(\alpha_0, \beta_0, T_0^*) = 0$, $\alpha_0, \beta_0 \in \{0, 1\}^n$, and $T_0^* \leq T$.

Proof. Since $(\alpha_0, \beta_0, T_0^*)$ is the lexicographically first tuple such that $\text{Trap}_C(\alpha_0, \beta_0, T_0^*) = 0$, then there are no values of $(x_0, x_{i-1}, i-1)$ or (x_0, x_i, i) which are smaller than $(\alpha_0, \beta_0, T_0^*)$ and which cause Trap_C to output 0. Thus, the new branches of computation in P and V are never invoked which means that P and V in $\mathbf{Hybrid}_{2,(\alpha_0, \beta_0, T_0^*),0}^A$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_1^A$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

Hybrid $_{2,(\alpha, \beta, T^*),1}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid** $_{2,(\alpha, \beta, T^*),0}^A$ except that we change IVC.Gen as follows:

- IVC.Gen($1^\lambda, T, 1^n, 1^m, C, 1^\gamma$):
 1. $K \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 2. $K_{\text{off}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 3. $K_{\text{chal}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 4. $K\{\alpha, \beta, T^*\} \leftarrow \text{PPRF.Punc}(K, (\alpha, \beta, T^*))$.
 5. $K_{\text{off}}\{\alpha, \beta, T^*\} \leftarrow \text{PPRF.Punc}(K_{\text{off}}, (\alpha, \beta, T^*))$.
 6. $z = F_K(\alpha, \beta, T^*)$.
 7. $\tilde{P} \leftarrow i\mathcal{O}(P[C, \text{Trap}_C, K\{\alpha, \beta, T^*\}, K_{\text{off}}\{\alpha, \beta, T^*\}, K_{\text{chal}}, z])$ where P is padded to length $\ell_{i\mathcal{O}}$ and Trap_C is the trapdoor function for C .

$P[C, \text{Trap}_C, K\{\alpha, \beta, T^*\}, K_{\text{off}}\{\alpha, \beta, T^*\}, K_{\text{chal}}, z]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$:

- (a) $x_i = C(x_{i-1}, w_i)$.
- (b) **Verify** π_{i-1} : If $i > 0$,
 - i. if $(x_0, x_{i-1}, i-1) < (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 - A. if $\pi_{i-1} \neq F_{K_{\text{off}}\{\alpha, \beta, T^*\}}(x_0, x_{i-1}, i-1) \oplus F_{K_{\text{chal}}}(i-1)$, output $\perp$.
 - ii. **Else if** $(x_0, x_{i-1}, i-1) = (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$
 - A. if $\pi_{i-1} \neq z$, output $\perp$.
 - iii. Else,
 - A. if $\pi_{i-1} \neq F_{K\{\alpha, \beta, T^*\}}(x_0, x_{i-1}, i-1)$, output $\perp$.
- (c) **Generate** π_i :
 - i. if $(x_0, x_i, i) < (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 - A. Output $\pi_i = F_{K_{\text{off}}\{\alpha, \beta, T^*\}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$.
 - ii. **Else if** $(x_0, x_i, i) = (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 - A. Output $\pi_i = z$.
 - iii. Else, output $\pi_i = F_{K\{\alpha, \beta, T^*\}}(x_0, x_i, i)$.

8. $\tilde{V} \leftarrow i\mathcal{O}(V[\text{Trap}_C, K\{\alpha, \beta, T^*\}, K_{\text{off}}\{\alpha, \beta, T^*\}, K_{\text{chal}}, z])$ where V is padded to length $\ell_{i\mathcal{O}}$ and Trap_C is the trapdoor function for C .

$V[\text{Trap}_C, K\{\alpha, \beta, T^*\}, K_{\text{off}}\{\alpha, \beta, T^*\}, K_{\text{chal}}, z]((x_0, x_i, i), \pi_i)$:

- (a) if $(x_0, x_i, i) < (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 - i. if $\pi_i \neq F_{K_{\text{off}}\{\alpha, \beta, T^*\}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$, output 0.
- (b) **Else if** $(x_0, x_i, i) = (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 - i. **if** $\pi_i \neq z$, **output 0.**
- (c) Else,
 - i. if $\pi_i \neq F_{K\{\alpha, \beta, T^*\}}(x_0, x_i, i)$, output 0.
- (d) Output 1.

9. Output $\text{crs} = (\tilde{P}, \tilde{V})$.

Lemma 5.8. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, all $T^* \leq T$, and all $\alpha, \beta \in \{0, 1\}^n$ such that $(\alpha, \beta, T^*) \notin \mathcal{L}_C$.

$$\left| \Pr[\mathbf{Hybrid}_{2,(\alpha, \beta, T^*),1}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_{2,(\alpha, \beta, T^*),0}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}}.$$

Proof. Observe that since $(\alpha, \beta, T^*) \notin \mathcal{L}_{\text{IVC}}$, then we have $\text{Trap}_C(\alpha, \beta, T^*) = 0$. Thus, P and V in $\mathbf{Hybrid}_{2,(T^*, \alpha, \beta),1}^{\mathcal{A}}$ will never need to compute $F_{K\{\alpha, \beta, T^*\}}$ on its punctured point (α, β, T^*) . Furthermore, P and V will never need to compute $F_{K_{\text{off}}\{\alpha, \beta, T^*\}}$ on its punctured point since P and V only use K_{off} on points smaller than (α, β, T^*) . Therefore, by the correctness of the PPRF punctured keys and since we set $z = F_K(\alpha, \beta, T^*)$ accordingly, then P and V in $\mathbf{Hybrid}_{2,(T^*, \alpha, \beta),1}^{\mathcal{A}}$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_{2,(T^*, \alpha, \beta),0}^{\mathcal{A}}$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

Hybrid $_{2,(\alpha, \beta, T^*),2}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid** $_{2,(\alpha, \beta, T^*),1}^{\mathcal{A}}$ except that we change step 6 of IVC.Gen as follows:

6. $z \leftarrow \{0, 1\}^{\lambda_{\text{PRG}}}$

Lemma 5.9. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, all $T^* \leq T$, and all $\alpha, \beta \in \{0, 1\}^n$ such that $(\alpha, \beta, T^*) \notin \mathcal{L}_C$.

$$\left| \Pr[\mathbf{Hybrid}_{2,(\alpha, \beta, T^*),2}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_{2,(\alpha, \beta, T^*),1}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2^{-\lambda_{\text{PPRF}}^{\delta_{\text{PPRF}}}}.$$

Proof. Observe that we can run both hybrids without knowing K as long as we are given $K\{\alpha, \beta, T^*\}$ and z . Thus, the lemma follows using a straightforward reduction to the security of the PPRF. $\square$

Hybrid $_{2,(T^*,\alpha,\beta),3}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid** $_{2,(T^*,\alpha,\beta),2}^A$ except that we change step 6 of IVC.Gen as follows:

6. Sample $r \leftarrow \{0, 1\}^{\lambda_{\text{PRG}}}$ and set $z = r \oplus F_{K_{\text{chal}}}(T^*)$

Lemma 5.10. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, all $T^* \leq T$, and all $\alpha, \beta \in \{0, 1\}^n$ such that $(\alpha, \beta, T^*) \notin \mathcal{L}_C$.

$$\left| \Pr[\mathbf{Hybrid}_{2,(\alpha,\beta,T^*),3}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_{2,(\alpha,\beta,T^*),2}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| = 0.$$

Proof. The value z is identically distributed in both hybrids, so the hybrids have identical output distributions. $\square$

Hybrid $_{2,(T^*,\alpha,\beta),4}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid** $_{2,(T^*,\alpha,\beta),3}^A$ except that we change step 6 of IVC.Gen as follows:

6. $z = F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$

Lemma 5.11. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, all $T^* \leq T$, and all $\alpha, \beta \in \{0, 1\}^n$ such that $(\alpha, \beta, T^*) \notin \mathcal{L}_C$.

$$\left| \Pr[\mathbf{Hybrid}_{2,(\alpha,\beta,T^*),4}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_{2,(\alpha,\beta,T^*),3}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2^{-\lambda_{\text{PPRF}}^{\delta_{\text{PPRF}}}}.$$

Proof. Observe that we can run both hybrids without knowing K_{off} as long as we are given the punctured key $K_{\text{off}}\{\alpha, \beta, T^*\}$ along with r where r is either a random value or equal to $F_{K_{\text{off}}}(\alpha, \beta, T^*)$. Thus, the lemma follows using a straightforward reduction to the security of the PPRF. $\square$

Hybrid $_{2,(\alpha,\beta,T^*),5}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid** $_{2,(\alpha,\beta,T^*),4}^A$ except that we change IVC.Gen as follows:

- IVC.Gen($1^\lambda, T, 1^n, 1^m, C, 1^\gamma$):
 1. $K \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 2. $K_{\text{off}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 3. $K_{\text{chal}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 4. $K\{\alpha, \beta, T^*\} \leftarrow \text{PPRF.Punc}(K, (\alpha, \beta, T^*))$.
 5. $K_{\text{off}}\{\alpha, \beta, T^*\} \leftarrow \text{PPRF.Punc}(K_{\text{off}}, (\alpha, \beta, T^*))$.
 6. $z = F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$.
 7. $\tilde{P} \leftarrow i\mathcal{O}[P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}]]$ where P is padded to length $\ell_{i\mathcal{O}}$ and Trap_C is the trapdoor function for C .

$P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}]]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i):$

- (a) $x_i = C(x_{i-1}, w_i)$.
- (b) **Verify** π_{i-1} : If $i > 0$,
 - i. if $(x_0, x_{i-1}, i-1) < (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 - A. if $\pi_{i-1} \neq F_{K_{\text{off}}}(x_0, x_{i-1}, i-1) \oplus F_{K_{\text{chal}}}(i-1)$, output $\perp$.
 - ii. Else if $(x_0, x_{i-1}, i-1) = (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$
 - A. if $\pi_{i-1} \neq F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$, output $\perp$.
 - iii. Else,
 - A. if $\pi_{i-1} \neq F_K(x_0, x_{i-1}, i-1)$, output $\perp$.
- (c) **Generate** π_i :
 - i. if $(x_0, x_i, i) < (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 - A. Output $\pi_i = F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$.
 - ii. Else if $(x_0, x_i, i) = (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 - A. Output $\pi_i = F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$.
 - iii. Else, output $\pi_i = F_K(x_0, x_i, i)$.

8. $\tilde{V} \leftarrow i\mathcal{O}(V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}])$ where V is padded to length $\ell_{i\mathcal{O}}$ and Trap_C is the trapdoor function for C .

$V[\text{Trap}_C, K\{\alpha, \beta, T^*\}, K_{\text{off}}\{\alpha, \beta, T^*\}, K_{\text{chal}}, z]((x_0, x_i, i), \pi_i):$

- (a) if $(x_0, x_i, i) < (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 - i. if $\pi_i \neq F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$, output 0.
- (b) Else if $(x_0, x_i, i) = (\alpha, \beta, T^*)$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 - i. if $\pi_i \neq F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$, output 0.
- (c) Else,
 - i. if $\pi_i \neq F_{K\{\alpha, \beta, T^*\}}(x_0, x_i, i)$, output 0.
- (d) Output 1.

9. Output $\text{crs} = (\tilde{P}, \tilde{V})$.

Lemma 5.12. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, all $T^* \leq T$, and all $\alpha, \beta \in \{0, 1\}^n$ such that $(\alpha, \beta, T^*) \notin \mathcal{L}_C$.

$$\left| \Pr[\text{Hybrid}_{2,(\alpha, \beta, T^*), 5}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\text{Hybrid}_{2,(\alpha, \beta, T^*), 4}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}}.$$

Proof. By the correctness of the PPRF punctured keys and since we had previously set $z = F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$, P and V in $\text{Hybrid}_{2,(\alpha, \beta, T^*), 5}^{\mathcal{A}}$ are functionally equivalent to their counterparts in $\text{Hybrid}_{2,(\alpha, \beta, T^*), 4}^{\mathcal{A}}$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

Lemma 5.13. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all

polynomial-sized adversaries $\mathcal{A}$, all $T^* \leq T$, and all $\alpha, \beta \in \{0, 1\}^n$ such that $(\alpha, \beta, T^*) \notin \mathcal{L}_C$.

$$\left| \Pr[\mathbf{Hybrid}_{2,(\alpha_{\text{succ}}, \beta_{\text{succ}}, T_{\text{succ}}^*),0}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right. \\ \left. - \Pr[\mathbf{Hybrid}_{2,(\alpha\beta, T^*),5}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}}.$$

where $\text{succ}(\alpha, \beta, T^*) = (\alpha_{\text{succ}}, \beta_{\text{succ}}, T_{\text{succ}}^*)$ is the lexicographically first tuple larger than (α, β, T^*) that satisfies $\text{Trap}_C(\alpha_{\text{succ}}, \beta_{\text{succ}}, T_{\text{succ}}^*) = 0$, $\alpha_{\text{succ}}, \beta_{\text{succ}} \in \{0, 1\}^n$, and $T_{\text{succ}}^* \leq T$.

Proof. It is easy to see that P and V in $\mathbf{Hybrid}_{2,(\alpha_{\text{succ}}, \beta_{\text{succ}}, T_{\text{succ}}^*),0}^{\mathcal{A}}$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_{2,(\alpha\beta, T^*),5}^{\mathcal{A}}$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

Adding a PRG. After concluding the previous hybrids, the proof for each $(\alpha, \beta, T^*) \notin \mathcal{L}_C$ has been changed to its off path proof $F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$. Now, instead of verifying the off path proofs by checking whether $\pi = F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$, we instead verify them by checking whether $G(\pi \oplus F_{K_{\text{off}}}(\alpha, \beta, T^*)) = G(F_{K_{\text{chal}}}(T^*))$ where G is an injective PRG. This turns the challenge value $y_{T^*} = F_{K_{\text{chal}}}(T^*)$ for each T^* into a PRG seed which is useful in future hybrids.

Hybrid₃^A($1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C$) : This hybrid is the same as **Hybrid_{2,(\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*),5}^A** where $(\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*)$ is the lexicographically last tuple such that $\text{Trap}_C(\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*) = 0$, $\alpha_{\text{fin}}, \beta_{\text{fin}} \in \{0, 1\}^n$, and $T_{\text{fin}}^* \leq T$ except that we change IVC.Gen as follows:

- **IVC.Gen**($1^\lambda, T, 1^n, 1^m, C, 1^\gamma$):
 1. $K \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 2. $K_{\text{off}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 3. $K_{\text{chal}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 4. $\tilde{P} \leftarrow i\mathcal{O}(P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G])$ where P is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is the injective PRG.

$P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$:

- (a) $x_i = C(x_{i-1}, w_i)$.
- (b) **Verify** π_{i-1} : If $i > 0$,
 - i. if $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 - A. if $G(\pi_{i-1} \oplus F_{K_{\text{off}}}(x_0, x_{i-1}, i-1)) \neq G(F_{K_{\text{chal}}}(i-1))$, output $\perp$.
 - ii. Else,
 - A. if $\pi_{i-1} \neq F_K(x_0, x_{i-1}, i-1)$, output $\perp$.
- (c) **Generate** π_i :
 - i. if $\text{Trap}_C(x_0, x_i, i) = 0$,
 - A. Output $\pi_i = F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$.
 - ii. Else, output $\pi_i = F_K(x_0, x_i, i)$.

5. $\tilde{V} \leftarrow i\mathcal{O}(V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G])$ where V is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G]((x_0, x_i, i), \pi_i)$:

- (a) if $\text{Trap}_C(x_0, x_i, i) = 0$,
 - i. if $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) \neq G(F_{K_{\text{chal}}}(i))$, output 0.
- (b) Else,
 - i. if $\pi_i \neq F_K(x_0, x_i, i)$, output 0.
- (c) Output 1.

6. Output $\text{crs} = (\tilde{P}, \tilde{V})$.

Lemma 5.14. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , and all polynomial-sized adversaries $\mathcal{A}$,

$$\left| \Pr[\mathbf{Hybrid}_3^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_{2,(\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*),5}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}}.$$

where $(\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*)$ is the lexicographically last tuple such that $\text{Trap}_C(\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*) = 0$, $\alpha_{\text{fin}}, \beta_{\text{fin}} \in \{0, 1\}^n$, and $T_{\text{fin}}^* \leq T$.

Proof. In $\mathbf{Hybrid}_3^{\mathcal{A}}$ we have changed the proof for each false statement $(\alpha, \beta, T^*) \notin \mathcal{L}_C$ where $(\alpha, \beta, T^*) \leq (\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*)$ to its off path proof $F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$. Since $(\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*)$ is the lexicographically last such false tuple, we no longer need to explicitly check for $(\alpha, \beta, T^*) \leq (\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*)$. Furthermore, since G is injective, checking whether $\pi \neq F_{K_{\text{off}}}(\alpha, \beta, T^*) \oplus F_{K_{\text{chal}}}(T^*)$ yields the same output as checking whether $G(\pi \oplus F_{K_{\text{off}}}(\alpha, \beta, T^*)) \neq G(F_{K_{\text{chal}}}(T^*))$. Thus, P and V in $\mathbf{Hybrid}_3^{\mathcal{A}}$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_{2,(\alpha_{\text{fin}}, \beta_{\text{fin}}, T_{\text{fin}}^*),5}^{\mathcal{A}}$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

Removing the proofs of false statements. For each T^* from 0 to T , we use a sequence of subhybrids to remove the proofs of all false statements $(x_0, x_i, i) \notin \mathcal{L}_C$ where $i = T^*$. We briefly sketch the hybrids here before formally defining them below.

- **Hybrid** $_{4,T^*,0}^{\mathcal{A}}$: We no longer prove or verify false tuples $(x_0, x_i, i) \notin \mathcal{L}_C$ where $i < T^*$. Instead, when we see such tuples, we immediately halt and output $\perp$.
- **Hybrid** $_{4,T^*,1}^{\mathcal{A}}$: For $i = T^*$, we will no longer output proofs for false tuples $(x_0, x_{T^*}, T^*) \notin \mathcal{L}_C$ (though we will still allow such tuples to be verified with an off path proof). This removes the need for the proof generating step to compute $y = F_{K_{\text{chal}}}(T^*)$. This change is justified by observing that if $(x_0, x_{T^*}, T^*) \notin \mathcal{L}_C$, then P will not pass the verification step in the first place and will thus not need to output a proof. Observe that if $(x_0, x_{T^*}, T^*) \notin \mathcal{L}_C$ and if $x_{T^*} = C(x_{T^*-1}, w_{T^*})$ is correctly computed from x_{T^*-1} , it must be the case that $(x_0, x_{T^*-1}, T^*-1) \notin \mathcal{L}_C$. However, since $T^*-1 < T^*$, our previous hybrid ensured that tuples $(x_0, x_{T^*-1}, T^*-1) \notin \mathcal{L}_C$ cannot be verified by any proof.
- **Hybrid** $_{4,T^*,2}^{\mathcal{A}}$: We puncture the PPRF key K_{chal} on T^* . Since P and V still need to know $z = G(y) = G(F_{K_{\text{chal}}}(T^*))$ for verification, we precompute and hardwire z into P and V . Since we are not changing the input/output behavior of P and V , we can rely on the security of $i\mathcal{O}$.

- **Hybrid $_{4,T^*,3}^A$** : We replace y with a uniformly random value using the security of the PPRF for K_{chal} at the punctured point.
- **Hybrid $_{4,T^*,4}^A$** : We replace z with a uniformly random value using the security of the PRG G .
- **Hybrid $_{4,T^*,5}^A$** : We no longer verify proofs of false statements $(x_0, x_{T^*}, T^*) \notin \mathcal{L}_C$, but instead immediately output $\perp$. Observe that a valid proof of such a statement would allow P to find a preimage under G of z . However, since G is length-doubling and z is uniformly random, with all but negligible probability, z is not in the image of G and thus has no valid preimage.
- **Hybrid $_{4,T^*,6}^A$** : We remove the puncturing of K_{chal} . Since we are not changing the input/output behavior of P and V , we can rely on the security of $i\mathcal{O}$.

Note that the final hybrid for T^* has the same behavior as the first hybrid for $T^* + 1$.

Hybrid $_{4,T^*,0}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid $_3^A$** except that we change IVC.Gen as follows:

- $\text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$:
 1. $K \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 2. $K_{\text{off}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 3. $K_{\text{chal}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 4. $\tilde{P} \leftarrow i\mathcal{O}(P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G])$ where P is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$:

(a) $x_i = C(x_{i-1}, w_i)$.

(b) **Verify** π_{i-1} : If $i > 0$,

i. if $i-1 < T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,

A. Output $\perp$.

ii. Else if $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,

A. if $G(\pi_{i-1} \oplus F_{K_{\text{off}}}(x_0, x_{i-1}, i-1)) \neq G(F_{K_{\text{chal}}}(i-1))$, output $\perp$.

iii. Else,

A. if $\pi_{i-1} \neq F_K(x_0, x_{i-1}, i-1)$, output $\perp$.

(c) **Generate** π_i :

i. if $i < T^*$ and $\text{Trap}_C(x_0, x_i, i) = 0$,

A. Output $\perp$.

ii. Else if $\text{Trap}_C(x_0, x_i, i) = 0$,

A. Output $\pi_i = F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$.

iii. Else, output $\pi_i = F_K(x_0, x_i, i)$.

5. $\tilde{V} \leftarrow i\mathcal{O}(V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G])$ where V is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G]((x_0, x_i, i), \pi_i)$:

- (a) **if $i < T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,**
 - i. Output 0.**
- (b) **Else if $\text{Trap}_C(x_0, x_i, i) = 0$,**
 - i. if $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) \neq G(F_{K_{\text{chal}}}(i))$, output 0.**
- (c) **Else,**
 - i. if $\pi_i \neq F_K(x_0, x_i, i)$, output 0.**
- (d) **Output 1.**

6. Output $\text{crs} = (\tilde{P}, \tilde{V})$.

Lemma 5.15. *For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , and all polynomial-sized adversaries $\mathcal{A}$,*

$$\left| \Pr[\mathbf{Hybrid}_{4,0,0}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_3^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_i^{\delta_i \mathcal{O}}}.$$

Proof. When $T^* = 0$, the new branches of computation in P and V are never invoked which means that P and V in $\mathbf{Hybrid}_{4,0,0}^{\mathcal{A}}$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_3^{\mathcal{A}}$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

$\mathbf{Hybrid}_{4,T^*,1}^{\mathcal{A}}$ $(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: For $i = T^*$, we no longer generate proofs for $(x_0, x_i, i) \notin \mathcal{L}_C$. This hybrid is the same as $\mathbf{Hybrid}_{4,T^*,0}^{\mathcal{A}}$ except that we change step 4 of IVC.Gen as follows:

- 4. $\tilde{P} \leftarrow i\mathcal{O}(P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G])$ where P is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$:

1. $x_i = C(x_{i-1}, w_i)$.
2. **Verify** π_{i-1} : If $i > 0$,
 - (a) if $i-1 < T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 - i. Output $\perp$.
 - (b) Else if $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 - i. if $G(\pi_{i-1} \oplus F_{K_{\text{off}}}(x_0, x_{i-1}, i-1)) \neq G(F_{K_{\text{chal}}}(i-1))$, output $\perp$.
 - (c) Else,
 - i. if $\pi_{i-1} \neq F_K(x_0, x_{i-1}, i-1)$, output $\perp$.
3. **Generate** π_i :
 - (a) if $i < T^* + 1$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
 - i. Output $\perp$.
 - (b) Else if $\text{Trap}_C(x_0, x_i, i) = 0$,
 - i. Output $\pi_i = F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}}(i)$.
 - (c) Else, output $\pi_i = F_K(x_0, x_i, i)$.

Lemma 5.16. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, and all $T^* \leq T$,

$$\left| \Pr[\mathbf{Hybrid}_{4,T^*,1}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_{4,T^*,0}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}}.$$

Proof. If $x_i = C(x_{i-1}, w_i)$ and $(x_0, x_i, i) \notin \mathcal{L}_C$, then by definition of $\mathcal{L}_C$, it must be the case that $(x_0, x_{i-1}, i-1) \notin \mathcal{L}_C$. Thus, when $i = T^*$, since P must verify that $\text{Trap}_C(x_0, x_{i-1}, i-1) = 1$ (i.e. that $(x_0, x_{i-1}, i-1) \in \mathcal{L}_C$), it is impossible for P to get past the verification step and end up with $(x_0, x_i, i) \notin \mathcal{L}_C$. Therefore, the altered branch of computation in P is never invoked on $i = T^*$ which means that P in $\mathbf{Hybrid}_{4,0,1,1}^{\mathcal{A}}$ is functionally equivalent to its counterpart in $\mathbf{Hybrid}_3^{\mathcal{A}}$. The lemma then follows using a straightforward reductions to the security of $i\mathcal{O}$. $\square$

Hybrid $_{4,T^*,2}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid** $_{4,T^*,1}^{\mathcal{A}}$ except that we change IVC.Gen as follows:

- IVC.Gen($1^\lambda, T, 1^n, 1^m, C, 1^\gamma$):
 1. $K \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 2. $K_{\text{off}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 3. $K_{\text{chal}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 4. $K_{\text{chal}}\{T^*\} \leftarrow \text{PPRF.Punc}(K_{\text{chal}}, T^*)$.
 5. $y = F_{K_{\text{chal}}}(T^*)$.
 6. $z = G(y)$.

7. $\tilde{P} \leftarrow i\mathcal{O}(P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}\{T^*\}, G, z])$ where P is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}\{T^*\}, G, z]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$:

- (a) $x_i = C(x_{i-1}, w_i)$.
- (b) **Verify** π_{i-1} : If $i > 0$,
 - i. if $i < T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
A. Output $\perp$.
 - ii. if $i = T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
A. if $G(\pi_{i-1} \oplus F_{K_{\text{off}}}(x_0, x_{i-1}, i-1)) \neq z$, output $\perp$.
 - iii. Else if $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
A. if $G(\pi_{i-1} \oplus F_{K_{\text{off}}}(x_0, x_{i-1}, i-1)) \neq G(F_{K_{\text{chal}}\{T^*\}}(i-1))$, output $\perp$.
 - iv. Else,
A. if $\pi_{i-1} \neq F_K(x_0, x_{i-1}, i-1)$, output $\perp$.
- (c) **Generate** π_i :
 - i. if $i < T^* + 1$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
A. Output $\perp$.
 - ii. Else if $\text{Trap}_C(x_0, x_i, i) = 0$,
A. Output $\pi_i = F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}\{T^*\}}(i)$.
 - iii. Else, output $\pi_i = F_K(x_0, x_i, i)$.

8. $\tilde{V} \leftarrow i\mathcal{O}(V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}\{T^*\}, G, z])$ where V is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}\{T^*\}, G, z]((x_0, x_i, i), \pi_i)$:

- (a) if $i < T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
i. Output 0.
- (b) if $i = T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
i. if $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) \neq z$, output 0.
- (c) Else if $\text{Trap}_C(x_0, x_i, i) = 0$,
i. if $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) \neq G(F_{K_{\text{chal}}\{T^*\}}(i))$, output 0.
- (d) Else,
i. if $\pi_i \neq F_K(x_0, x_i, i)$, output 0.
- (e) Output 1.

9. Output $\text{crs} = (\tilde{P}, \tilde{V})$.

Lemma 5.17. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all

polynomial-sized adversaries $\mathcal{A}$, and all $T^* \leq T$,

$$\left| \Pr[\mathbf{Hybrid}_{4,T^*,2}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right. \\ \left. - \Pr[\mathbf{Hybrid}_{4,T^*,1}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}}.$$

Proof. We observe that P and V will never need to compute $F_{K_{\text{chal}}}$ on the punctured point T^* . Thus, by the correctness of the PPRF punctured keys and since we set $z = G(K_{\text{chal}}(T^*))$, then P and V in $\mathbf{Hybrid}_{4,T^*,2}^{\mathcal{A}}$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_{4,T,1}^{\mathcal{A}}$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

Hybrid $_{4,T^*,3}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: We replace y with a random value. This hybrid is the same as **Hybrid** $_{4,T^*,2}^{\mathcal{A}}$ except that we change step 5 of IVC.Gen as below:

5. $y \leftarrow \{0, 1\}^{\lambda_{\text{PRG}}}$

Lemma 5.18. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, and all $T^* \leq T$,

$$\left| \Pr[\mathbf{Hybrid}_{4,T^*,3}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right. \\ \left. - \Pr[\mathbf{Hybrid}_{4,T^*,2}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2^{-\lambda_{\text{PPRF}}^{\delta_{\text{PPRF}}}}.$$

Proof. Observe that we can run both hybrids without knowing K_{chal} as long as we are given $K_{\text{chal}}\{T^*\}$ and y . Thus, the lemma follows using a straightforward reduction to the security of the PPRF. $\square$

Hybrid $_{4,T^*,4}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: We replace z with a random value. This hybrid is the same as **Hybrid** $_{4,T^*,3}^{\mathcal{A}}$ except that we change step 6 of IVC.Gen as below:

6. $z \leftarrow \{0, 1\}^{2\lambda_{\text{PRG}}}$

Lemma 5.19. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, and all $T^* \leq T$,

$$\left| \Pr[\mathbf{Hybrid}_{4,T^*,4}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right. \\ \left. - \Pr[\mathbf{Hybrid}_{4,T^*,3}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2^{-\lambda_{\text{PRG}}^{\delta_{\text{PRG}}}}$$

Proof. Observe that $z = G(y)$ for a random value y that is not used anywhere else in the hybrid. Thus, the lemma follows using a straightforward reduction to the security of the PRG. $\square$

Hybrid $_{4,T^*,5}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid** $_{4,T^*,4}^A$ except that we change IVC.Gen as follows:

- $\text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$:
 1. $K \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 2. $K_{\text{off}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 3. $K_{\text{chal}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 4. $K_{\text{chal}}\{T^*\} \leftarrow \text{PPRF.Punc}(K_{\text{chal}}, T^*)$.
 5. ~~$y \leftarrow \{0,1\}^{\lambda_{\text{PRG}}}$.~~
 6. ~~$z \leftarrow \{0,1\}^{2\lambda_{\text{PRG}}}$.~~
 7. $\tilde{P} \leftarrow i\mathcal{O}(P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}\{T^*\}, G])$ where P is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}\{T^*\}, G]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$:

- (a) $x_i = C(x_{i-1}, w_i)$.
- (b) **Verify** π_{i-1} : If $i > 0$,
 - i. if $i < T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
A. Output $\perp$.
 - ii. if $i = T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
A. **Output $\perp$.**
 - iii. Else if $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
A. if $G(\pi_{i-1} \oplus F_{K_{\text{off}}}(x_0, x_{i-1}, i-1)) \neq G(F_{K_{\text{chal}}\{T^*\}}(i-1))$, output $\perp$.
 - iv. Else,
A. if $\pi_{i-1} \neq F_K(x_0, x_{i-1}, i-1)$, output $\perp$.
- (c) **Generate** π_i :
 - i. if $i < T^* + 1$ and $\text{Trap}_C(x_0, x_i, i) = 0$,
A. Output $\perp$.
 - ii. Else if $\text{Trap}_C(x_0, x_i, i) = 0$,
A. Output $\pi_i = F_{K_{\text{off}}}(x_0, x_i, i) \oplus F_{K_{\text{chal}}\{T^*\}}(i)$.
 - iii. Else, output $\pi_i = F_K(x_0, x_i, i)$.

8. $\tilde{V} \leftarrow i\mathcal{O}(V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}\{T^*\}, G])$ where V is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}\{T^*\}, G]((x_0, x_i, i), \pi_i)$:
 (a) if $i < T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 i. Output 0.
 (b) if $i = T^*$ and $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 i. **Output 0.**
 (c) Else if $\text{Trap}_C(x_0, x_i, i) = 0$,
 i. if $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) \neq G(F_{K_{\text{chal}}\{T^*\}}(i))$, output 0.
 (d) Else,
 i. if $\pi_i \neq F_K(x_0, x_i, i)$, output 0.
 (e) Output 1.

9. Output $\text{crs} = (\tilde{P}, \tilde{V})$.

Lemma 5.20. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, and all $T^* \leq T$,

$$\begin{aligned}
 & \left| \Pr[\mathbf{Hybrid}_{4,T^*,5}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right. \\
 & \quad \left. - \Pr[\mathbf{Hybrid}_{4,T^*,4}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2^{-\lambda_{\text{PRG}}} + 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_i\mathcal{O}}}.
 \end{aligned}$$

Proof. For $\mathbf{Hybrid}_{4,T^*,4}^{\mathcal{A}}$, we have two cases.

- Case 1: $z \leftarrow \{0, 1\}^{2\lambda}$ has a preimage under G .
 This happens with probability $\frac{\text{size of domain of } G}{\text{size of codomain of } G} \leq \frac{2^{\lambda_{\text{PRG}}}}{2^{2\lambda_{\text{PRG}}}} = 2^{-\lambda_{\text{PRG}}}$.
- Case 2: $z \leftarrow \{0, 1\}^{2\lambda}$ has no preimage under G .
 In this case, P and V in $\mathbf{Hybrid}_{4,T^*,5}^{\mathcal{A}}$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_{4,T^*,4}^{\mathcal{A}}$ since it will never be the case that $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_i, i)) = z$ or $G(\pi_i \oplus F_{K_{\text{off}}}(x_0, x_{i-1}, i-1)) = z$. Therefore, by two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V , the distinguishing advantage in this case is at most $2\text{Adv}_{i\mathcal{O}}(\lambda_{i\mathcal{O}})$.

Thus, the distinguishing advantage is at most $2^{-\lambda_{\text{PRG}}} \cdot 1 + (1 - 2^{-\lambda_{\text{PRG}}}) \cdot 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_i\mathcal{O}}} \leq 2^{-\lambda_{\text{PRG}}} + 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_i\mathcal{O}}}$. $\square$

Hybrid $_{4,T^*,6}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C)$: This hybrid is the same as **Hybrid** $_{4,T^*,5}^{\mathcal{A}}$ except that we no longer puncture K_{chal} at T^* . P and V will directly use K_{chal} rather than the punctured key $K_{\text{chal}}\{T^*\}$.

Lemma 5.21. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, and all $T^* \leq T$,

$$\begin{aligned}
 & \left| \Pr[\mathbf{Hybrid}_{4,T^*,6}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right. \\
 & \quad \left. - \Pr[\mathbf{Hybrid}_{4,T^*,5}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_i\mathcal{O}}}.
 \end{aligned}$$

Proof. By the correctness of the PPRF punctured keys, P and V in $\mathbf{Hybrid}_{4,T^*,6}^A$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_{4,T^*,5}^A$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

Lemma 5.22. *For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda), m = m(\lambda), \gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$, and all $T^* \leq T$,*

$$\left| \Pr[\mathbf{Hybrid}_{4,T^*,6}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_{4,T^*+1,0}^A(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}}.$$

Proof. It is easy to see that P and V in $\mathbf{Hybrid}_{4,T^*,6}^A$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_{4,T^*+1,0}^A$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

Final Hybrid. After concluding the previous hybrids for all $T^* \leq T$, we no longer allow proofs or verifications of any false statements $(x_0, x_i, i) \notin \mathcal{L}_C$ to be verified. Thus our final hybrid will be perfectly sound.

Hybrid₅^A($1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C$):

1. $\text{crs} \leftarrow \text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$.
2. $(x_0, x_t, 1^t, \pi_t) \leftarrow \mathcal{A}(\text{crs})$.
3. Output 1 if $\text{IVC.V}(\text{crs}, (x_0, x_t, t), \pi_t) = 1$, $\text{Trap}_C(x_0, x_t, t) = 1$, and $t \leq T$. Otherwise, output 0.

where we have modified IVC.Gen from our original construction as shown below:

- $\text{IVC.Gen}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma)$:
 1. $K \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 2. $K_{\text{off}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 3. $K_{\text{chal}} \leftarrow \text{PPRF.Setup}(1^{\lambda_{\text{PPRF}}})$.
 4. $\tilde{P} \leftarrow i\mathcal{O}(P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G])$ where P is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$P[C, \text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G]((x_0, x_{i-1}, i-1), \pi_{i-1}, w_i)$:

- (a) $x_i = C(x_{i-1}, w_i)$.
- (b) **Verify** π_{i-1} : If $i > 0$,
 - i. if $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 - A. Output $\perp$.
 - ii. Else,
 - A. if $\pi_{i-1} \neq F_K(x_0, x_{i-1}, i-1)$, output $\perp$.
- (c) **Generate** π_i :
 - i. if $\text{Trap}_C(x_0, x_i, i) = 0$,
 - A. Output $\perp$.
 - ii. Else, output $\pi_i = F_K(x_0, x_i, i)$.

5. $\tilde{V} \leftarrow i\mathcal{O}(V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G])$ where V is padded to length $\ell_{i\mathcal{O}}$, Trap_C is the trapdoor function for C , and G is our injective PRG.

$V[\text{Trap}_C, K, K_{\text{off}}, K_{\text{chal}}, G]((x_0, x_i, i), \pi_i)$:

- (a) if $\text{Trap}_C(x_0, x_{i-1}, i-1) = 0$,
 - i. Output 0.
- (b) Else,
 - i. if $\pi_i \neq F_K(x_0, x_i, i)$, output 0.
- (c) Output 1.

6. Output $\text{crs} = (\tilde{P}, \tilde{V})$.

Lemma 5.23. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$,

$$\left| \Pr[\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] - \Pr[\mathbf{Hybrid}_{4,T+1,0}^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] \right| \leq 2 \cdot 2^{-\lambda_i^{\delta_{i\mathcal{O}}}}.$$

Proof. Since the index i can be at most T , then we do not need to explicitly check that $i < T^* + 1$ and do not need to include a case where $\text{Trap}_C(x_0, x_i, i) = 0$, but $i > T$. Removing these items from P and V in $\mathbf{Hybrid}_{4,T+1,0}^{\mathcal{A}}$ gives us $\mathbf{Hybrid}_{4,T+1,0}^{\mathcal{A}}$. Thus P and V in $\mathbf{Hybrid}_5^{\mathcal{A}}$ are functionally equivalent to their counterparts in $\mathbf{Hybrid}_5^{\mathcal{A}}$. The lemma then follows using two straightforward reductions to the security of $i\mathcal{O}$, once for P and once for V . $\square$

Lemma 5.24. For all sufficiently large λ , all $T \leq 2^\lambda$, all polynomials $n = n(\lambda)$, $m = m(\lambda)$, $\gamma = \gamma(\lambda)$, all polynomial-sized circuits $C : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ which have a (γ, T) -trapdoor circuit Trap_C , all polynomial-sized adversaries $\mathcal{A}$,

$$\Pr[\mathbf{Hybrid}_5^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma, \text{Trap}_C) = 1] = 0$$

Proof. The adversary only wins if they output (x_0, x_t, t, π_t) such that $\text{IVC}.\mathcal{V}(\text{crs}, (x_0, x_t, t), \pi_t) = 1$, $\text{Trap}_C(x_0, x_t, t) = 1$, and $t \leq T$. However, by the perfect correctness of $i\mathcal{O}$ since V in $\mathbf{Hybrid}_5^{\mathcal{A}}$ does

not verify any tuples where $\text{Trap}_C(x_0, x_t, t) = 1$ then neither does $\text{IVC}.\mathcal{V}$ which runs $\tilde{V}$. Thus, $\mathcal{A}$ cannot win the game. $\square$

Conclusion. By combining our lemmas and based on the parameters described in Section 5.2, we get that

$$\begin{aligned}
& \Pr[\mathbf{Hybrid}_0^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma) = 1] \\
& \leq 6 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}} \\
& \quad + 2^{2n+\log T} \cdot (6 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}} + 2 \cdot 2^{-\lambda_{\text{PPRF}}^{\delta_{\text{PPRF}}}}) \\
& \quad + T \cdot (8 \cdot 2^{-\lambda_{i\mathcal{O}}^{\delta_{i\mathcal{O}}}} + 2^{-\lambda_{\text{PPRF}}^{\delta_{\text{PPRF}}}} + 2^{-\lambda_{\text{PRG}}^{\delta_{\text{PRG}}}} + 2^{-\lambda_{\text{PRG}}}) \\
& \leq 6 \cdot 2^{-(\lambda+2n+\log T)} \\
& \quad + 2^{2n+\log T} \cdot (6 \cdot 2^{-(\lambda+2n+\log T)} + 2 \cdot 2^{-(\lambda+2n+\log T)}) \\
& \quad + T \cdot (8 \cdot 2^{-(\lambda+2n+\log T)} + 2^{-(\lambda+2n+\log T)} + 2^{-(\lambda+\log T)} + 2^{-(\lambda+\log T)^{1/\delta_{\text{PRG}}}}) \\
& = O(2^{-\lambda}).
\end{aligned}$$

Thus, since $\Pr[\mathbf{Hybrid}_0^{\mathcal{A}}(1^\lambda, T, 1^n, 1^m, C, 1^\gamma) = 1]$ is the probability that a polynomial-sized adversary breaks the soundness of our scheme, our construction of IVC for Trapdoor Languages is sound.

Acknowledgements

A. Jain was supported in part by NSF CAREER 1942789, Johns Hopkins University Catalyst award, JP Morgan Faculty Award, and research gifts from Ethereum Foundation, Stellar Development Foundation, and Cisco.

S. Mathialagan was supported in part by NSF CNS-2154149, a Simons Investigator Award and Jane Street.

A. Korb and A. Sahai were supported in part from a Simons Investigator Award, DARPA SIEVE award, NTT Research, NSF grant 2333935, BSF grant 2022370, a Xerox Faculty Research Award, a Google Faculty Research Award, an Okawa Foundation Research Grant, and the Symantec Chair of Computer Science. This material is based upon work supported by the Defense Advanced Research Projects Agency through Award HR00112020024.

Part of this work was done while A. Korb and S. Mathialagan were interning at NTT Research.

6 References

- [AS15] Gilad Asharov and Gil Segev. Limits on the power of indistinguishability obfuscation and functional encryption. In Venkatesan Guruswami, editor, *56th FOCS*, pages 191–209. IEEE Computer Society Press, October 2015.
- [BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. Recursive composition and bootstrapping for SNARKS and proof-carrying data. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 111–120. ACM Press, June 2013.
- [BCG24] Annalisa Barbara, Alessandro Chiesa, and Ziyi Guan. Relativized succinct arguments in the ROM do not exist. *Cryptology ePrint Archive*, Report 2024/728, 2024.
- [BCL⁺21] Benedikt Bünz, Alessandro Chiesa, William Lin, Pratyush Mishra, and Nicholas Spooner. Proof-carrying data without succinct arguments. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 681–710, Virtual Event, August 2021. Springer, Cham.
- [BCMS20] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. Recursive proof composition from accumulation schemes. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part II*, volume 12551 of *LNCS*, pages 1–18. Springer, Cham, November 2020.
- [BCTV14] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Scalable zero knowledge via cycles of elliptic curves. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 276–294. Springer, Berlin, Heidelberg, August 2014.
- [BDFG21] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. Halo infinite: Proof-carrying data from additive polynomial commitments. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 649–680, Virtual Event, August 2021. Springer, Cham.
- [BDSZ24] Pedro Branco, Nico Döttling, Akshayaram Srinivasan, and Riccardo Zanotto. Rate-1 fully local somewhere extractable hashing from DDH. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14603 of *LNCS*, pages 356–386. Springer, Cham, April 2024.
- [BGI⁺01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, August 2001.
- [BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudo-random functions. In Hugo Krawczyk, editor, *PKC 2014*, volume 8383 of *LNCS*, pages 501–519. Springer, Berlin, Heidelberg, March 2014.
- [BKP⁺24] Nir Bitansky, Chethan Kamath, Omer Paneth, Ron D. Rothblum, and Prashant Nalini Vasudevan. Batch proofs are statistically hiding. In Bojan Mohar, Igor Shinkar, and Ryan O’Donnell, editors, *56th ACM STOC*, pages 435–443. ACM Press, June 2024.

- [BMRS20] Joseph Bonneau, Izaak Meckler, Vanishree Rao, and Evan Shapiro. Coda: Decentralized cryptocurrency at scale. Cryptology ePrint Archive, Report 2020/352, 2020.
- [BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part II*, volume 8270 of *LNCS*, pages 280–300. Springer, Berlin, Heidelberg, December 2013.
- [CCDW20] Weikeng Chen, Alessandro Chiesa, Emma Dauterman, and Nicholas P. Ward. Reducing participation costs via incremental verification for ledger systems. Cryptology ePrint Archive, Report 2020/1522, 2020.
- [CCG⁺23] Megan Chen, Alessandro Chiesa, Tom Gur, Jack O’Connor, and Nicholas Spooner. Proof-carrying data from arithmetized random oracles. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 379–404. Springer, Cham, April 2023.
- [CGG⁺21] Arka Rai Choudhuri, Aarushi Goel, Matthew Green, Abhishek Jain, and Gabriel Kaptchuk. Fluid MPC: Secure multiparty computation with dynamic participants. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 94–123, Virtual Event, August 2021. Springer, Cham.
- [CGJ⁺23] Arka Rai Choudhuri, Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Jiaheng Zhang. Correlation intractability and SNARGs from sub-exponential DDH. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 635–668. Springer, Cham, August 2023.
- [CGKS23] Matteo Campanelli, Chaya Ganesh, Hamidreza Khoshakhlagh, and Janno Siim. Impossibilities in succinct arguments: Black-box extraction and more. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23*, volume 14064 of *LNCS*, pages 465–489. Springer, Cham, July 2023.
- [CGSY23] Alessandro Chiesa, Ziyi Guan, Shahar Samocha, and Eylon Yogev. Security bounds for proof-carrying data from straightline extractors. Cryptology ePrint Archive, Report 2023/1646, 2023.
- [Cha81] David L Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. *Communications of the ACM*, 24(2):84–90, 1981.
- [CJJ21] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. Non-interactive batch arguments for NP from standard assumptions. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 394–423, Virtual Event, August 2021. Springer, Cham.
- [CJJ22] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. SNARGs for $\mathcal{P}$ from LWE. In *62nd FOCS*, pages 68–79. IEEE Computer Society Press, February 2022.
- [CKLM12] Melissa Chase, Markulf Kohlweiss, Anna Lysyanskaya, and Sarah Meiklejohn. Malleable proof systems and applications. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 281–300. Springer, Berlin, Heidelberg, April 2012.

- [CKLM13] Melissa Chase, Markulf Kohlweiss, Anna Lysyanskaya, and Sarah Meiklejohn. Succinct malleable NIZKs and an application to compact shuffles. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 100–119. Springer, Berlin, Heidelberg, March 2013.
- [COS20] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. Fractal: Post-quantum and transparent recursive proofs from holography. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 769–793. Springer, Cham, May 2020.
- [CTV13] Stephen Chong, Eran Tromer, and Jeffrey A. Vaughan. Enforcing language semantics using proof-carrying data. *Cryptology ePrint Archive*, Report 2013/513, 2013.
- [DGKV22] Lalita Devadas, Rishab Goyal, Yael Kalai, and Vinod Vaikuntanathan. Rate-1 non-interactive arguments for batch-NP and applications. In *63rd FOCS*, pages 1057–1068. IEEE Computer Society Press, October / November 2022.
- [DWW24] Lalita Devadas, Brent Waters, and David J. Wu. Batching adaptively-sound SNARGs for NP. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part II*, volume 15365 of *LNCS*, pages 339–370. Springer, Cham, December 2024.
- [GGM84] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. On the cryptographic applications of random functions. In G. R. Blakley and David Chaum, editors, *CRYPTO’84*, volume 196 of *LNCS*, pages 276–288. Springer, Berlin, Heidelberg, August 1984.
- [GMR85] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *17th ACM STOC*, pages 291–304. ACM Press, May 1985.
- [GOS06] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Perfect non-interactive zero knowledge for NP. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 339–358. Springer, Berlin, Heidelberg, May / June 2006.
- [GSWW22] Rachit Garg, Kristin Sheridan, Brent Waters, and David J. Wu. Fully succinct batch arguments for NP from indistinguishability obfuscation. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 526–555. Springer, Cham, November 2022.
- [GW11] Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd ACM STOC*, pages 99–108. ACM Press, June 2011.
- [HAN23] Mathias Hall-Andersen and Jesper Buus Nielsen. On valiant’s conjecture: Impossibility of incrementally verifiable computation from random oracles. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 438–469. Springer, Cham, April 2023.
- [HJKS22] James Hulett, Ruta Jawale, Dakshita Khurana, and Akshayaram Srinivasan. SNARGs for P from sub-exponential DDH and QR. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 520–549. Springer, Cham, May / June 2022.

- [JKLM25] Zhengzhong Jin, Yael Tauman Kalai, Alex Lombardi, and Surya Mathialagan. Universal SNARGs for NP from proofs of correctness. To appear at STOC 2025, 2025.
- [JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd ACM STOC*, pages 60–73. ACM Press, June 2021.
- [JLS22] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0 . In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 670–699. Springer, Cham, May / June 2022.
- [KLVW23] Yael Kalai, Alex Lombardi, Vinod Vaikuntanathan, and Daniel Wichs. Boosting batch arguments and RAM delegation. In Barna Saha and Rocco A. Servedio, editors, *55th ACM STOC*, pages 1545–1552. ACM Press, June 2023.
- [KPTZ13] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. Delegatable pseudorandom functions and applications. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 669–684. ACM Press, November 2013.
- [KPY19] Yael Tauman Kalai, Omer Paneth, and Lisa Yang. How to delegate computations publicly. In Moses Charikar and Edith Cohen, editors, *51st ACM STOC*, pages 1115–1124. ACM Press, June 2019.
- [KS22] Abhiram Kothapalli and Srinath Setty. SuperNova: Proving universal machine executions without universal circuits. Cryptology ePrint Archive, Report 2022/1758, 2022.
- [KST22] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 359–388. Springer, Cham, August 2022.
- [KVZ21] Yael Tauman Kalai, Vinod Vaikuntanathan, and Rachel Yun Zhang. Somewhere statistical soundness, post-quantum security, and SNARGs. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part I*, volume 13042 of *LNCS*, pages 330–368. Springer, Cham, November 2021.
- [Mic93] Silvio Micali. Fair public-key cryptosystems. In Ernest F. Brickell, editor, *CRYPTO’92*, volume 740 of *LNCS*, pages 113–138. Springer, Berlin, Heidelberg, August 1993.
- [MPV24] Surya Mathialagan, Spencer Peters, and Vinod Vaikuntanathan. Adaptively sound zero-knowledge SNARKs for UP. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 38–71. Springer, Cham, August 2024.
- [NPR19] Moni Naor, Omer Paneth, and Guy N. Rothblum. Incrementally verifiable computation via incremental PCPs. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part II*, volume 11892 of *LNCS*, pages 552–576. Springer, Cham, December 2019.
- [NSK04] Lan Nguyen, Reihaneh Safavi-Naini, and Kaoru Kurosawa. Verifiable shuffles: A formal model and a Paillier-based efficient construction with provable security. In

- Markus Jakobsson, Moti Yung, and Jianying Zhou, editors, *ACNS 04 International Conference on Applied Cryptography and Network Security*, volume 3089 of *LNCS*, pages 61–75. Springer, Berlin, Heidelberg, June 2004.
- [NT16] Assa Naveh and Eran Tromer. PhotoProof: Cryptographic image authentication for any set of permissible transformations. In *2016 IEEE Symposium on Security and Privacy*, pages 255–271. IEEE Computer Society Press, May 2016.
 - [PP22] Omer Paneth and Rafael Pass. Incrementally verifiable computation via rate-1 batch arguments. In *63rd FOCS*, pages 1045–1056. IEEE Computer Society Press, October / November 2022.
 - [PS19] Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part I*, volume 11692 of *LNCS*, pages 89–114. Springer, Cham, August 2019.
 - [SK95] Kazue Sako and Joe Kilian. Receipt-free mix-type voting scheme - a practical solution to the implementation of a voting booth. In Louis C. Guillou and Jean-Jacques Quisquater, editors, *EUROCRYPT’95*, volume 921 of *LNCS*, pages 393–403. Springer, Berlin, Heidelberg, May 1995.
 - [SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th ACM STOC*, pages 475–484. ACM Press, May / June 2014.
 - [Val08] Paul Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, March 2008.
 - [WW22] Brent Waters and David J. Wu. Batch arguments for NP and more from standard bilinear group assumptions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 433–463. Springer, Cham, August 2022.
 - [WW24a] Brent Waters and David J. Wu. Adaptively-sound succinct arguments for NP from indistinguishability obfuscation. In Bojan Mohar, Igor Shinkar, and Ryan O’Donnell, editors, *56th ACM STOC*, pages 387–398. ACM Press, June 2024.
 - [WW24b] Brent Waters and David J. Wu. Adaptively-sound succinct arguments for NP from indistinguishability obfuscation. Cryptology ePrint Archive, Report 2024/165, 2024.
 - [WW24c] Brent Waters and David J. Wu. A pure indistinguishability obfuscation approach to adaptively-sound SNARGs for NP. To appear at Crypto 2025. Cryptology ePrint Archive, Paper 2024/933, 2024.
 - [WW24d] Brent Waters and David J. Wu. A pure indistinguishability obfuscation approach to adaptively-sound SNARGs for NP. Cryptology ePrint Archive, Report 2024/933, 2024.
 - [WZ24] Brent Waters and Mark Zhandry. Adaptive security in SNARGs via iO and lossy functions. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 72–104. Springer, Cham, August 2024.